Assignment 6: AWS advance networkingwith VPC and Subnets

Phạm Trần Gia Hưng - 2331200153

Overview

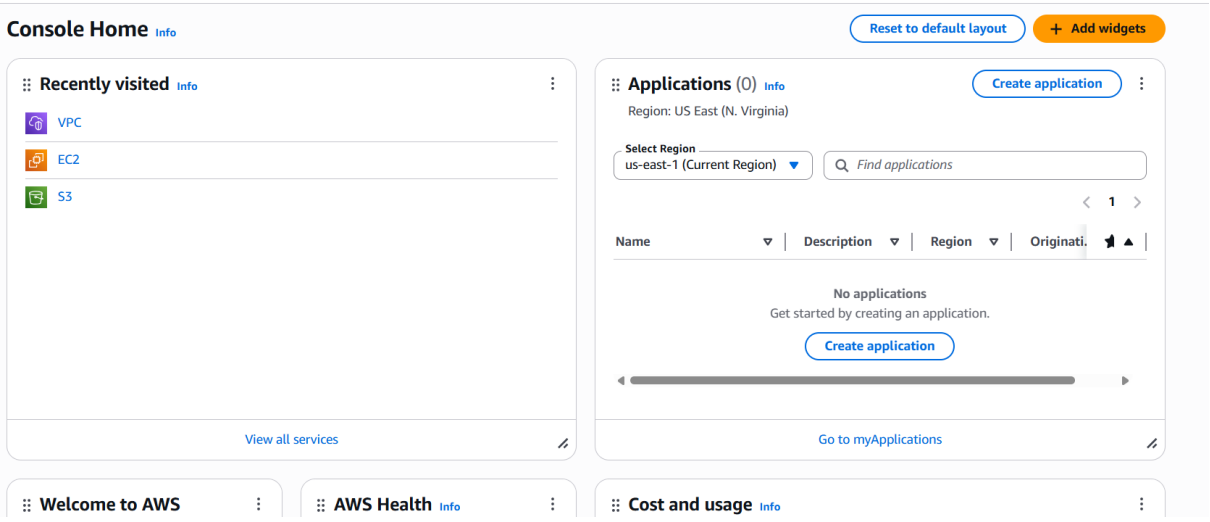
This assignment provides a hands-on experience in designing and implementing a secure, highlyavailable, and scalable network infrastructure using Amazon Virtual Private Cloud (VPC). Youwill build a multi-tier, multi-AZ network from the ground up, configure routing for publicandprivate subnets, implement layered security controls, and establish private connectivity betweenVPCs and to AWS services. The objective is to develop a deep, practical understandingof coreAWS networking components and best practices.

6. Clear explanation of each following completed task:

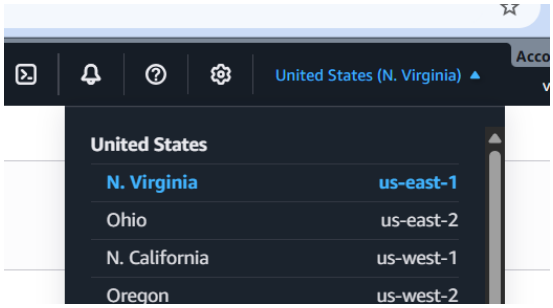
6.1. Task 1: Create a Custom VPC with Public and Private Subnets: Understand howtocreatean isolated VPC network and define public and private subnets within it.

- Instructions:

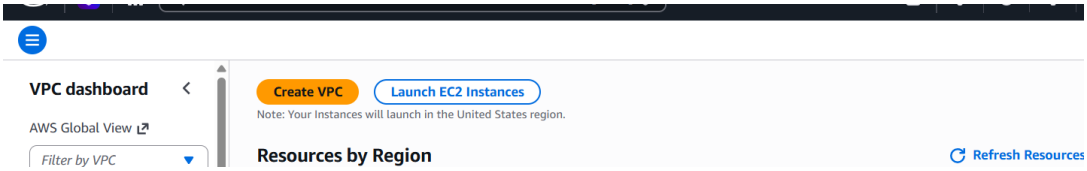
☑ Navigate to the VPC service in the AWS Management Console.



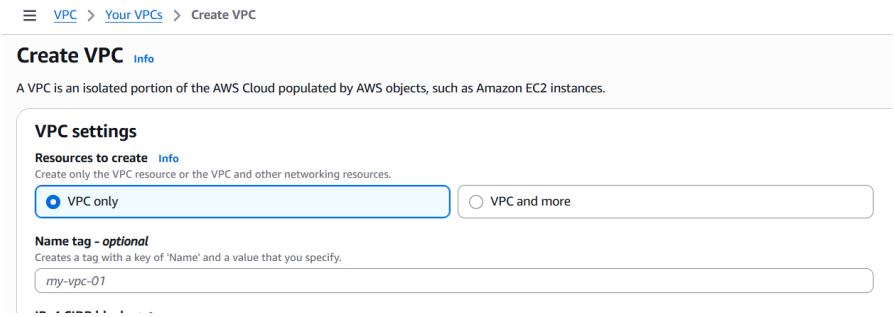
☑ In the top-right corner, ensure you are in your desired region (e.g., N. Virginia (us-east-1)).



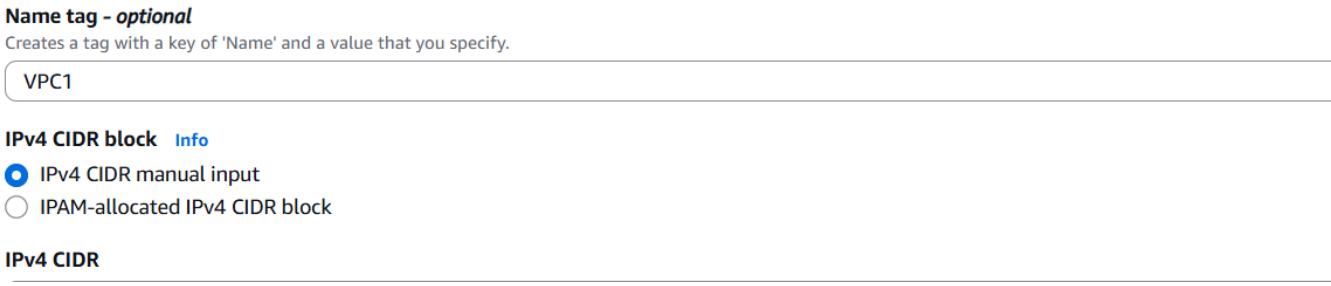
☑ Click Create VPC.



☑ Select VPC only.

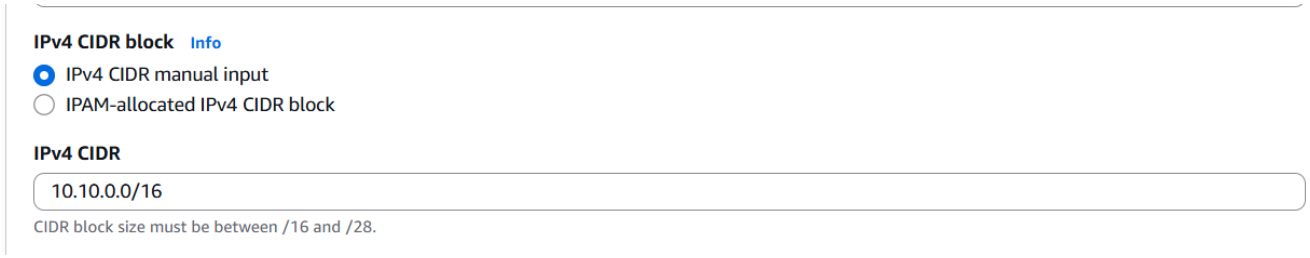


☑ Under Name tag, enter

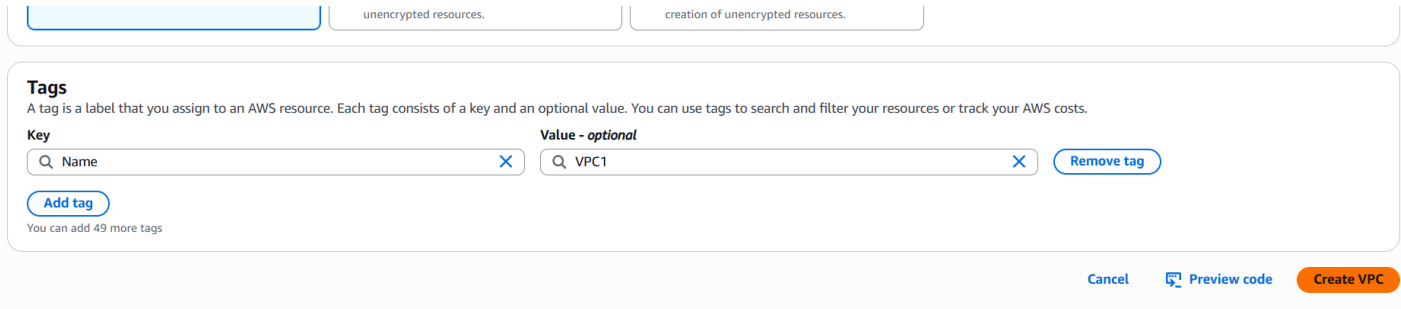


.

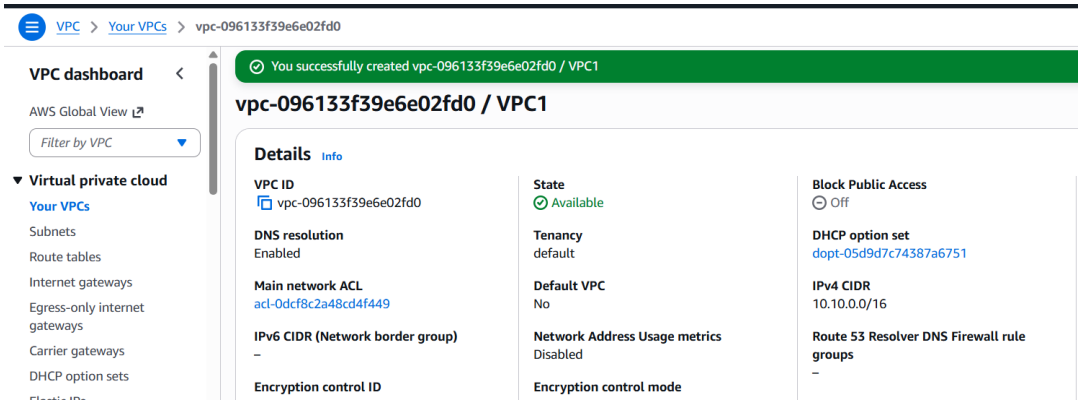
☑ For IPv4 CIDR block, enter `10.10.0.0/16`.

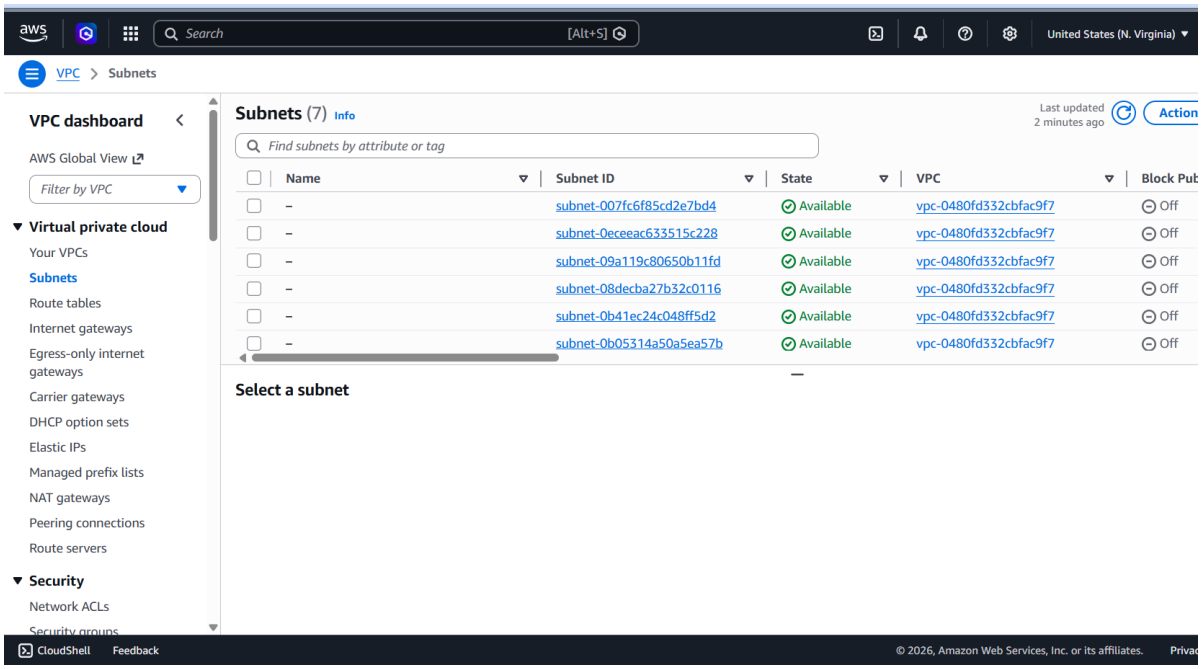


☑ Leave all other settings as default and click Create VPC.

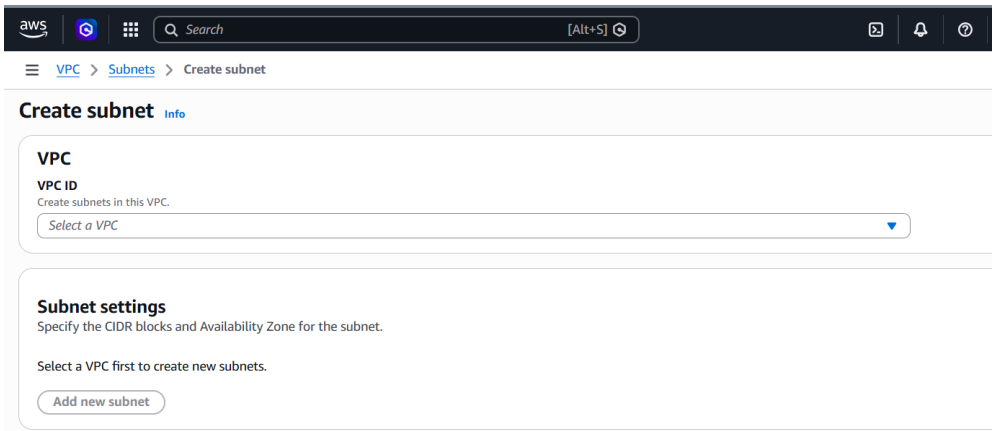


☑ Once the VPC is created, go to Subnets in the left-hand menu.

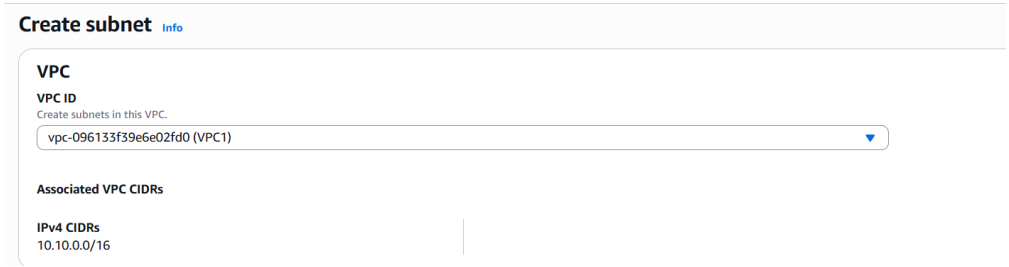




☑ Click Create subnet.

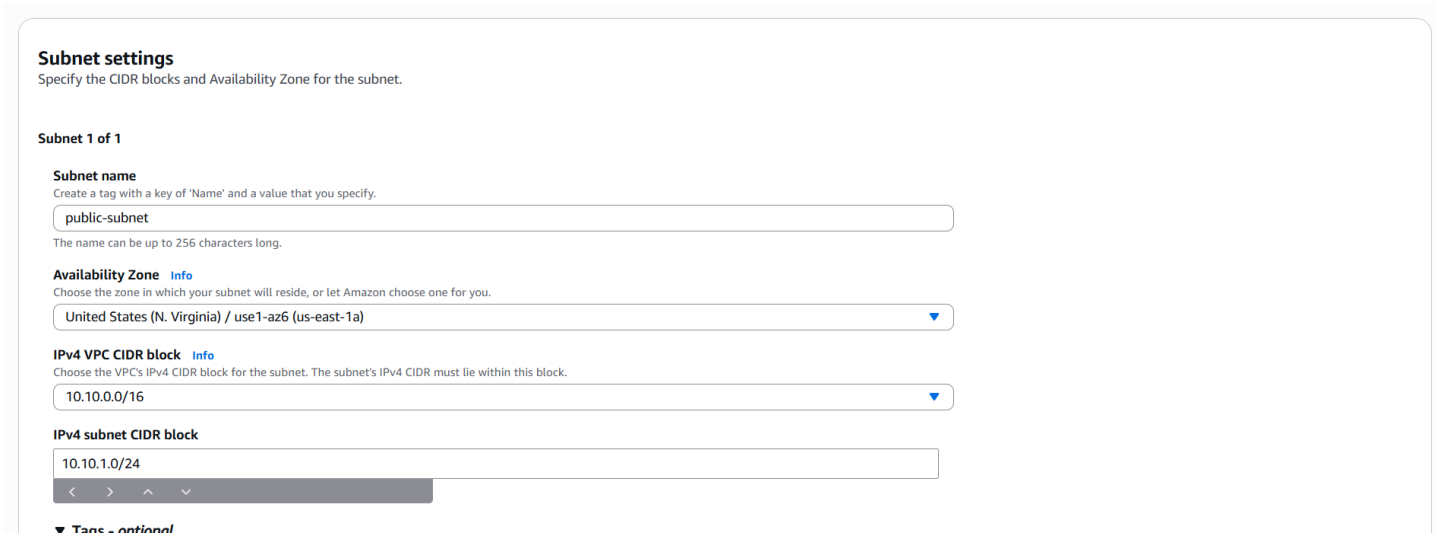


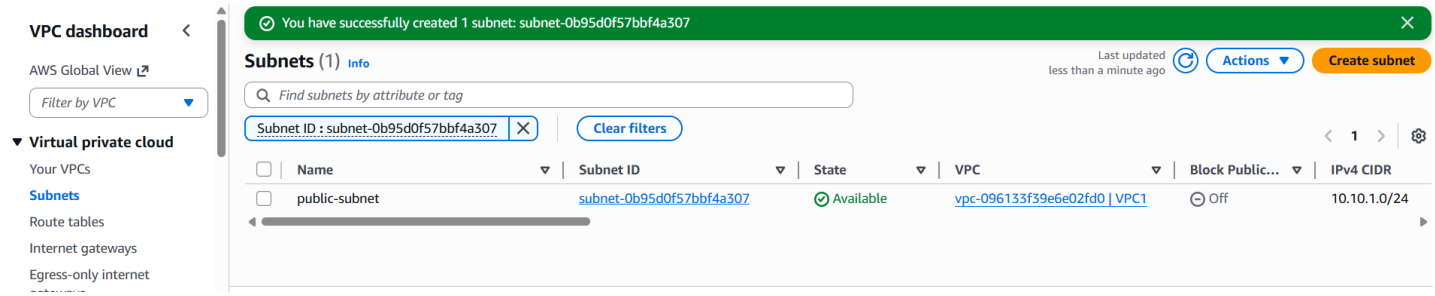
☑ Select VPC1 from the VPC ID dropdown.



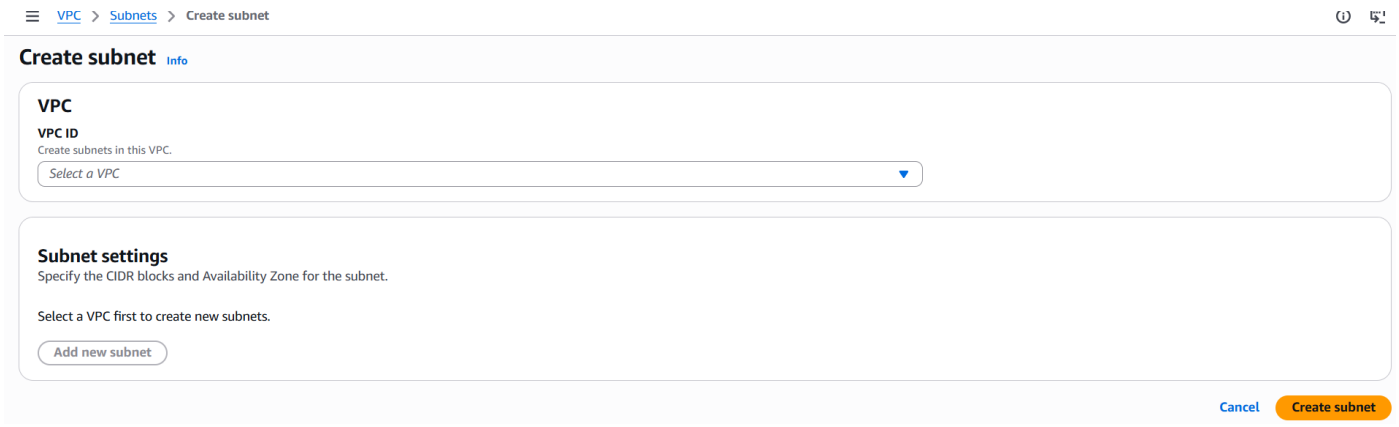
☑ Define your public subnet:

- ☐ Subnet name: public-subnet
- ☐ Availability Zone: Choose any (e.g., us-east-1a).
- ☐ IPv4 CIDR block: `10.10.1.0/24`



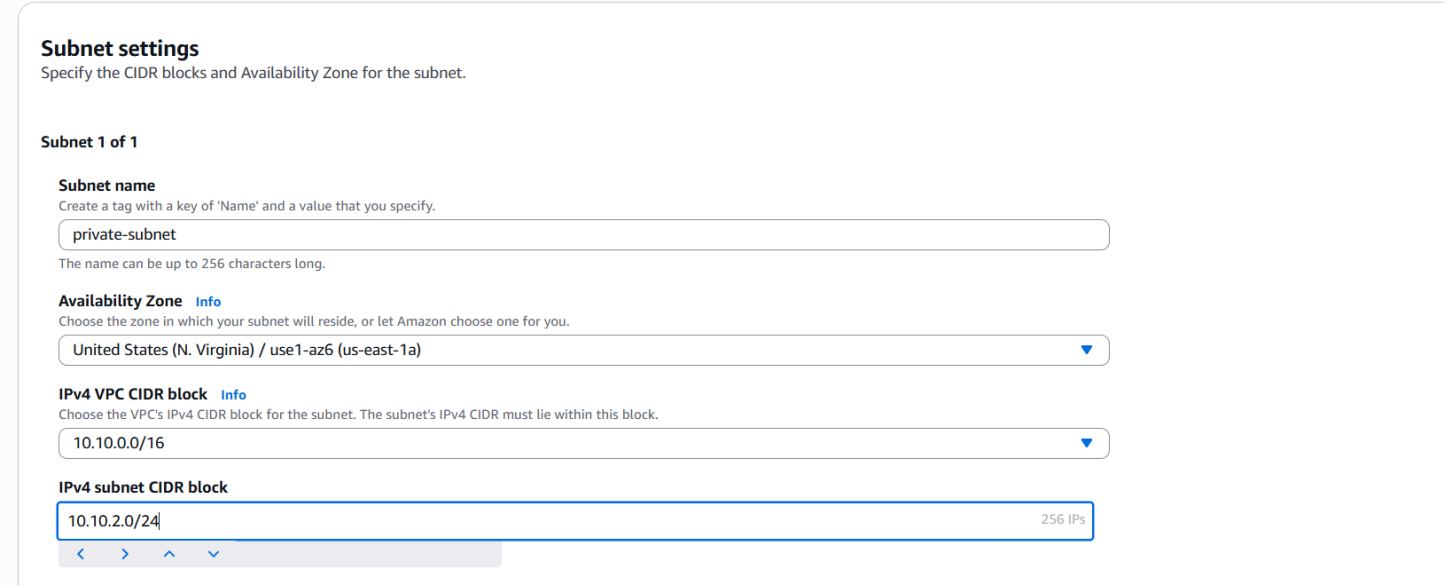


☑ Click Add new subnet to add a second subnet configuration in the same window.

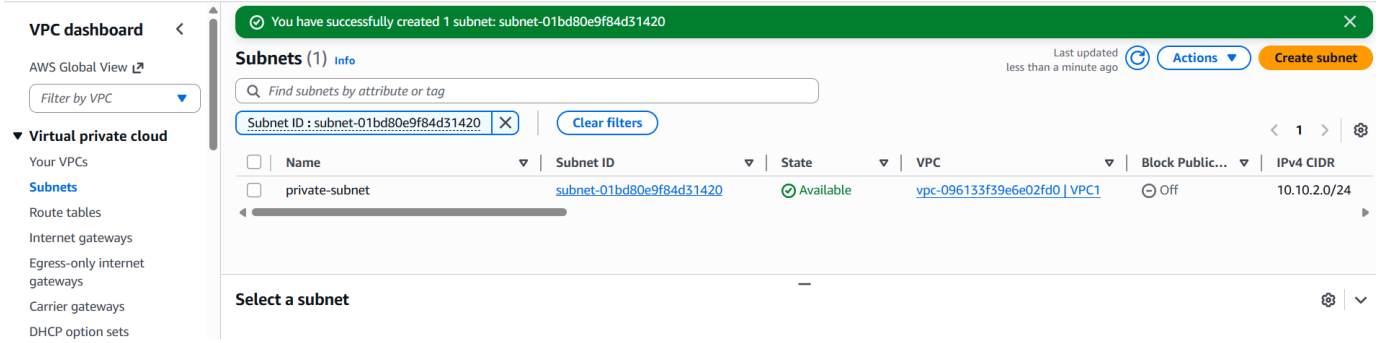


☑ Define your private subnet:

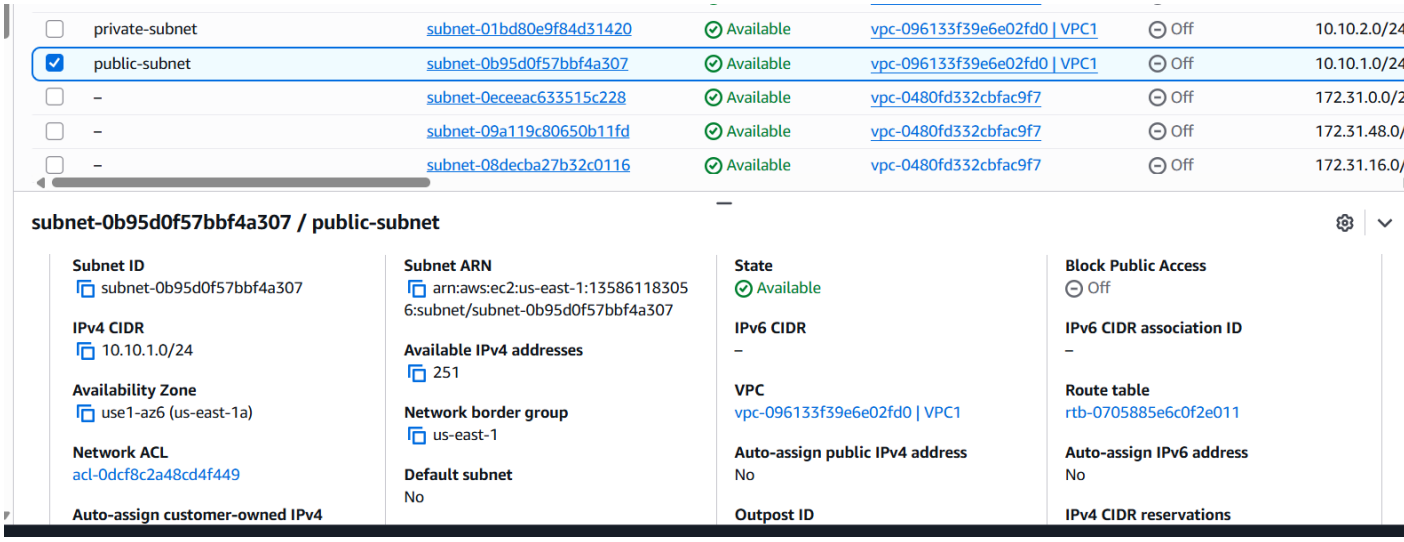
- ☐ Subnet name: private-subnet
- ☐ Availability Zone: Choose the same one as your public subnet (e.g., us-east-1a). ☐ IPv4 CIDR block: `10.10.2.0/24`



☑ . Click Cr eate subnet at the bottom of the page.



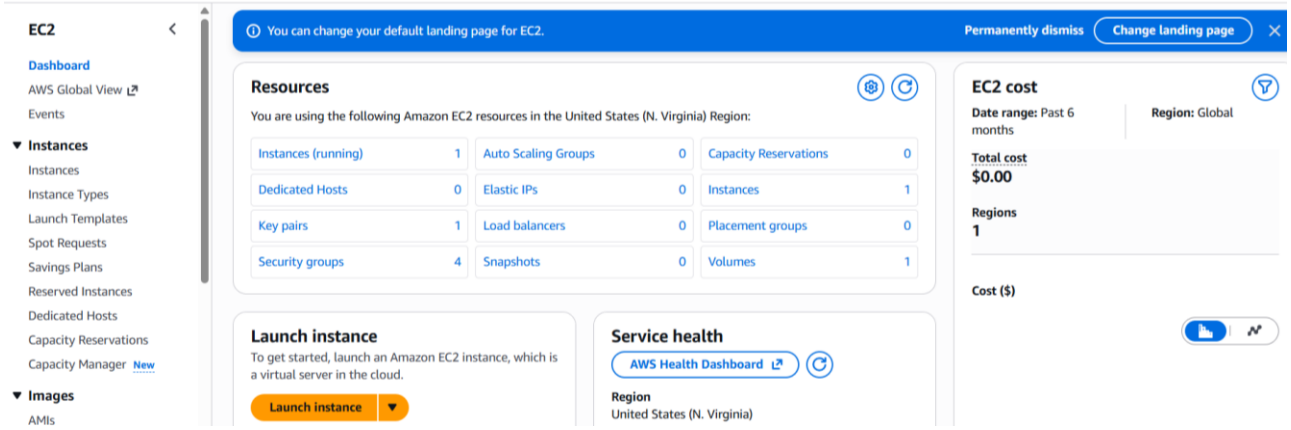
☑ . Select your public-subnet. In the Details tab, observe the Available IPv4 addr esses. It should be 251 (256 total IPs minus 5 reserved by AWS), confirming the address allocation.



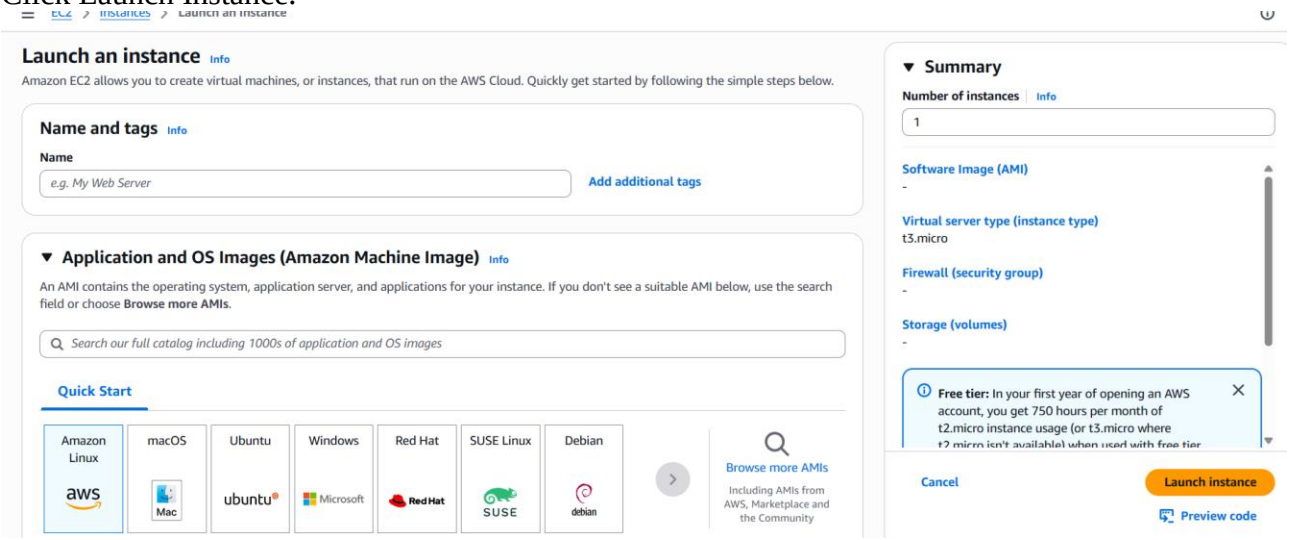
6.2. **Task 2: Launch EC2 Instances in Each Subnet:** Deploy public and private compute resources within your custom VPC.

- Instructions:

Navigate to the EC2 service.

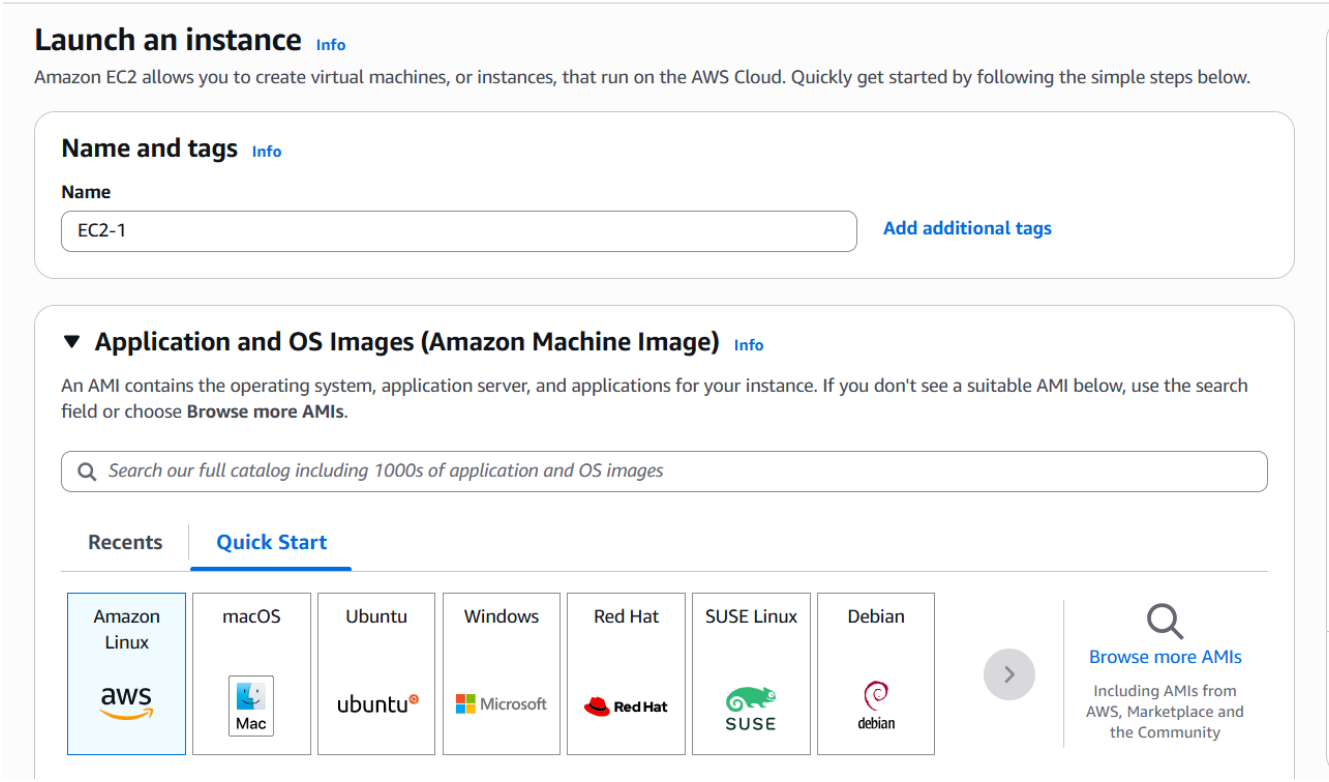


Click Launch Instance.

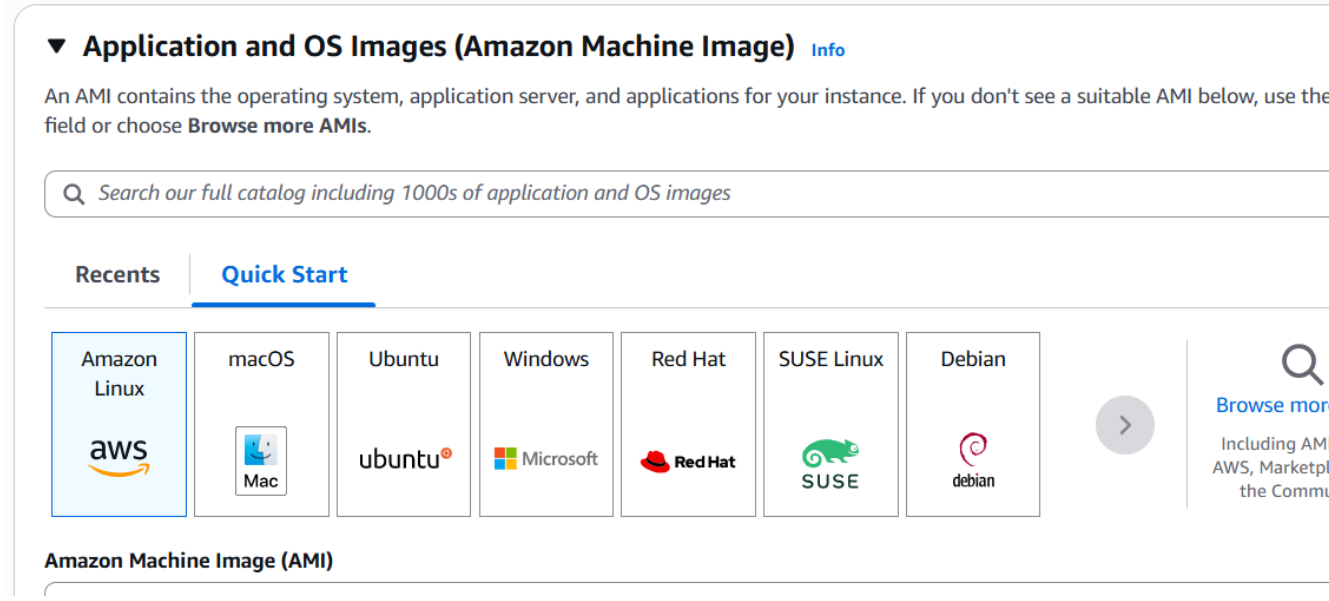


Launch EC2-1 (Public Instance):

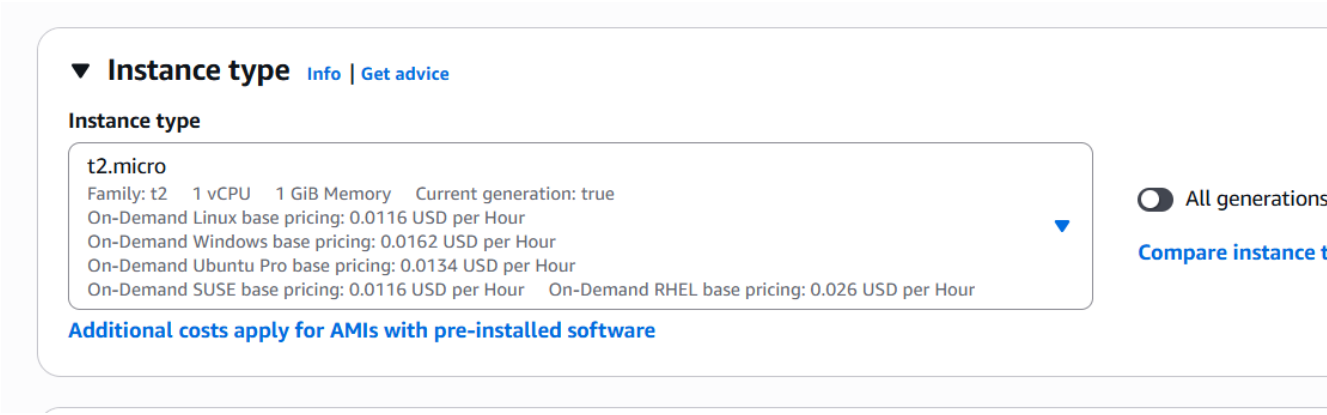
- Name: `EC2-1`



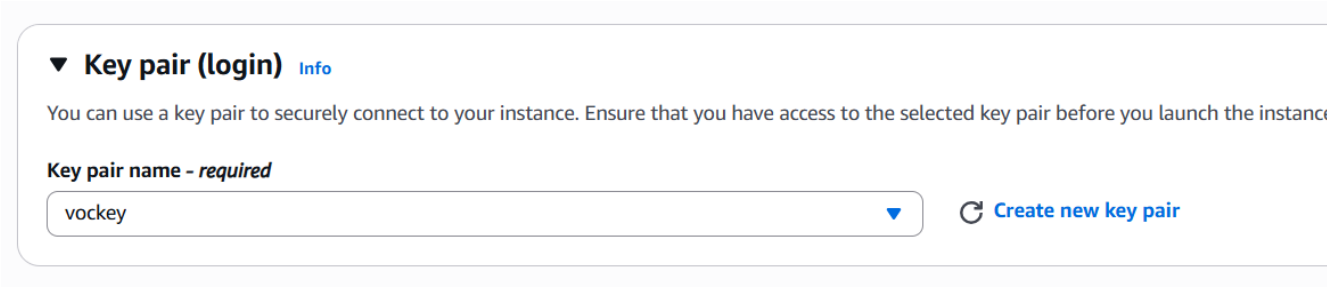
◆ Application and OS Images (AMI): Select Amazon Linux 2 AMI (Free tier eligible).



◆ Instance type: Select t2.micro (Free tier eligible).

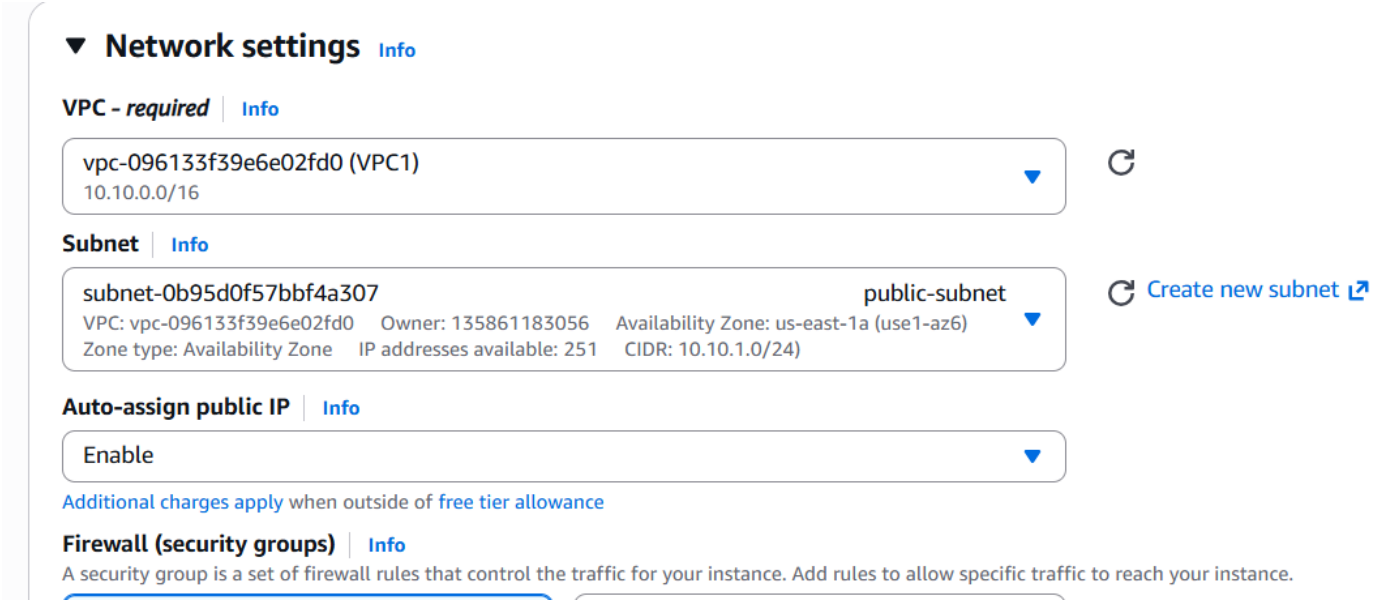


◆ Key pair (login): select an existing key pair.



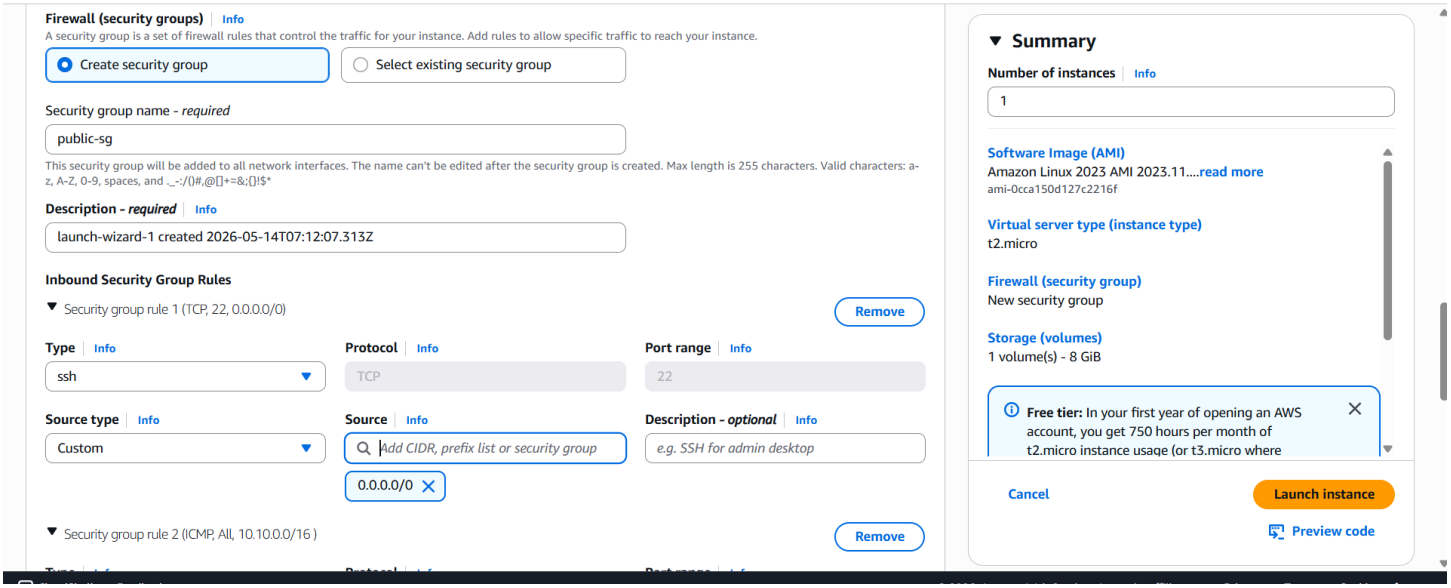
◆ Network settings: Click Edit.

- VPC: Select VPC1.
- Subnet: Select public-subnet.
- Auto-assign public IP: Select Enable.

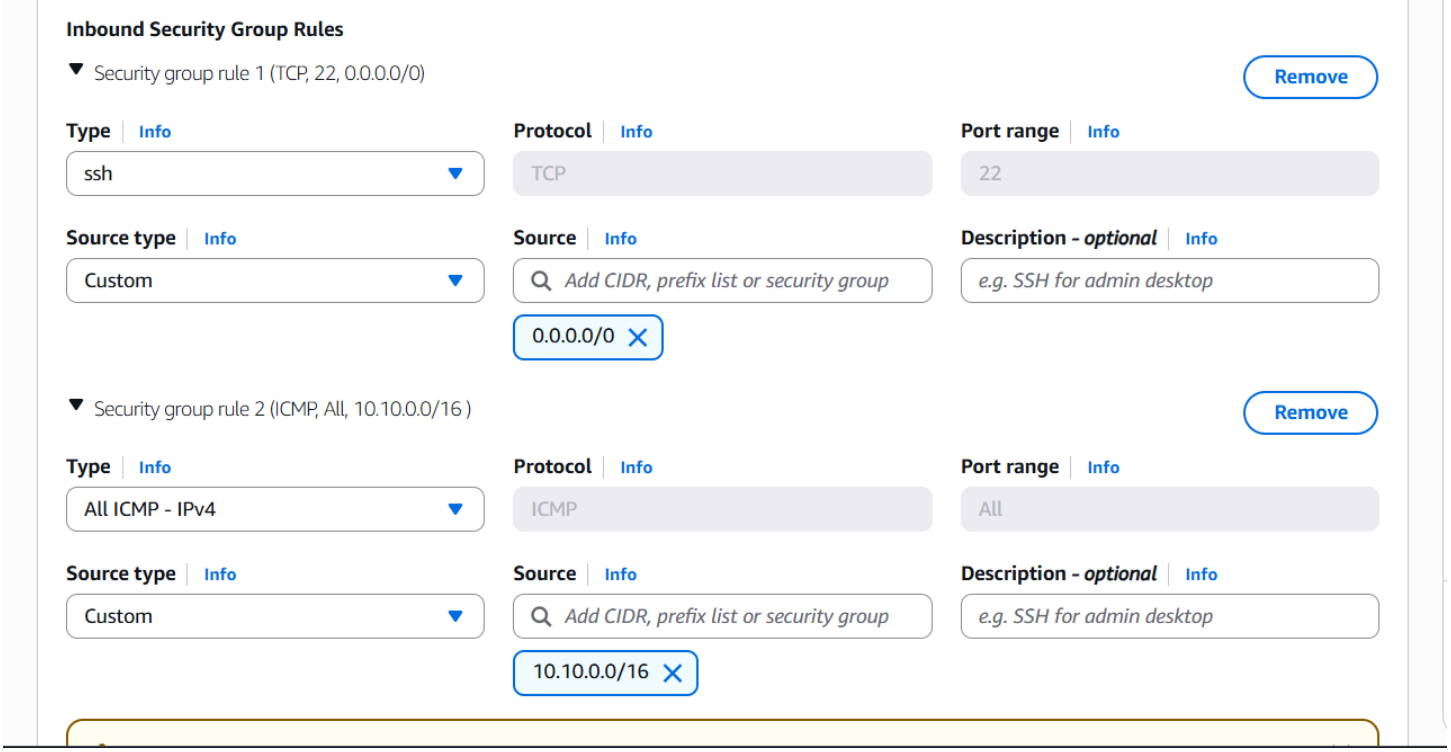


◆ Fir ewall (security groups):

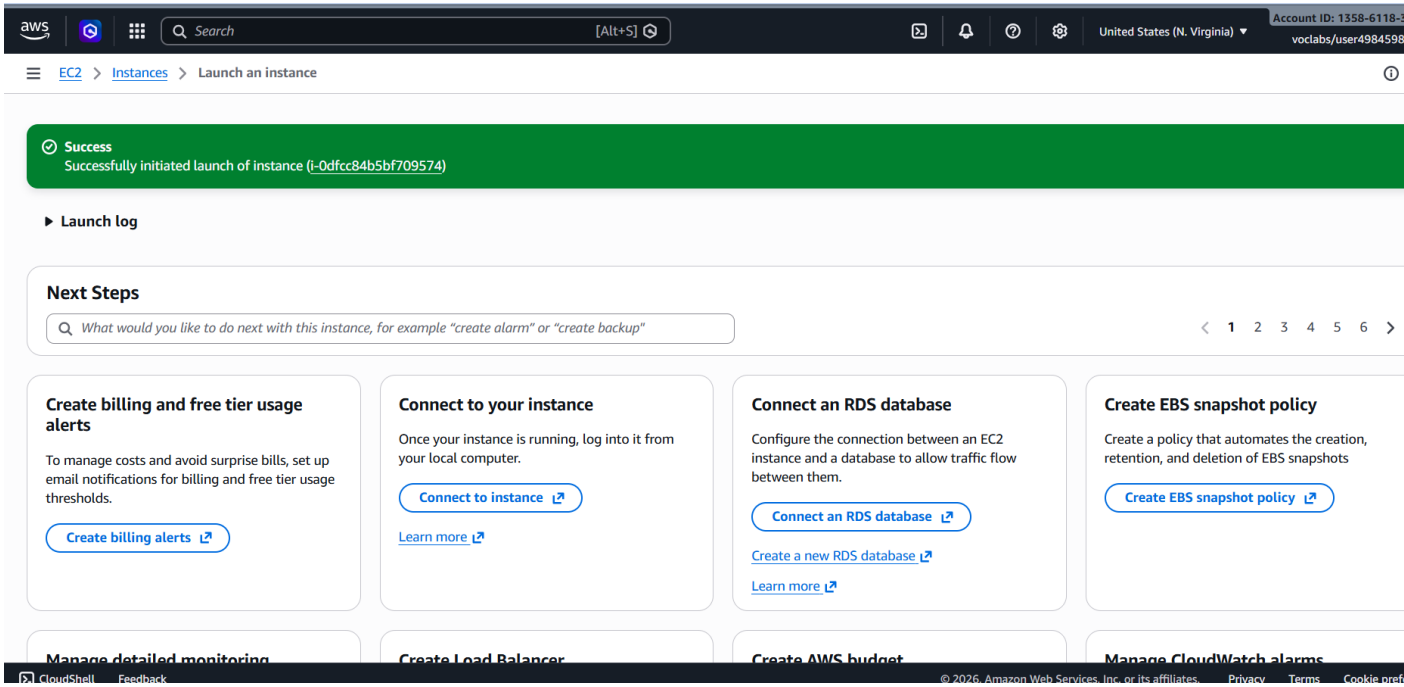
- Select Cr eate a new security group.
- Security group name: `public-sg`
- Description: Allows SSH and ICMP



- Inbound rules:
  - Rule 1: Type SSH, Source My IP (This is more secure. If your IPchanges, use `0.0.0.0/0`, but be aware this is less secure).
  - Rule 2: Click Add security group rule. Type All ICMP - IPv4, SourceCustom `10.10.0.0/16` (This allows ping from anywhere inside VPC1).



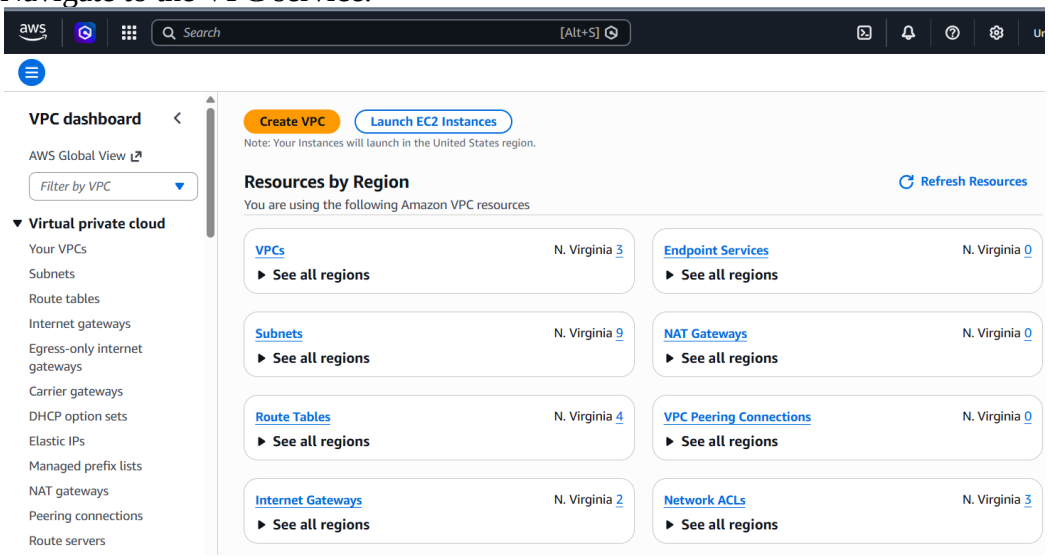
. Click Launch instance.



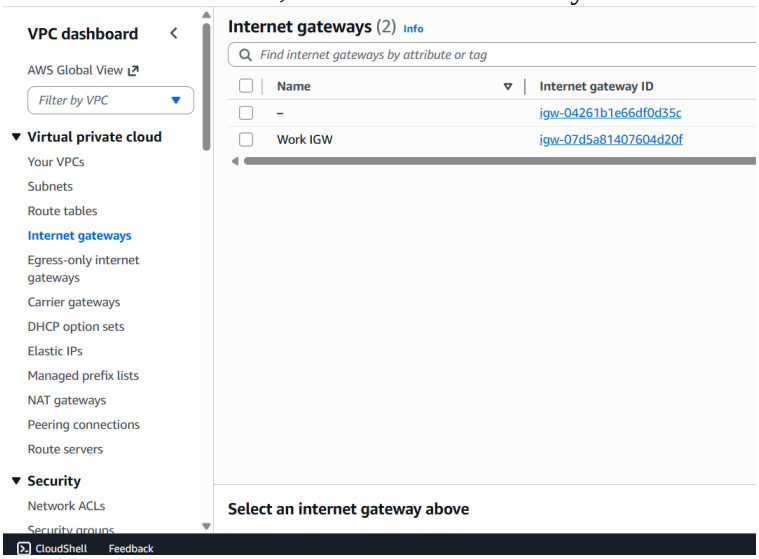
6.3. Task 3: Enable Internet Access for Public Subnet: Connect your VPC to the Internet usingan Internet Gateway (IGW) to allow access for public workloads.

- **Instructions:**

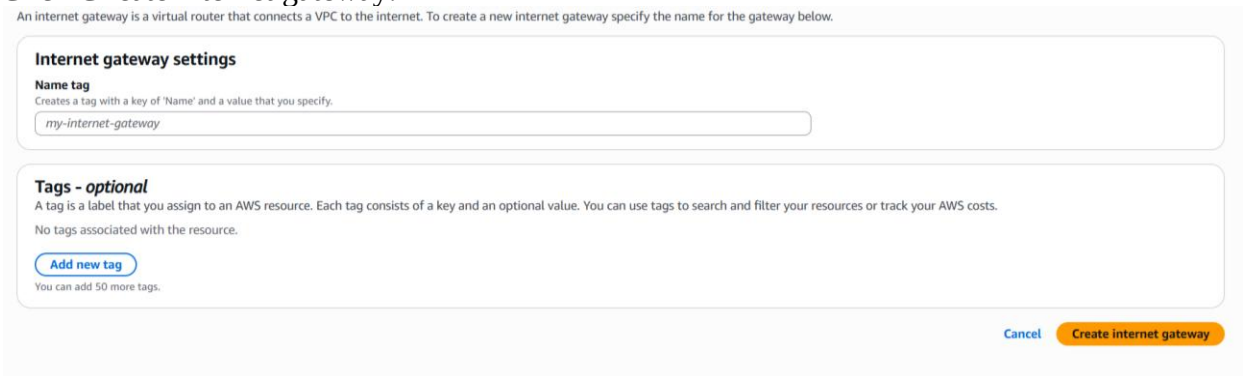
Navigate to the VPC service.



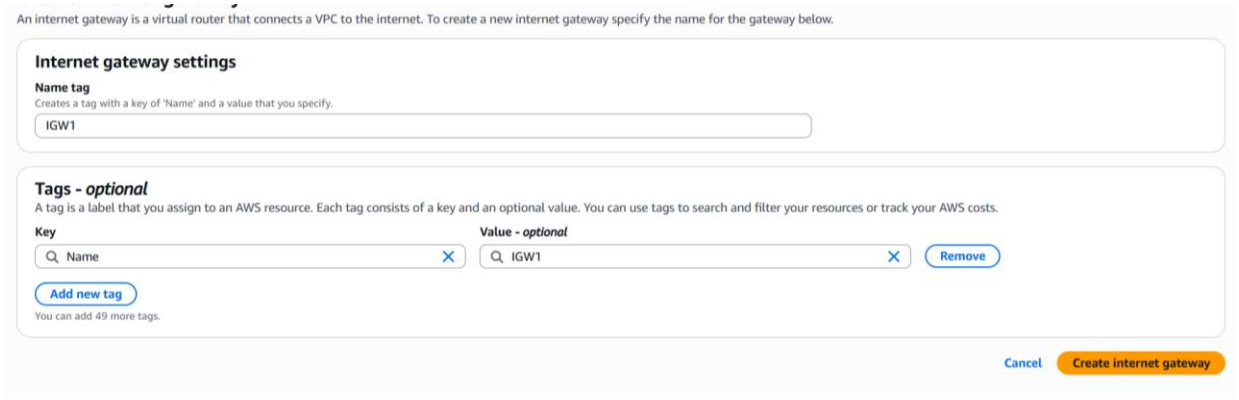
In the left-hand menu, click Internet Gateways.

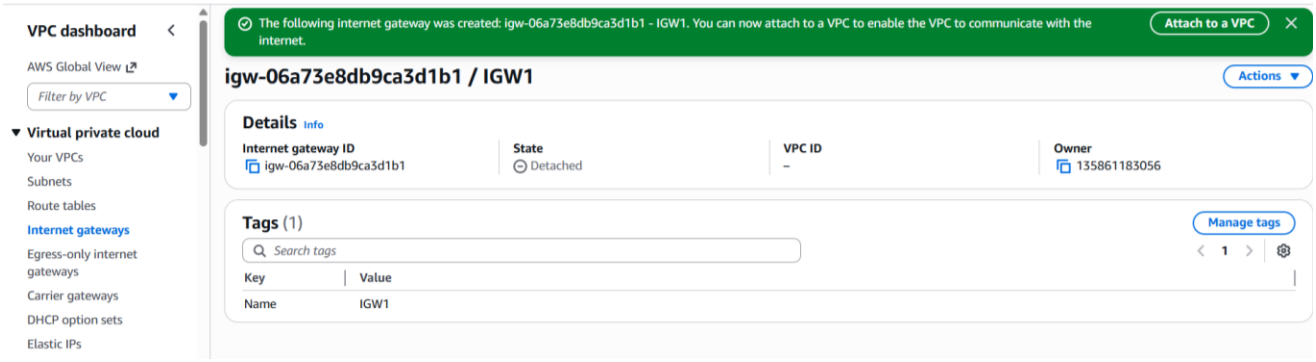


Click Cr eate internet gateway.

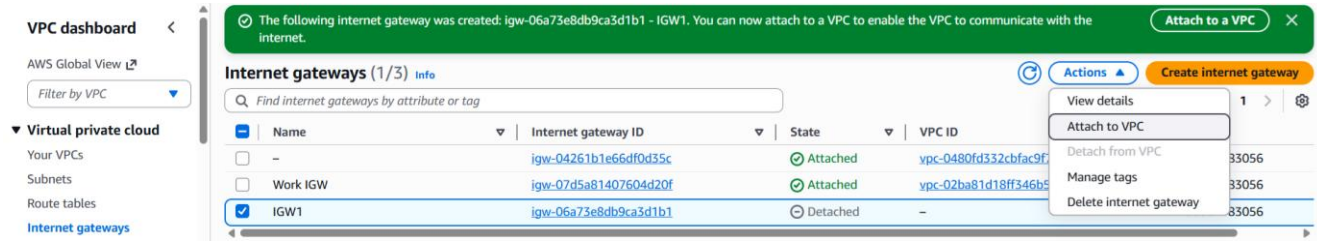


Name tag: IGW1. Click Cr eate internet gateway.

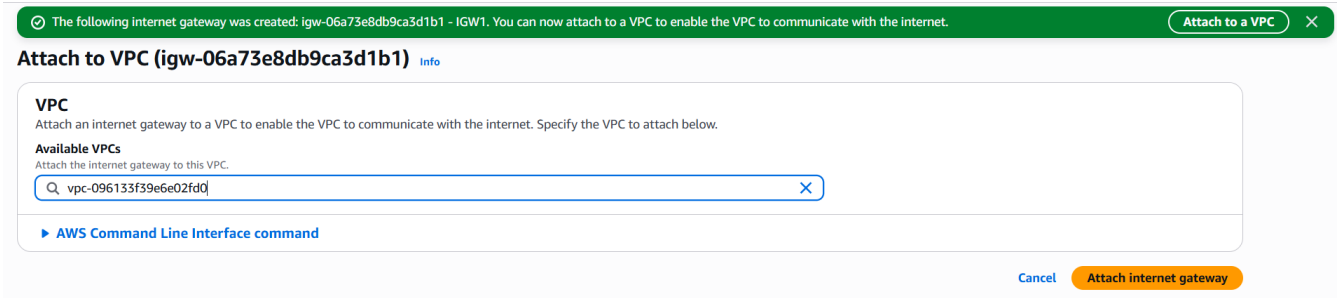


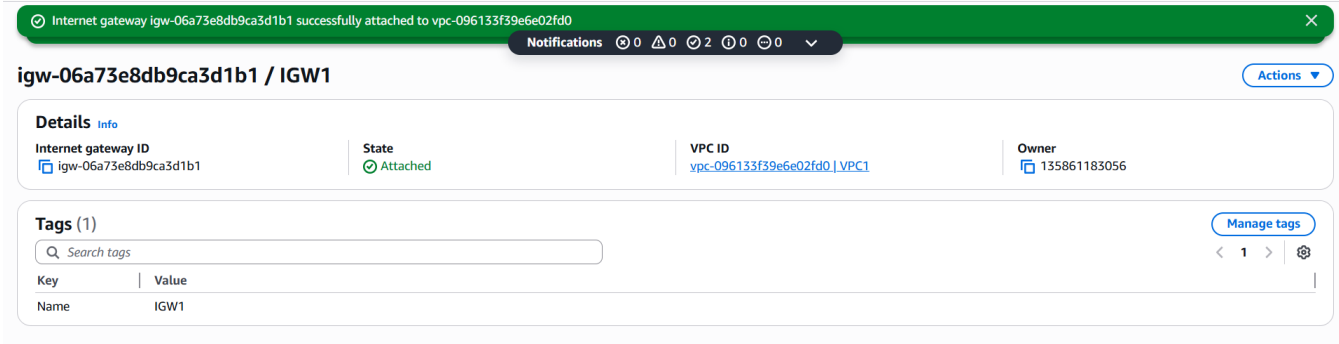


Select IGW1 from the list. Click Actions -> Attach to VPC.

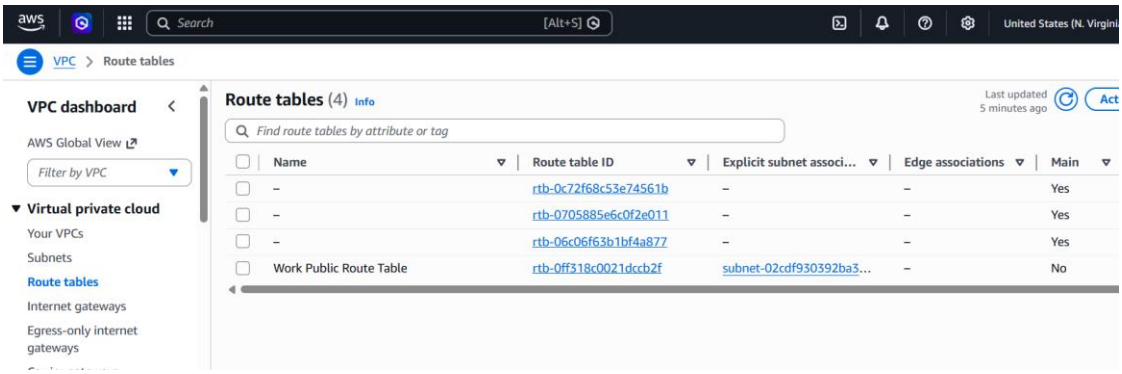


Select VPC1 from the dropdown and click Attach internet gateway.

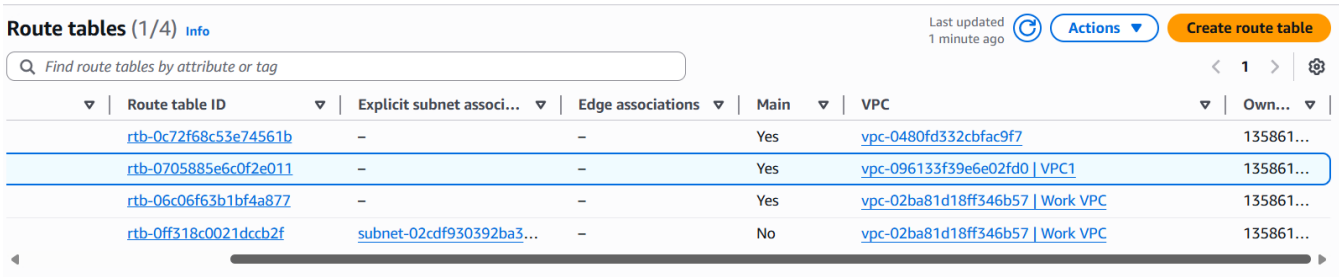




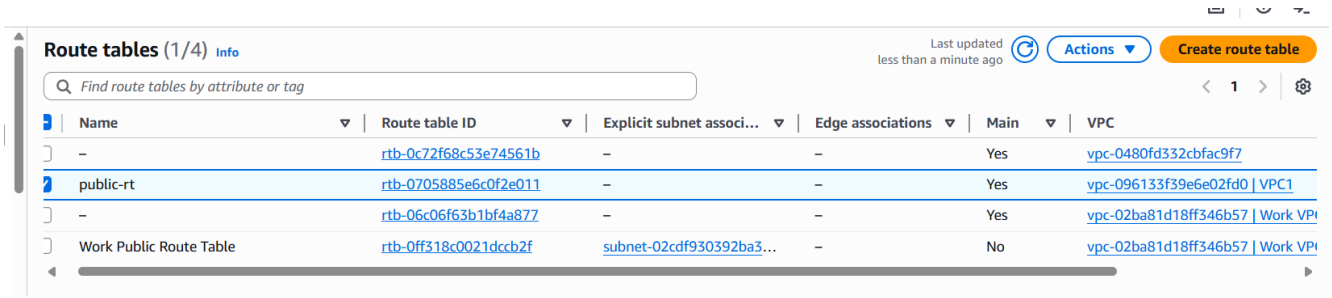
In the left-hand menu, click Route Tables.



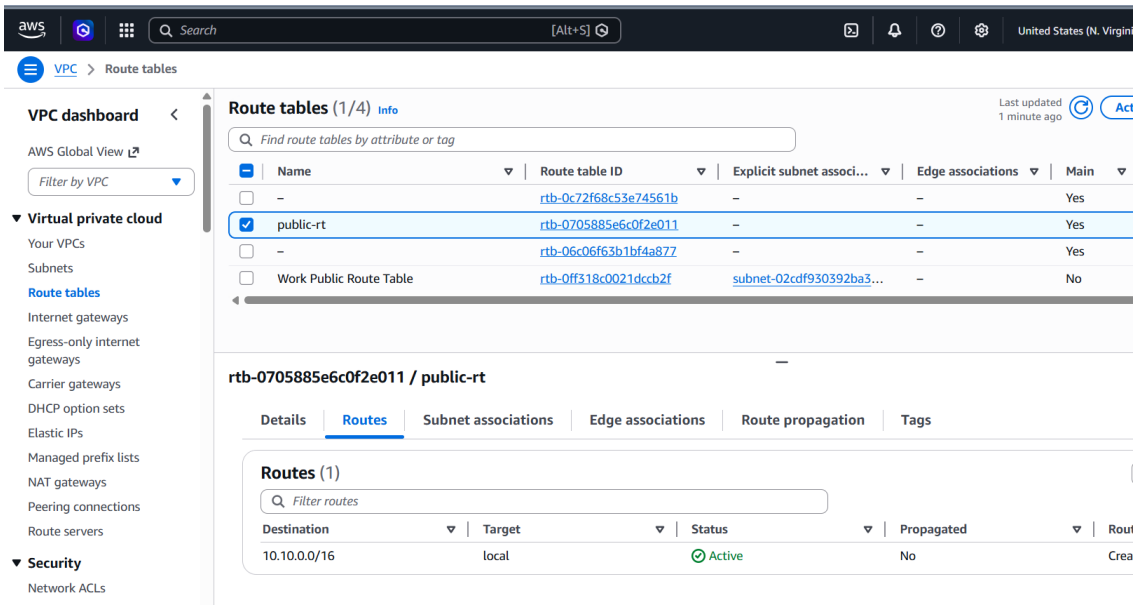
You will see a default route table for VPC1. Select it.



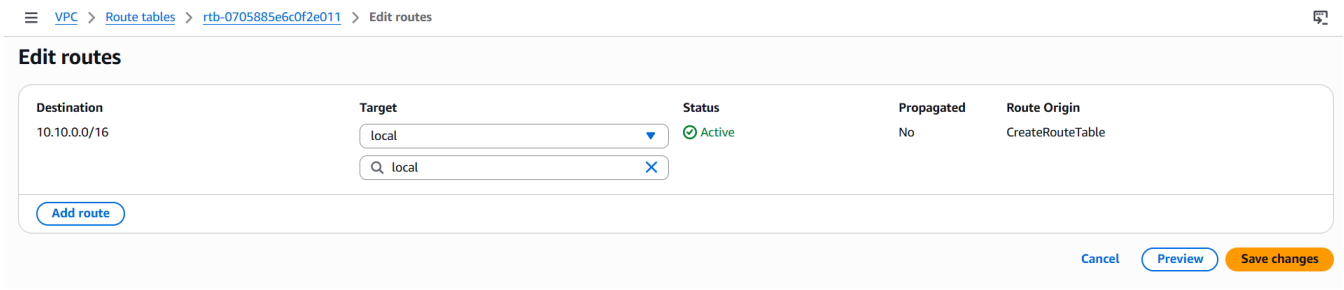
Name it `public-rt` for clarity.



With public-rt selected, click the Routes tab below.

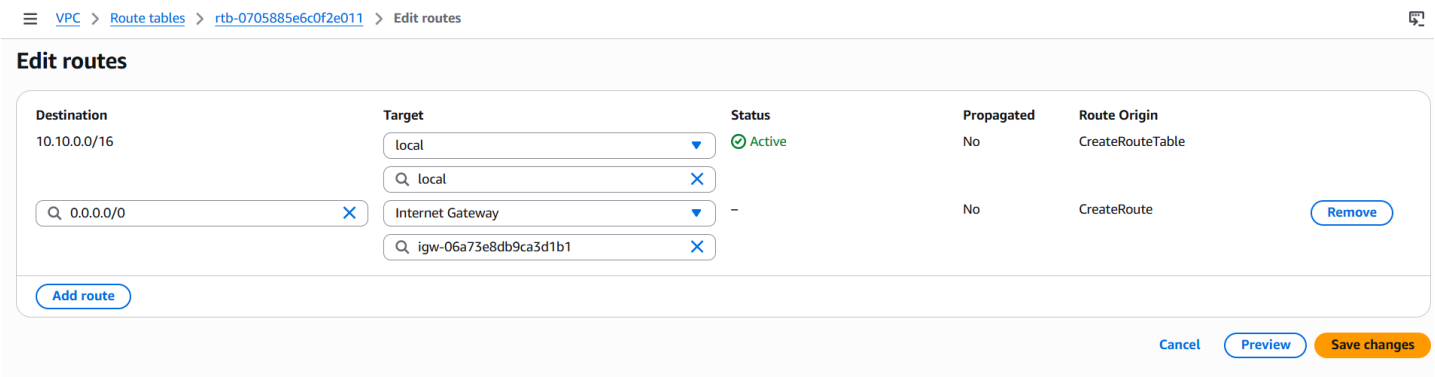


Click Edit routes.

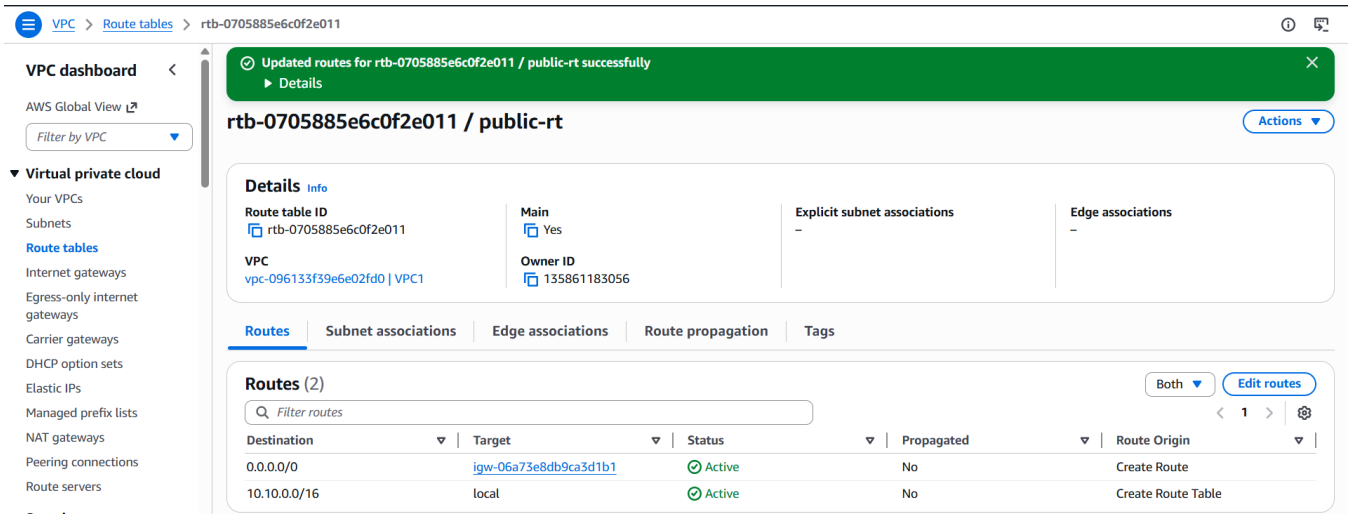


Click Add route.

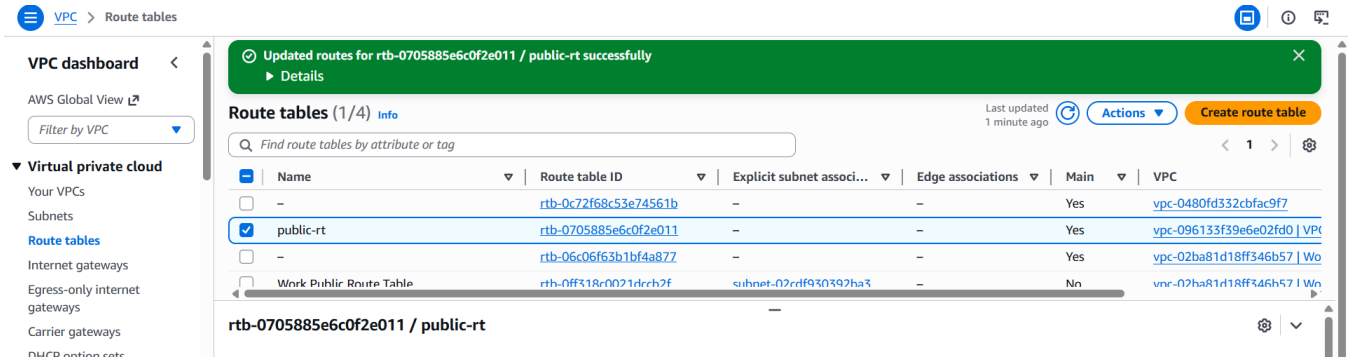
- Destination: `0.0.0.0/0`
- Target: Select Internet Gateway and then choose IGW1.



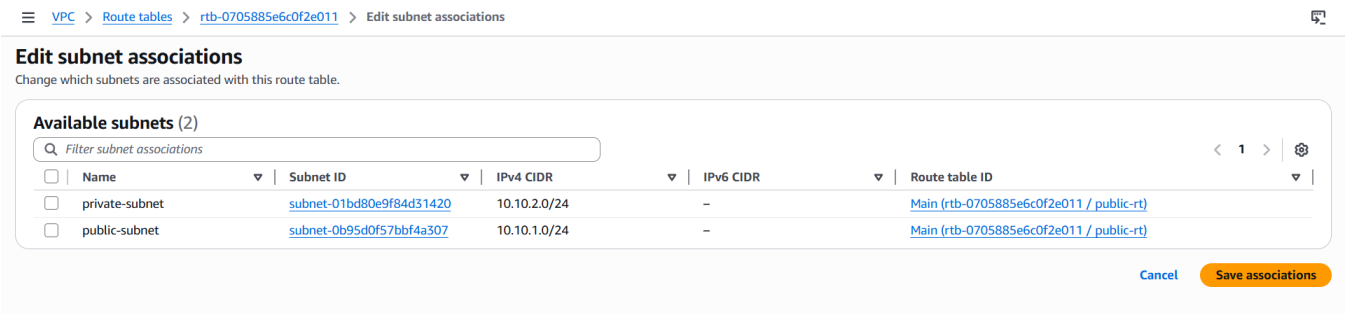
Click Save changes.



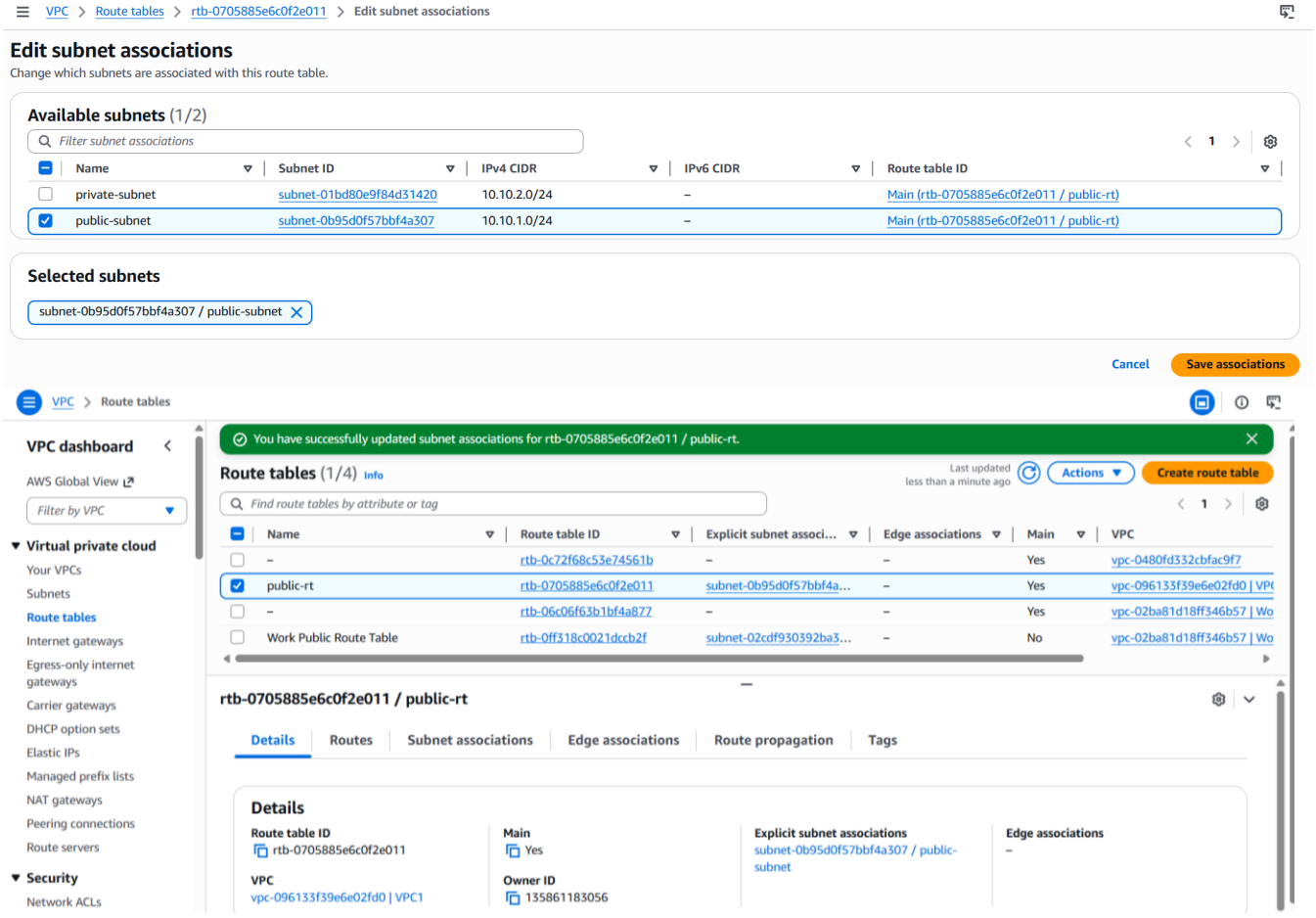
Now, you must associate this route table with your public subnet. Click the Subnet Associations tab.



Click Edit subnet associations.

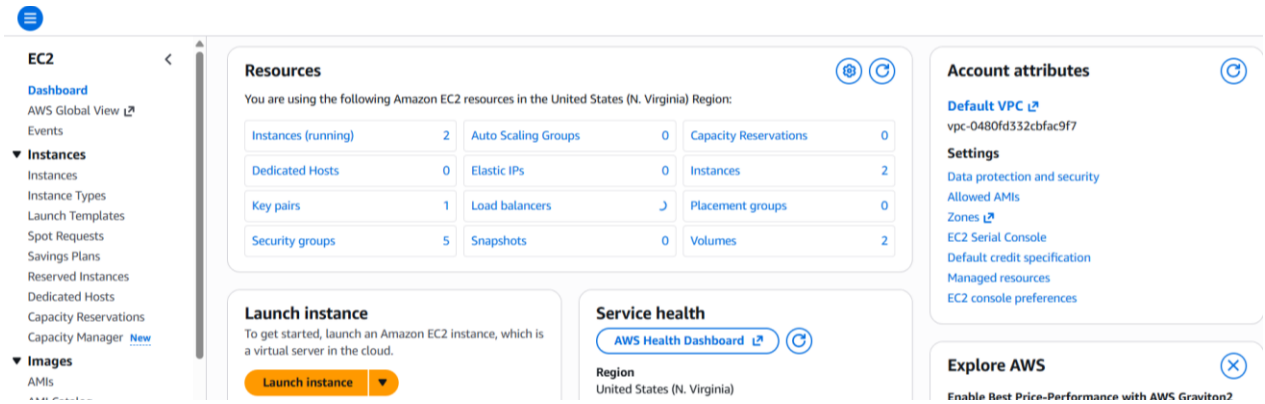


Check the box for public-subnet and click Save associations.

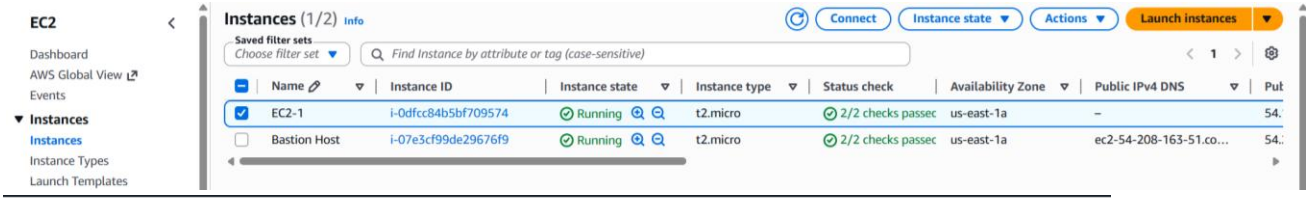


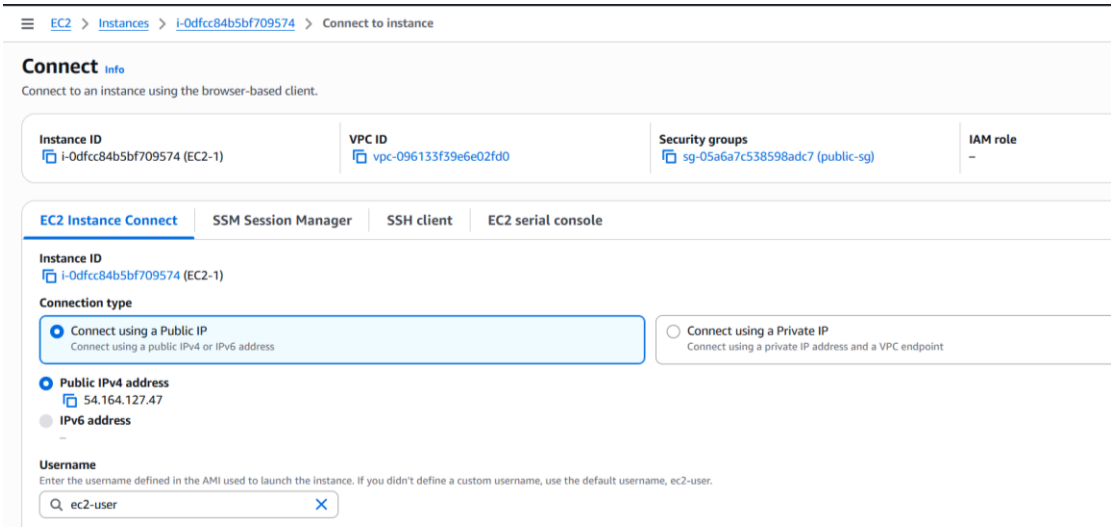
Verify Internet Access (EC2-1 via browser)

- Navigate to EC2 service.

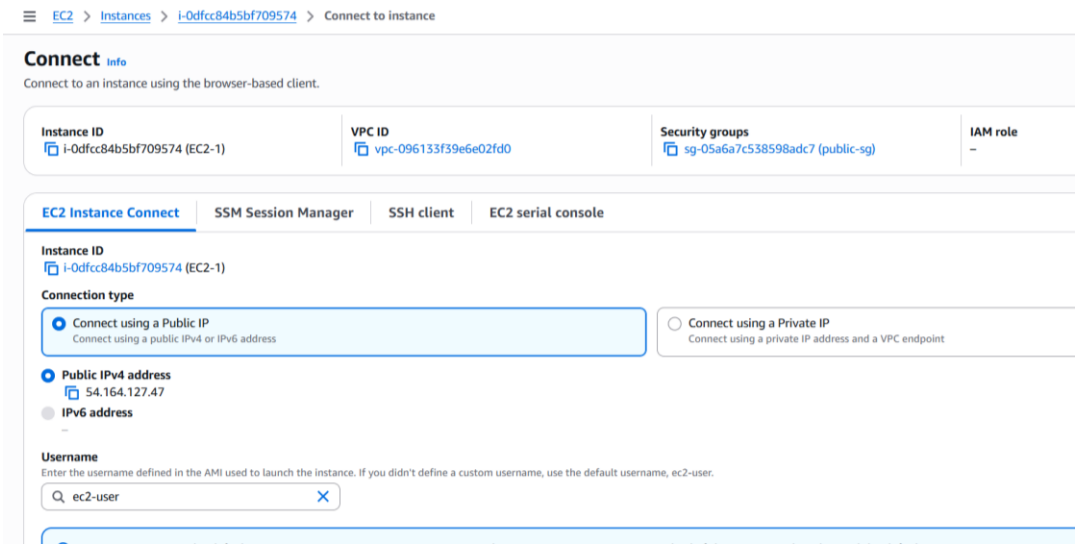


- Select EC2-1 instance. Then click on connect.

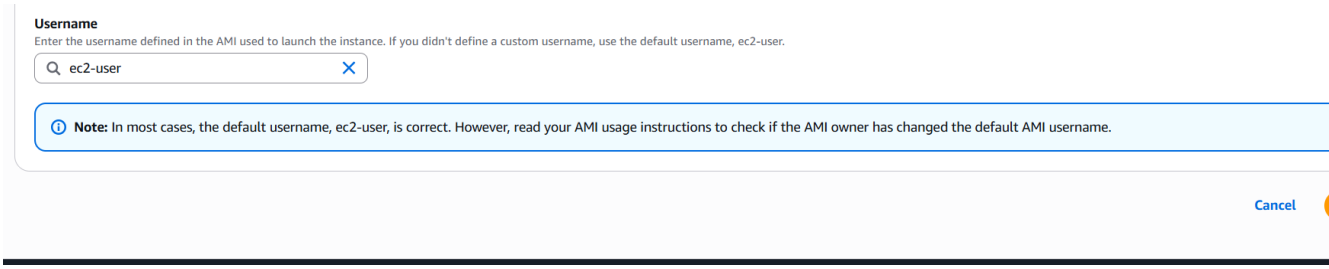




- Select EC2 Instance Connect option



- Then click on Connect.



- Wait for a minute AWS SSH client terminal of EC2-1 will be open.



- Test the connection via command ping `8.8.8.8`

```
        _/  _/
      _/m/'
[ec2-user@ip-10-10-1-124 ~]$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=1.80 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=1.94 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=1.82 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=1.76 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=1.77 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=1.90 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=117 time=3.21 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=117 time=1.78 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=117 time=1.81 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=117 time=1.78 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=117 time=1.85 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=117 time=1.92 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=117 time=1.90 ms
```

- After this, generate a key from EC2-1 terminal by entering command: `ssh-keygen -t rsa -f ~/.ssh/mykey`

```
[ec2-user@ip-10-10-1-124 ~]$ ssh-keygen -t rsa -f ~/.ssh/mykey
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Passphrases do not match.  Try again.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ec2-user/.ssh/mykey
Your public key has been saved in /home/ec2-user/.ssh/mykey.pub
The key fingerprint is:
SHA256:d6//tWL7itbMqsblxW7LG8p07lFEUx9i6Rb4QEULgKM ec2-user@ip-10-10-1-124.ec2.internal
The key's randomart image is:
+---[RSA 3072]----+
|        ..oo+=.+o|
|       o   ooo+ +|
|      . .   +....|
|     E       +.  |
|        S . +  . |
|         . o +.  |
|        . o.*+. .|
|         ooo*Xo o|
|        ..o=*XO+.|
+----[SHA256]-----+
```

- See the key via command: **ls -l ~/.ssh**

```
[ec2-user@ip-10-10-1-124 ~]$ ls -l ~/.ssh
total 12
-rw-------. 1 ec2-user ec2-user  388 May 14 07:25 authorized_keys
-rw-------. 1 ec2-user ec2-user 2635 May 14 07:43 mykey
-rw-r--r--. 1 ec2-user ec2-user  590 May 14 07:43 mykey.pub
[ec2-user@ip-10-10-1-124 ~]$
```

- Now display the key in terminal using command: **cat ~/.ssh/mykey.pub**



- The key should be look like: "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQCt5UZNBRXcycgAznrD5mbBic42yfP8zVhc5W5NMvJuuV8Uw8khTuAeEoK7AUYzWr3QjR6sak0JzHKG1I0wsI6xgGH41i8C3JAhL9w6jw8xkX/zrKg/qVUzByuxLdqy5jl2DA9+5928dEwpWq3vfUezqWJyNwTf7qco9Fl+hLa8ZYb/8Mc3RdLOy05mWjKNk3/4oBAu4qt10mMPqH6lO7xrY/zCjK7zUrPMOZrvJtaTg3OlOxtRR+EWb8SXCQid2Lg10KcFqMw8eGjc85cA2WWcIZOq7CT6tdFGVpNsqdvVNBpn/zBExPZnbS7I40TEbR/TSrj/+YSmqZjfOpD1BMO/H42rkTXpAVNgoLRCspG2rXBh79U8rPlTsymT0W00CK9Ubf9mlEGIc62HnjvcI5Jy4qEhWJN4UHQZVkBBRqxzcvaZKkOpnNjRNs6JhudIXzqF5X8KSd/ilcAD8KmV/kJi/ddI5oZu/NTC1xOiwBdkinhorLnAnLOm5pkGwgYT83E= ec2-user@ip-10-10-1-212.ec2.internal"



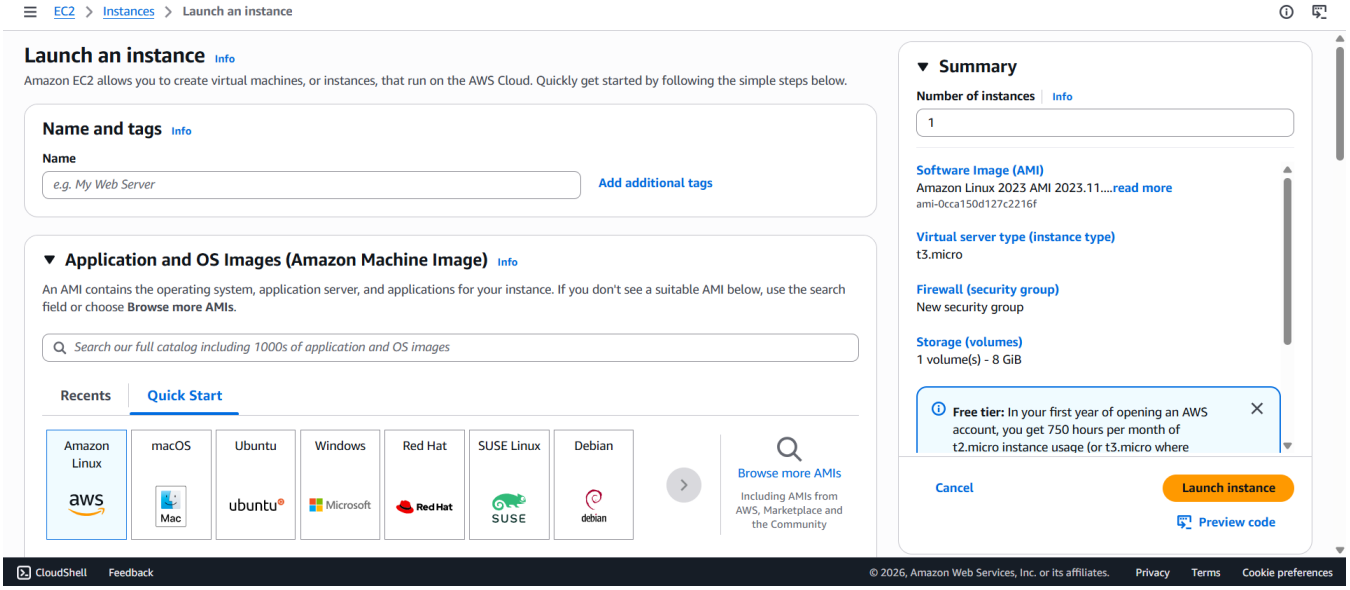
- Copy the full-key some were to use it later.

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQDEtfAIuYIicVIsnJO5UPugXZo0POpu+rFPcPL
XyTr5NDKeUpFGsi4gQiwO6NTtIXACM2GenRwqMxQ3xcJzQ4qlQT52a+3/EKzeCoZdg8
W5d8m/PsdZsfmfT79Lba0N8UjHA0AGQEZ21X7qOZDwLM2EfC+9YdLzaKO07ROVEZfIx
8eskOmxce+S6sU+k7Ze8MTx5+xkeIhV6DHPFatJlql5sNNFDyzi6vZ8n1Q7MSFl3DiC
vo6mPOxpERK5qI7/tgS51IS1gyCEkoQiHP3UU/7HmRH2FNr1hQFSVXp2DAHrEgCAs/G
ptQHNcY7kgmlFwDu0BkwHqWPLlXDnxpnuDf5SRpbwNJcfP/sAu86WVtMdWm4alFMa3X
/zpOjkBx2SHv8Azt0LiA0V4jKMQroIlOYiJUphd+aIKmCNY1JqUEGtyS7BgoOOUZR+J
kl/bRy7ABH14qxUGJUDo4F0bqMKHR7Nx65Em1LoSifXWMVYu1p/bPJcpAP5uZa+reKd
zX/QJcU= ec2-user@ip-10-10-1-124.ec2.internal
```

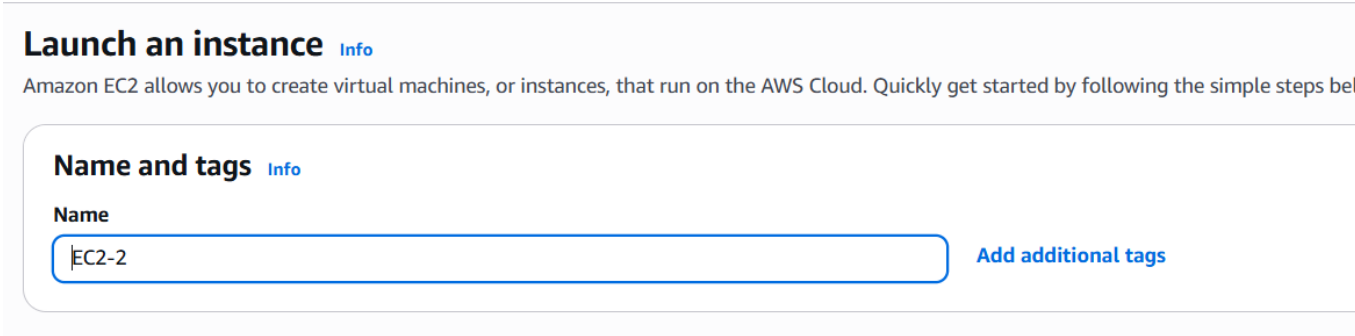
6.4. Task 4: Provide Outbound Internet Access for Private Subnet (NAT Gateway): Allowprivate instances (like EC2-2) to access the Internet for updates without exposing themtoinboundconnections.

- Instructions:
  - Launch EC2-2 (Private Instance):

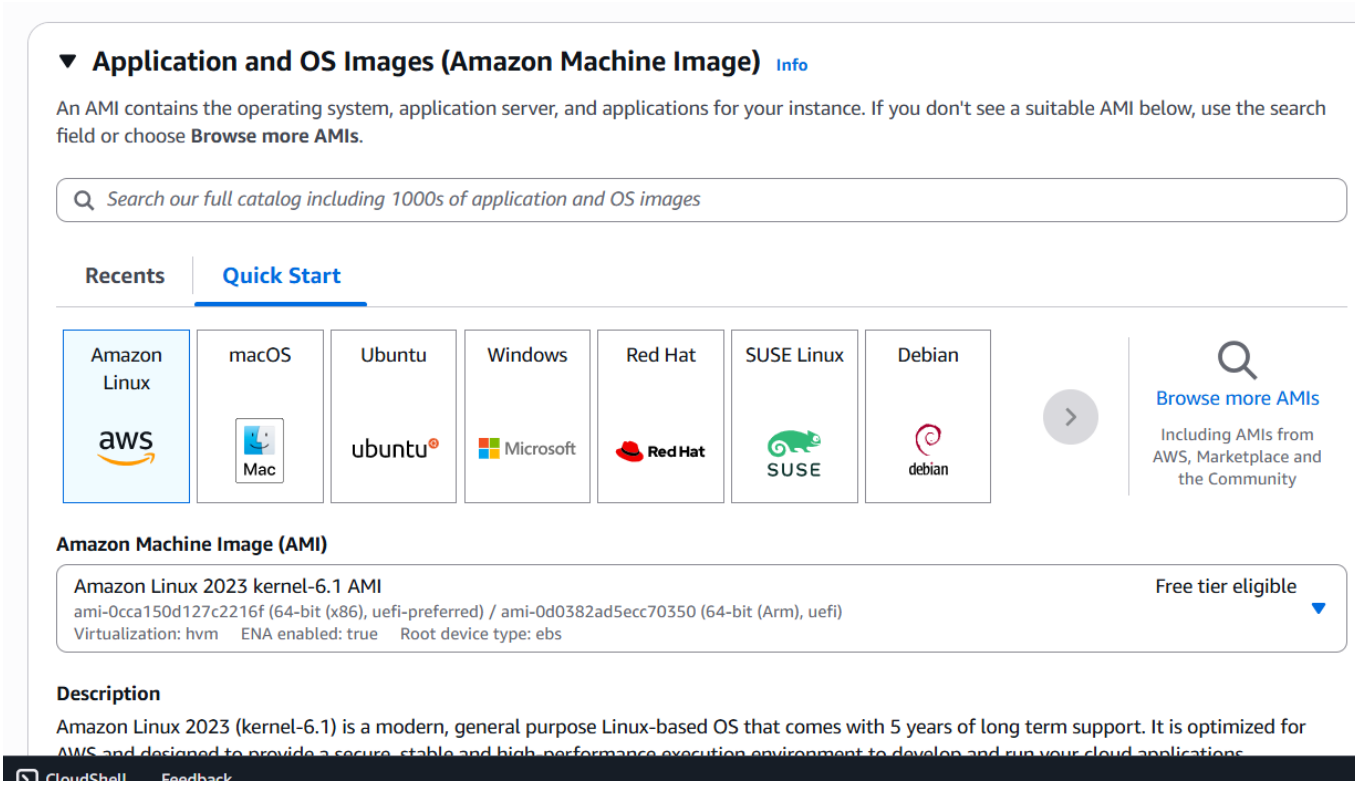
Click Launch Instance again.



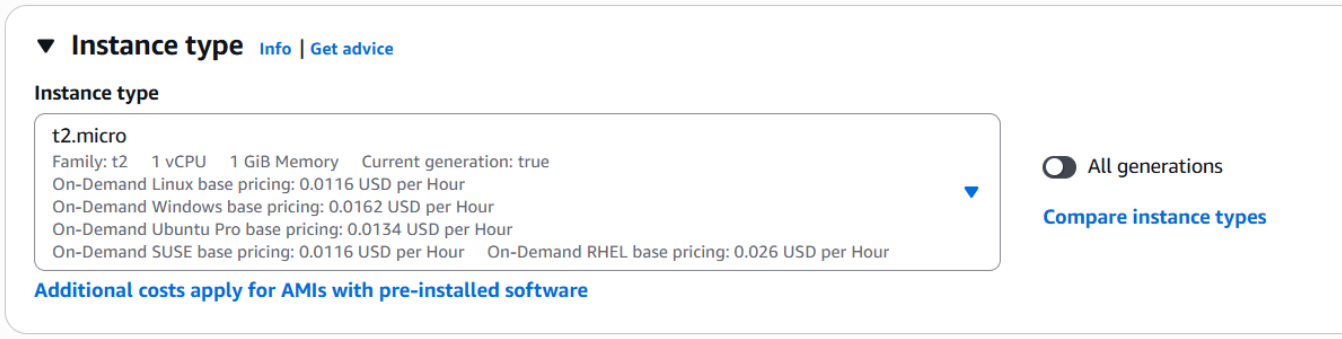
Name: EC2-2



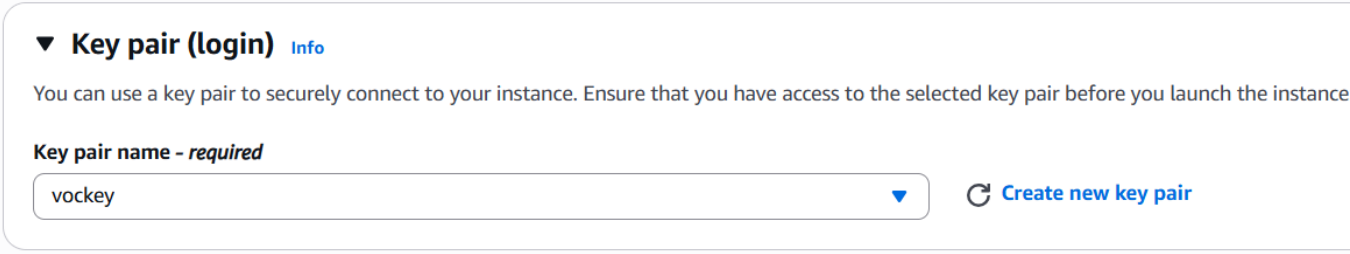
AMI: Select Amazon Linux 2 AMI.



Instance type: Select t2.micro.



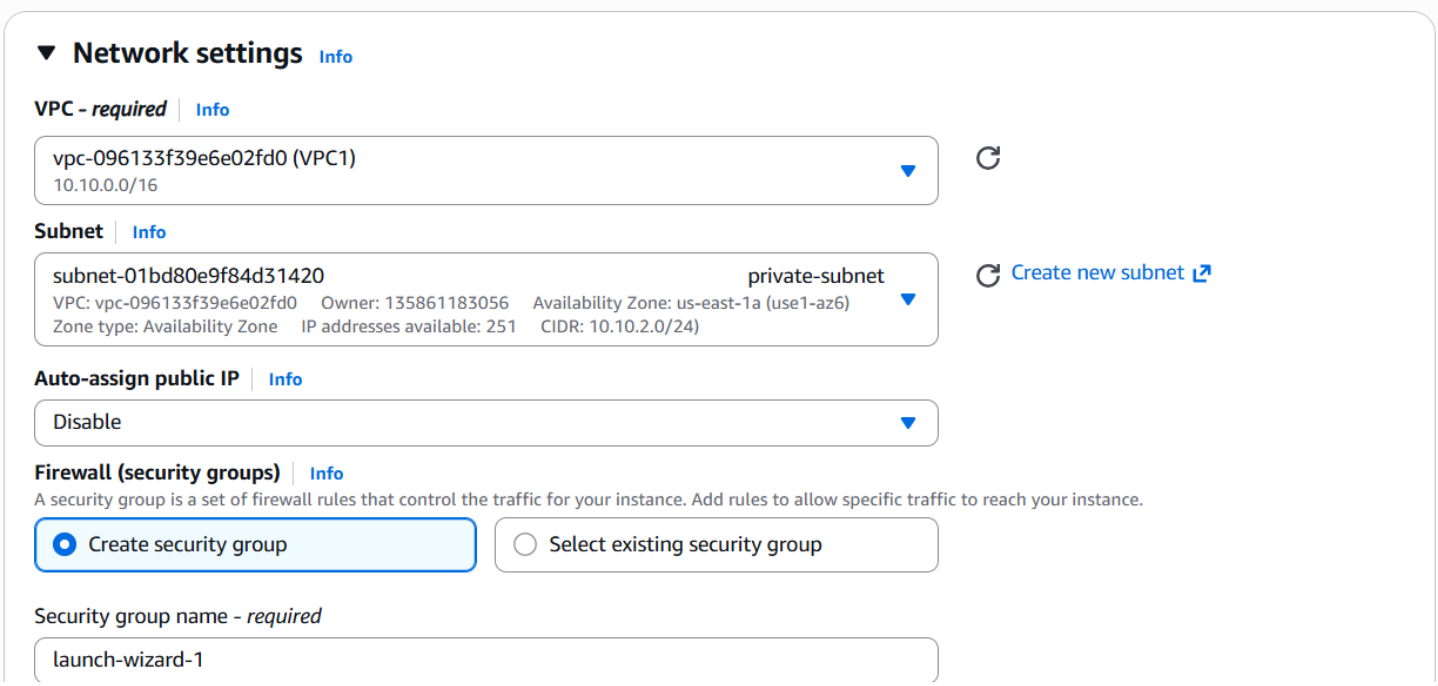
Key pair (login): Select the same key pair you used for EC2-1.



Network settings: Click Edit.

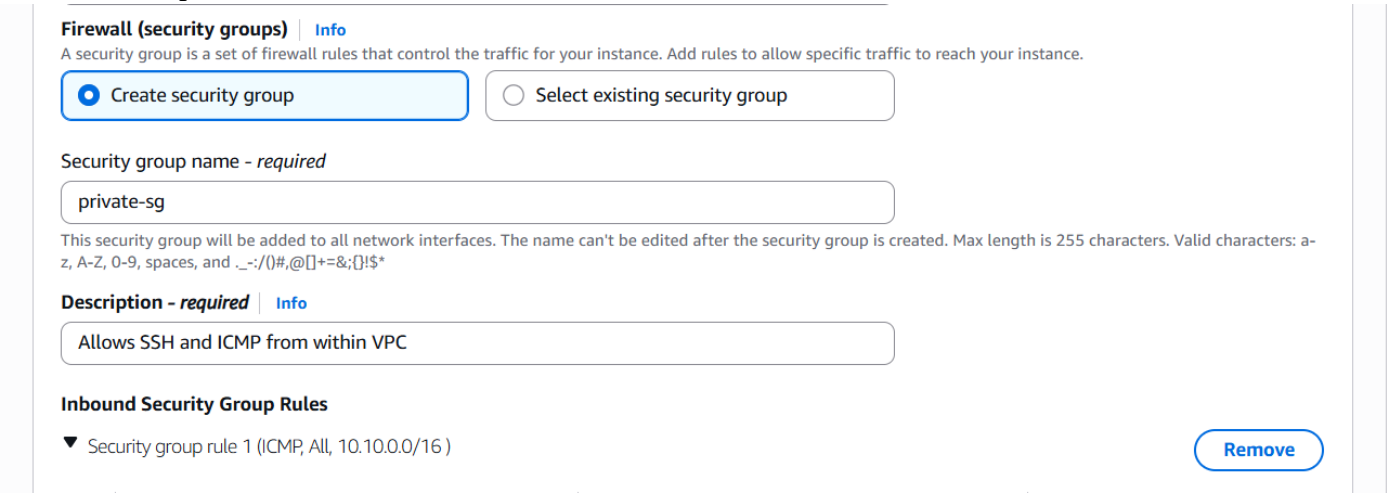
- VPC: Select VPC1.
- Subnet: Select private-subnet.

- Auto-assign public IP: Select Disable.



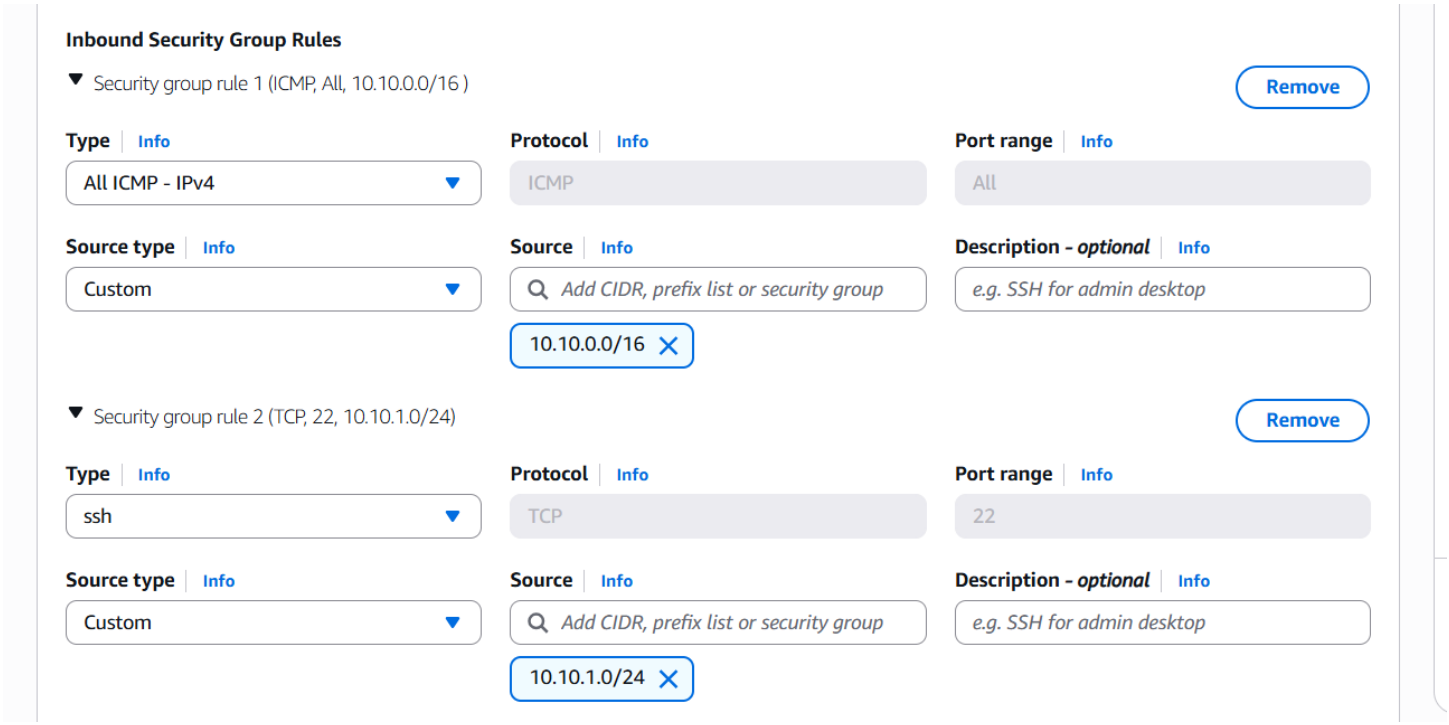
Fir ewall (security groups):

Select Cr eate a new security group.
Security group name: `private-sg`
Description: Allows SSH and ICMP from within VPC



Inbound rules:

- Rule 1: Type All ICMP - IPv4, Source Custom `10.10.0.0/16` (Allows pingfromVPC1).
- Rule 2: Click Add security group rule. Type SSH, Source Custom`10.10.1.0/24`(Allows SSH only from your public subnet).

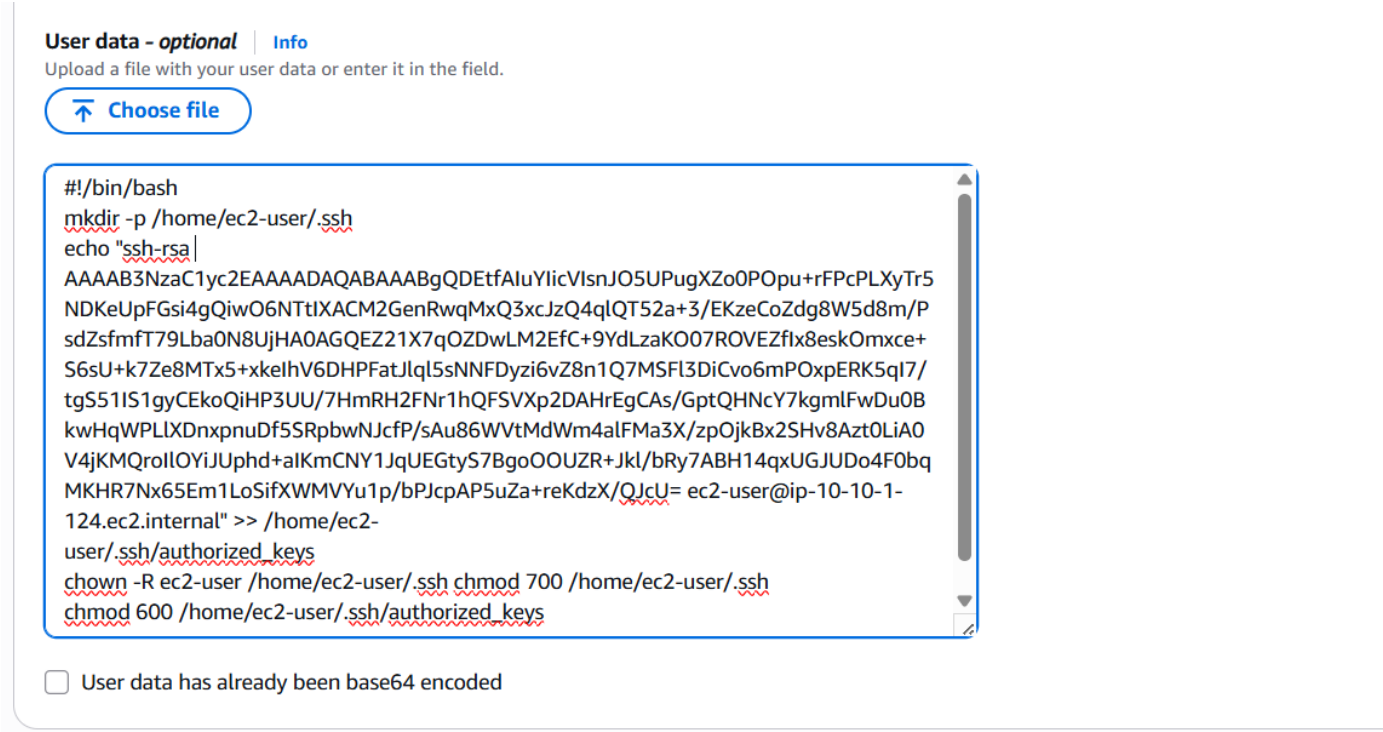


Go to Advance

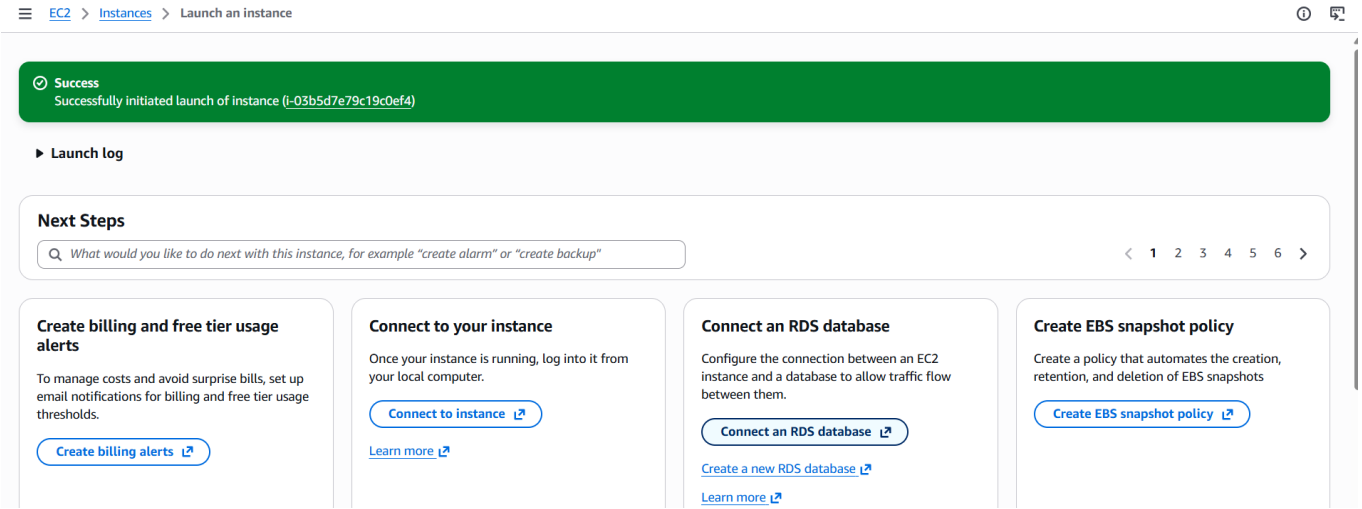
- Navigate to User data. Copy this code and paste it on User data field.

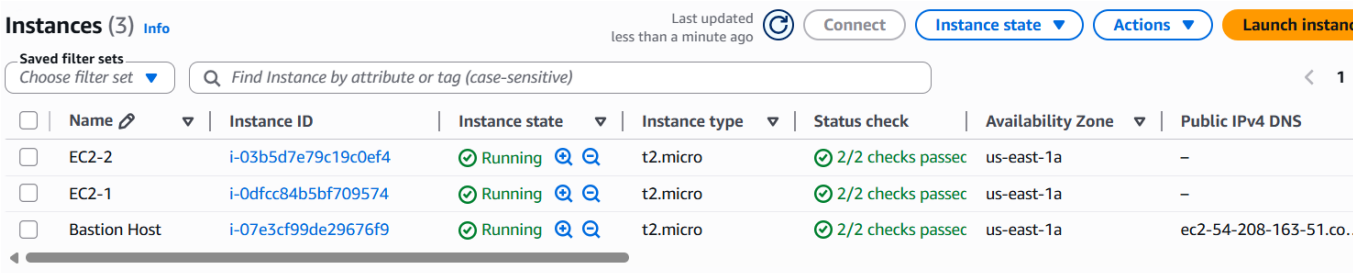
```
#!/bin/bash
mkdir -p /home/ec2-user/.ssh
    echo "PASTE_YOUR_mykey.pub_HERE" >> /home/ec2-
user/.ssh/authorized_keys
chown -R ec2-user /home/ec2-user/.ssh chmod 700
/home/ec2-user/.ssh
                    chmod 600 /home/ec2-user/.ssh/authorized_keys
```

- Replace PASTE_YOUR_mykey.pub_HERE with your actual key which you have created with EC2-1 user.

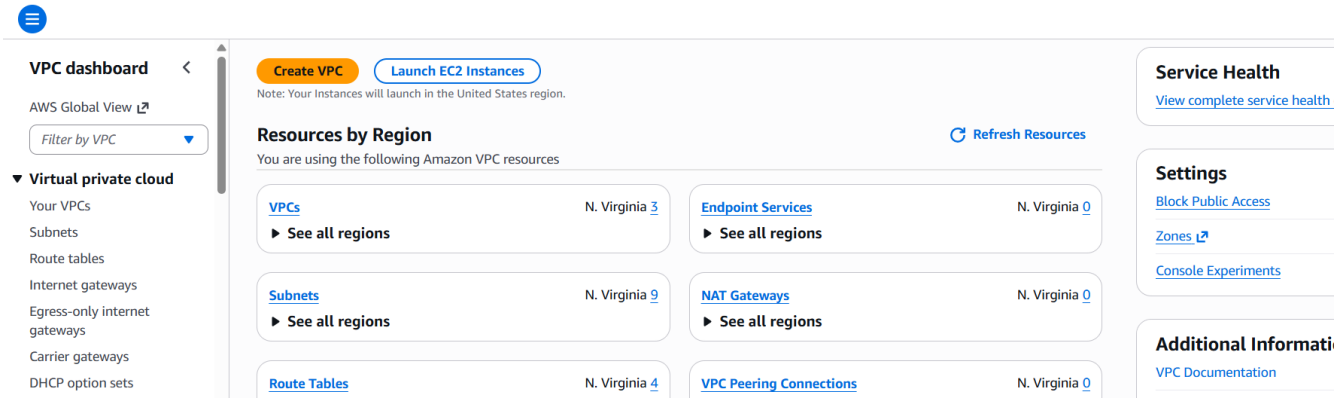


■ After click on Launch instance. Wait ~1 minute after it starts and pass checks.

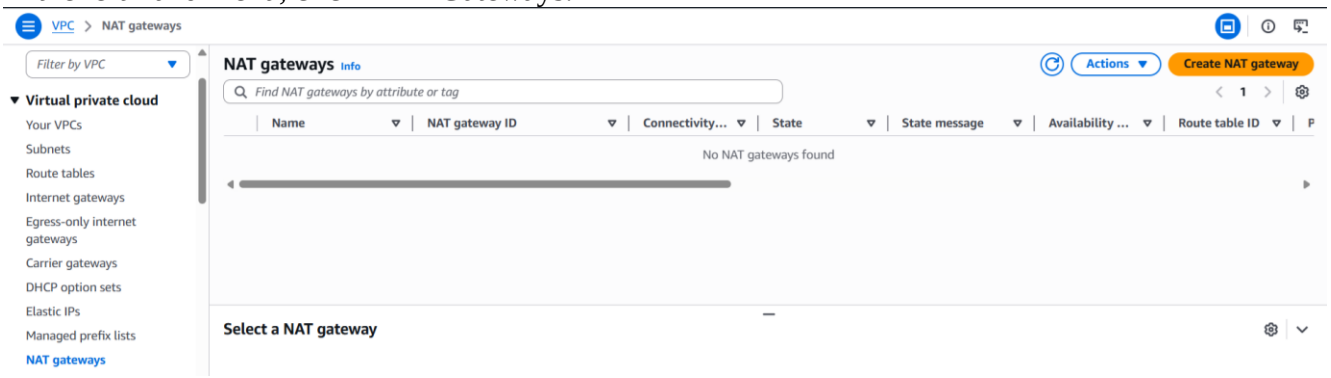




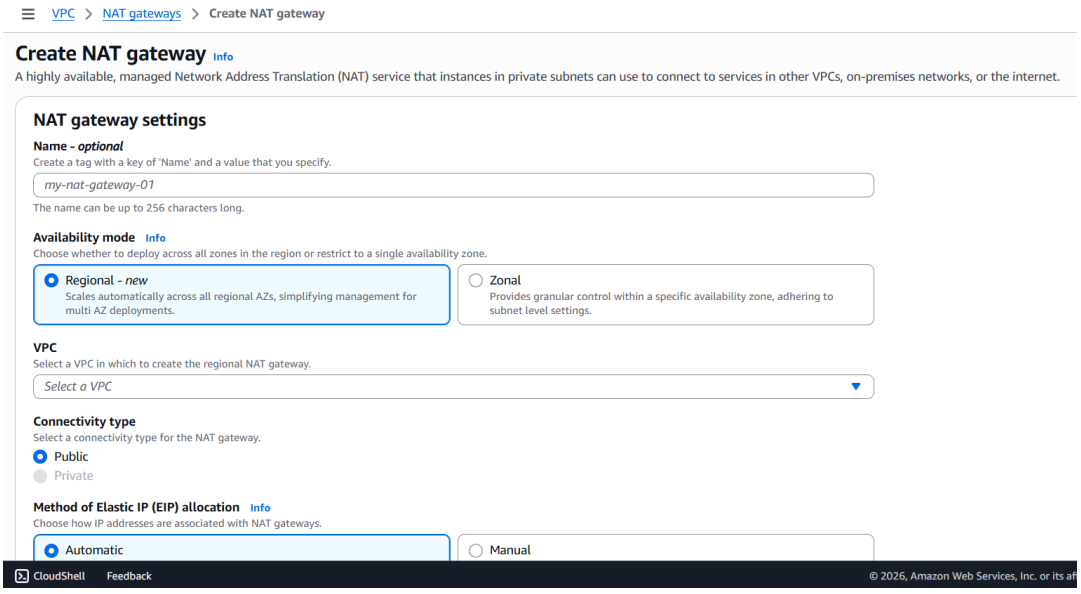
Now navigate to the VPC service.



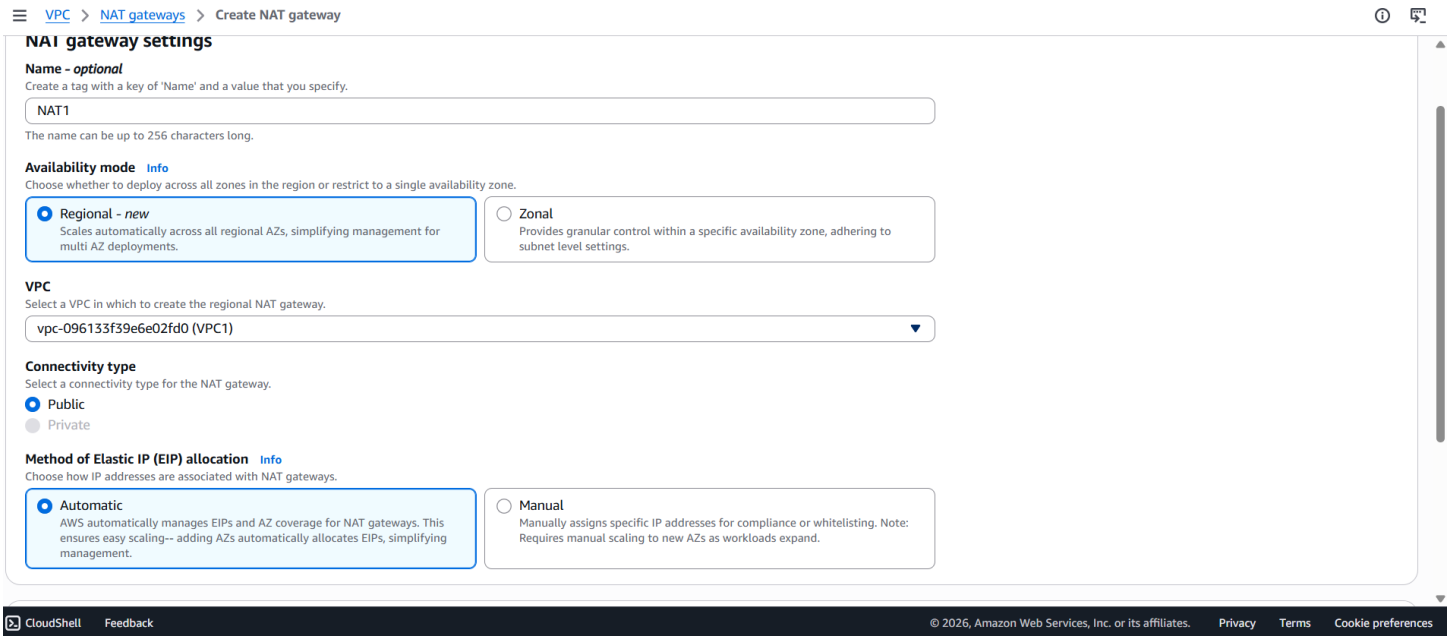
In the left-hand menu, click NAT Gateways.



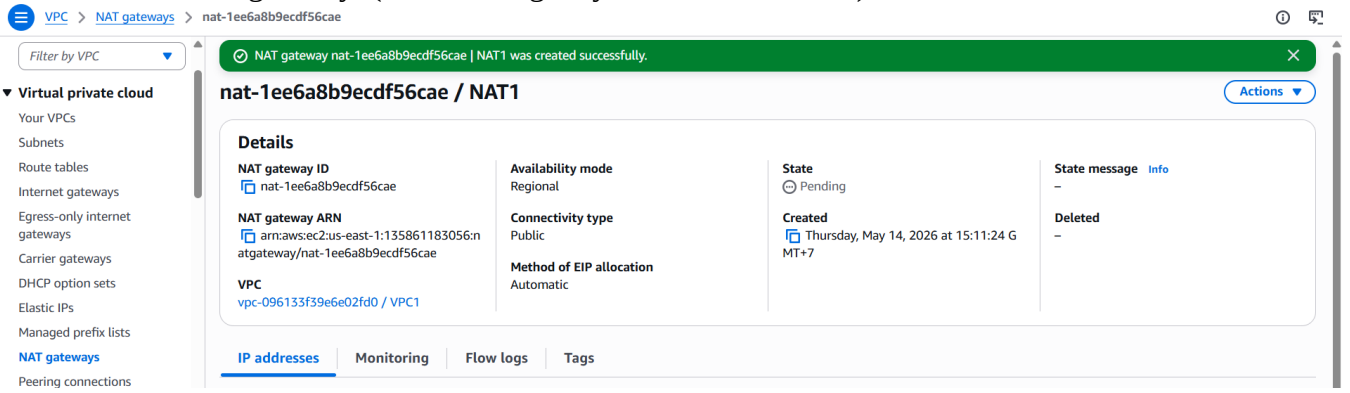
Click Cr eate NAT gateway.



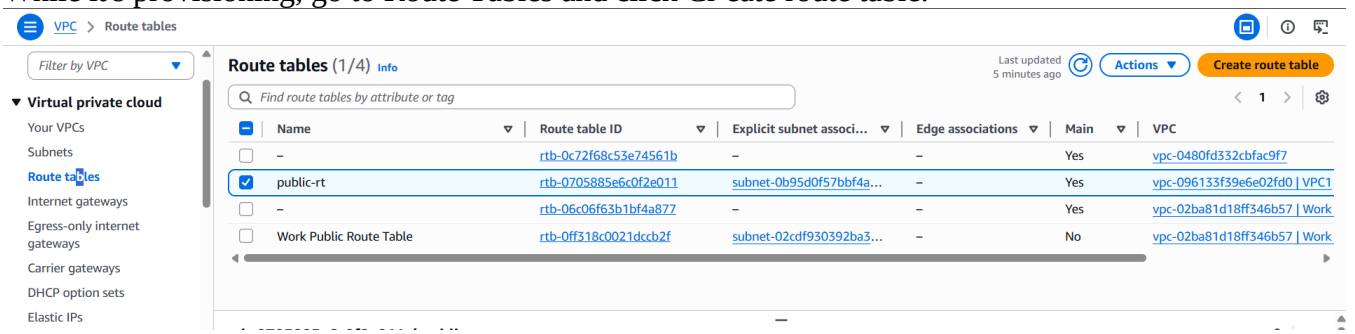
- Name: NAT1
- Subnet: Select public-subnet (A NAT Gateway must reside in a public subnet).
- Connectivity type: Public.
- Elastic IP allocation: Click Allocate Elastic IP.

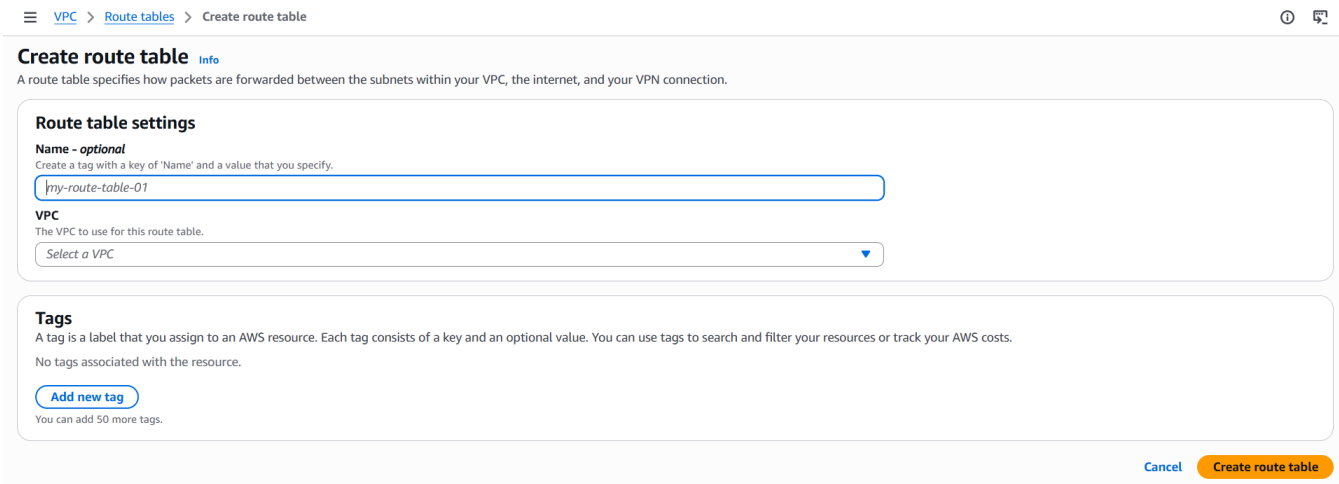


Click Cr eate NAT gateway. (Provisioning may take a few minutes).

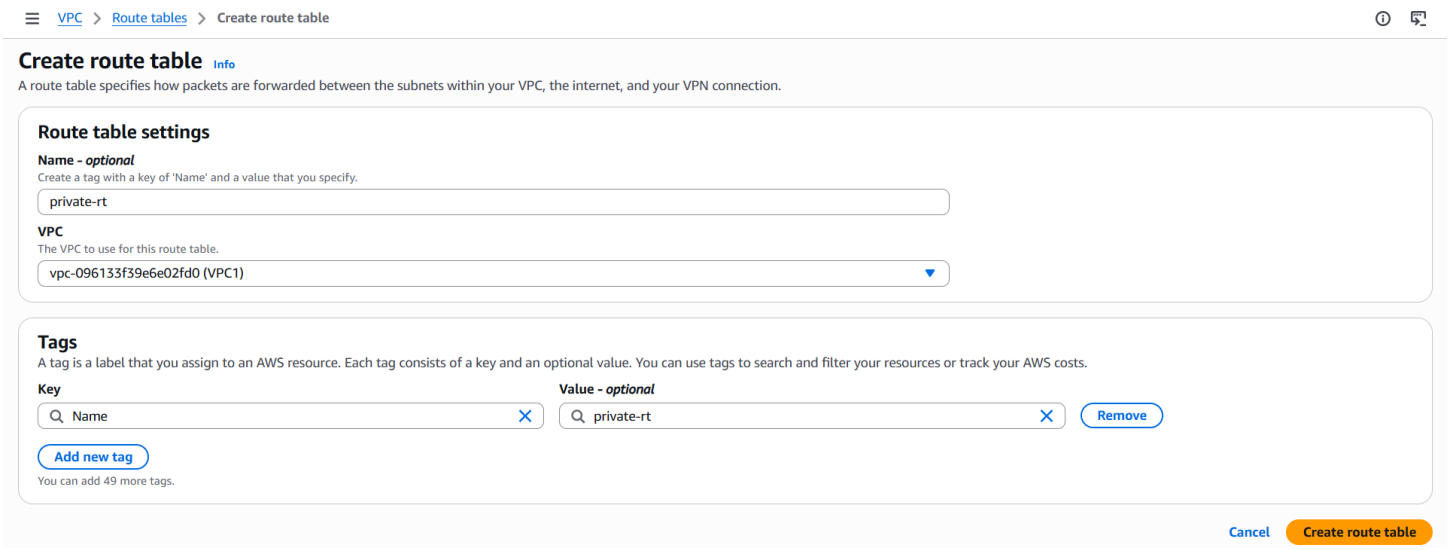


While it's provisioning, go to Route Tables and click Cr eate route table.

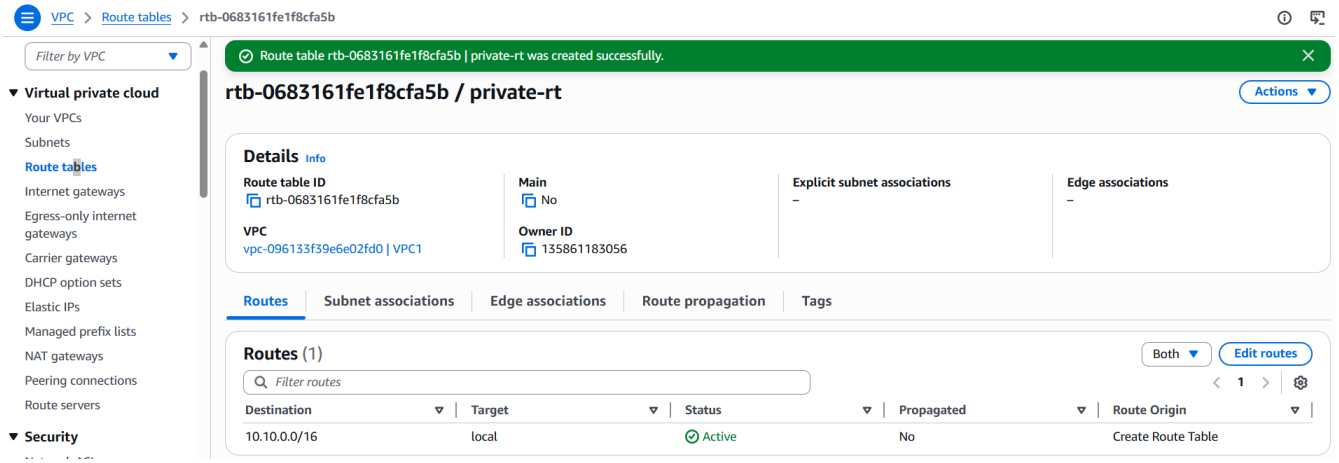




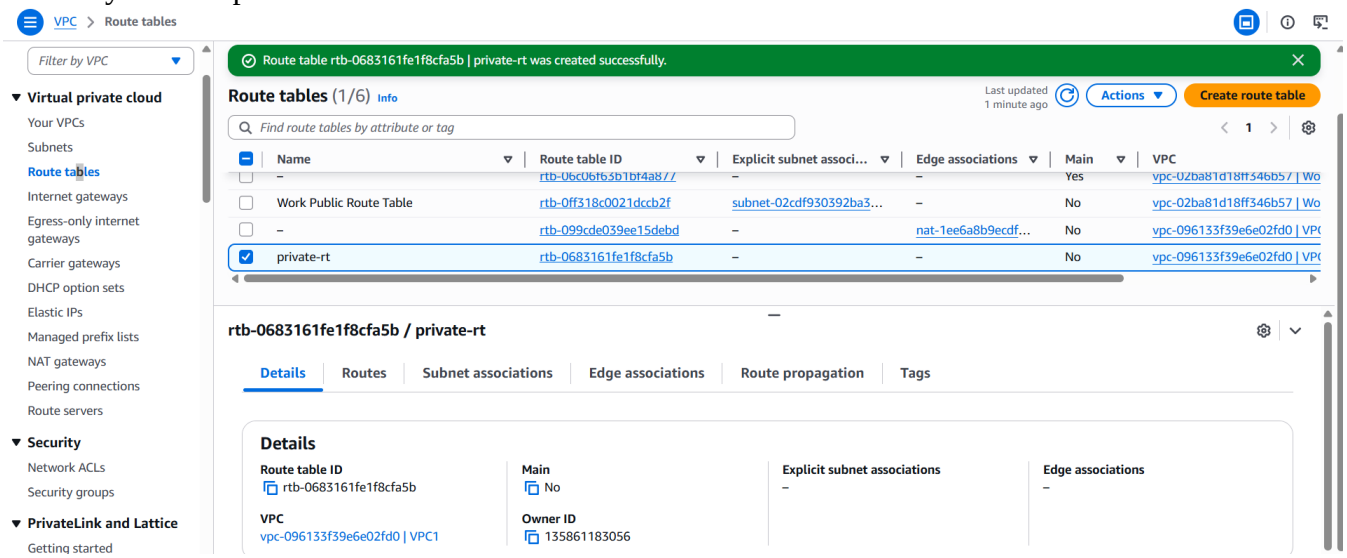
- Name: `private-rt`
- VPC: Select VPC1.



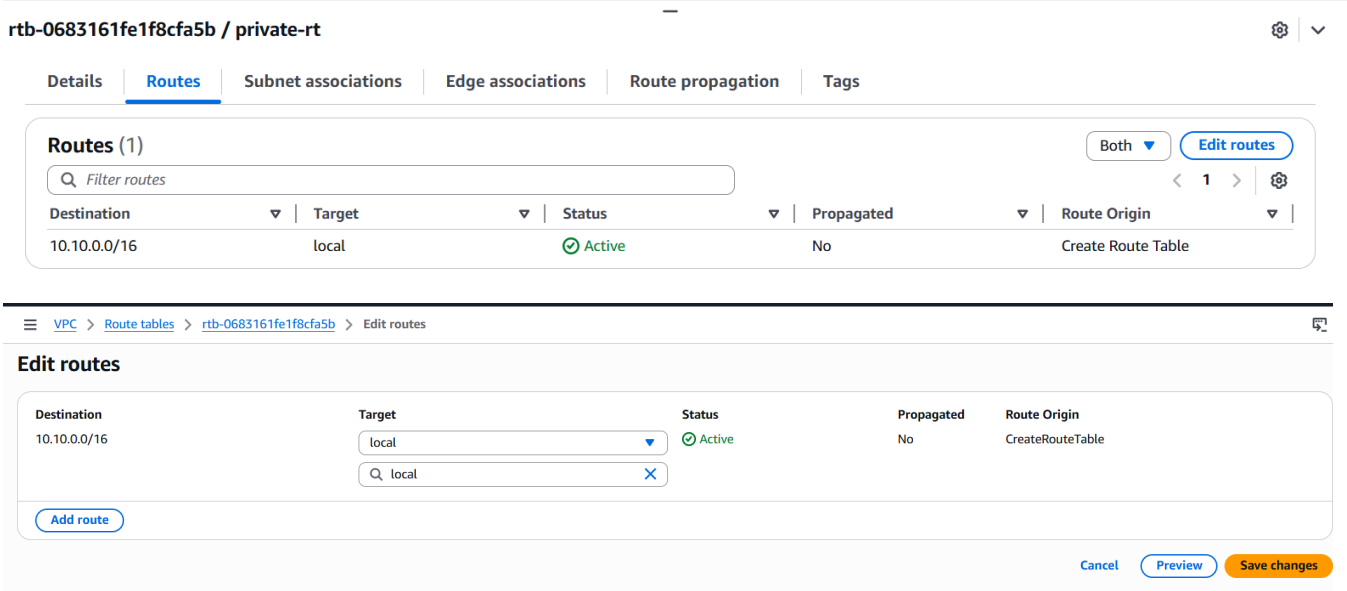
. Click Cr eate route table.



. Select your new private-rt from the list.

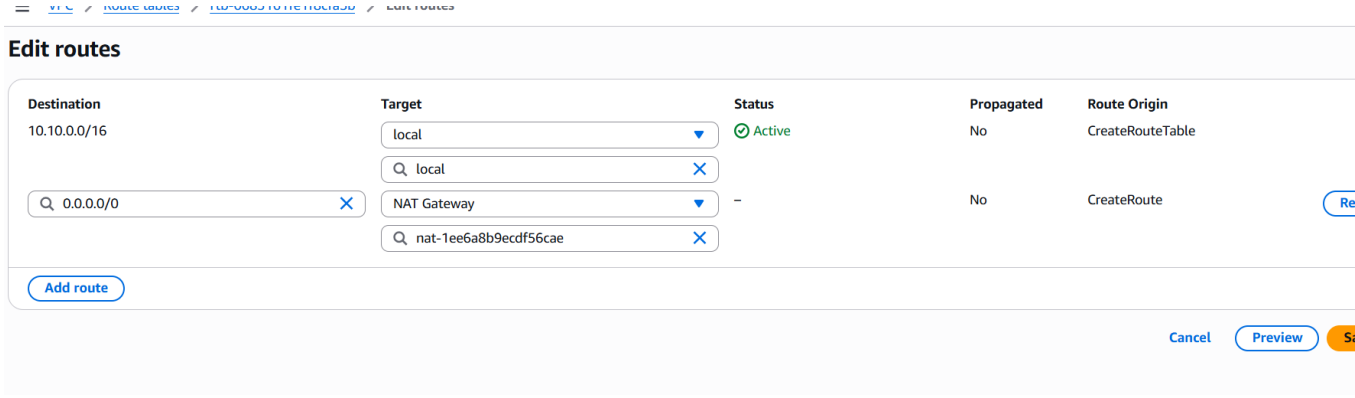


. In the Routes tab, click Edit routes.

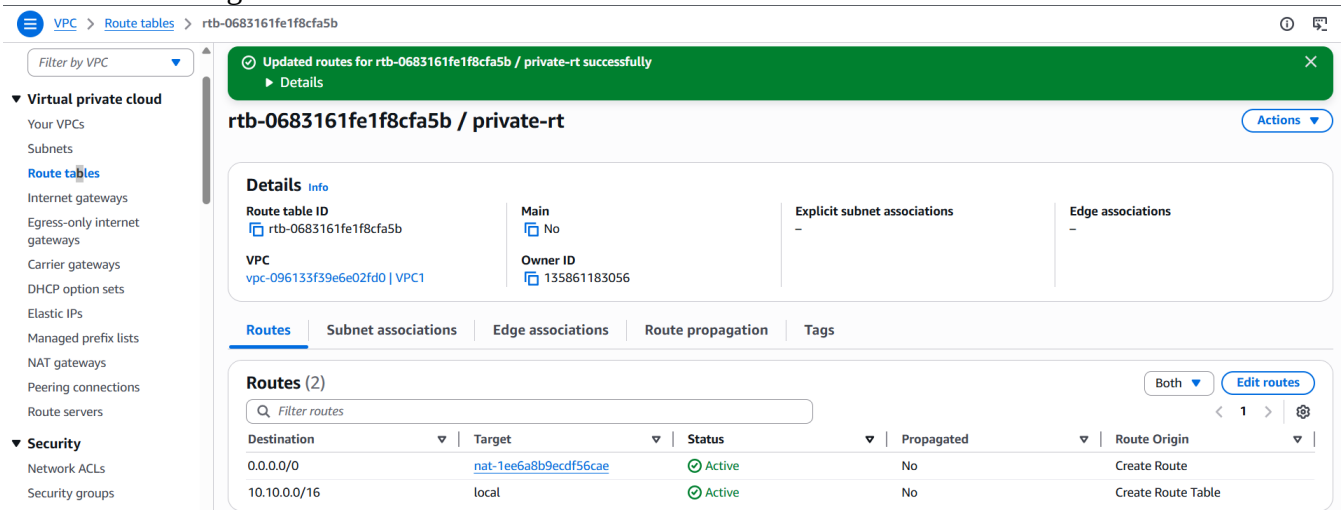


. Click Add route.

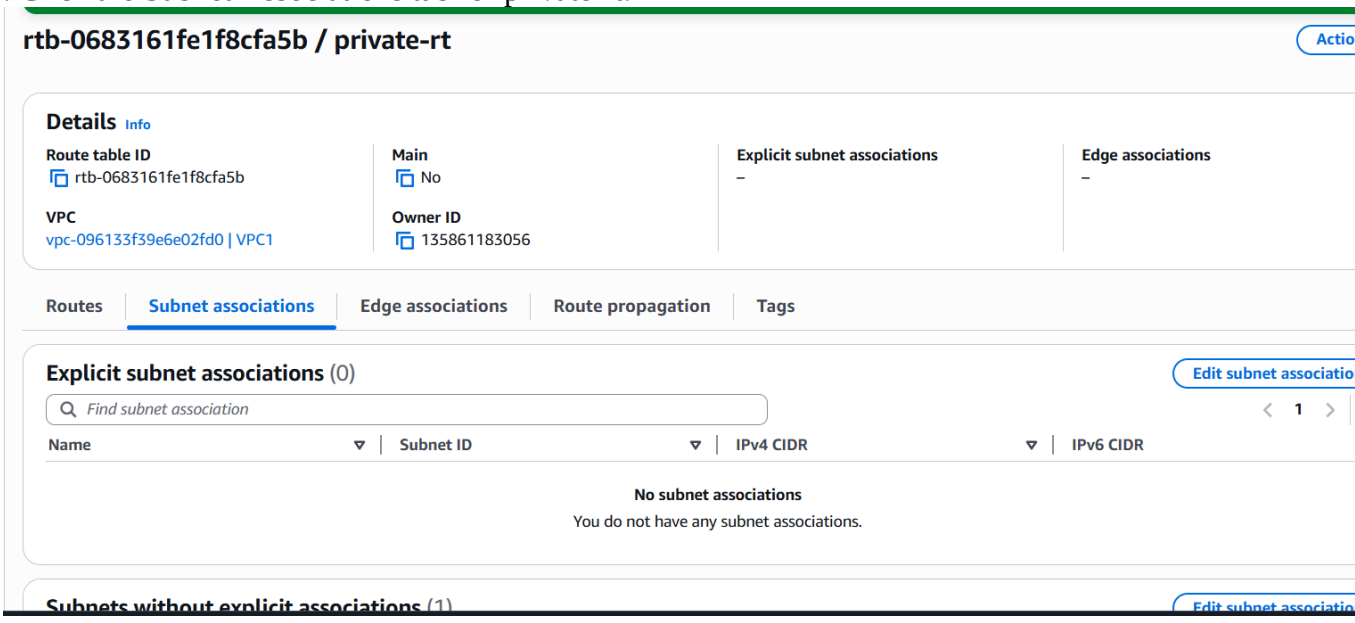
- Destination: 0.0.0.0/0
- Target: Select NAT Gateway and then choose NAT1.



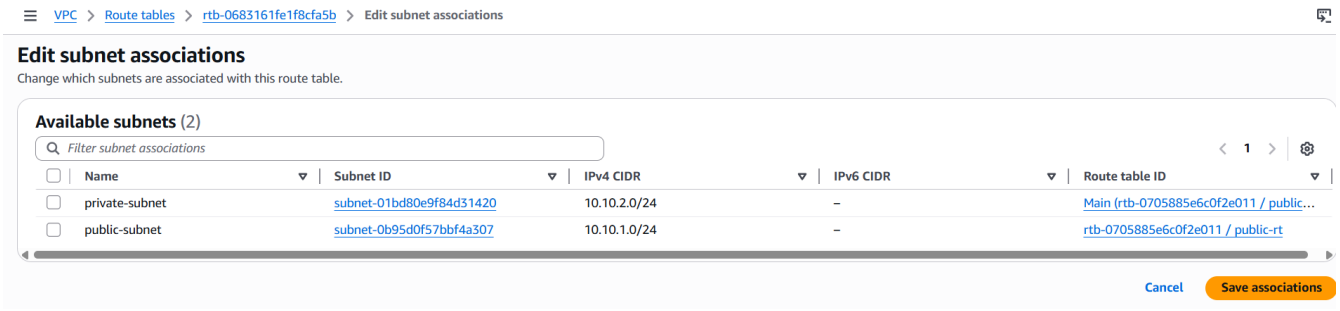
. Click Save changes.



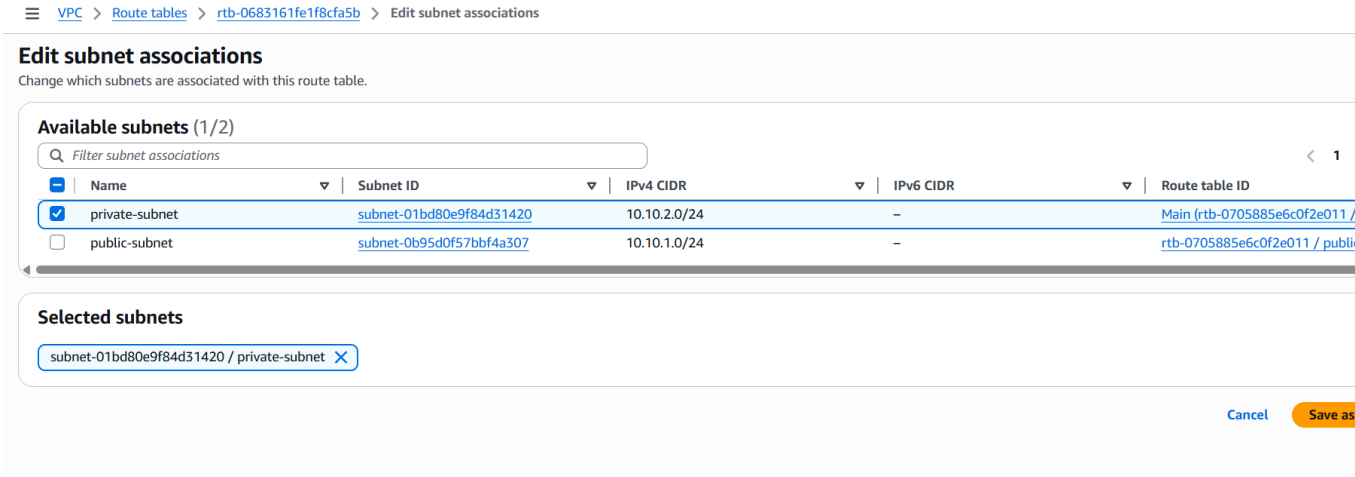
. Click the Subnet Associations tab for private-rt.

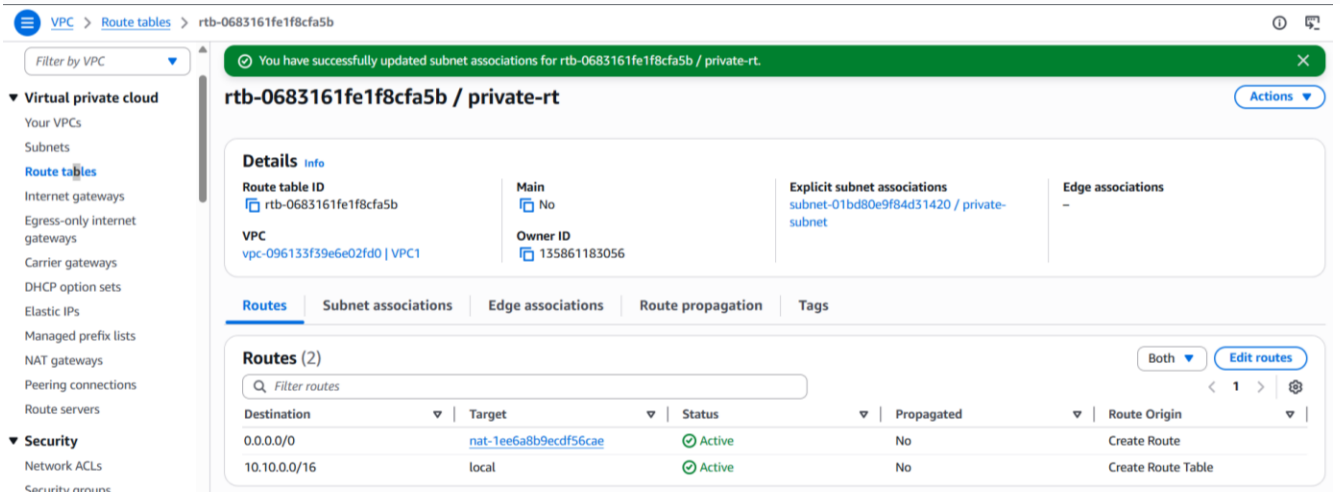


. Click Edit subnet associations.



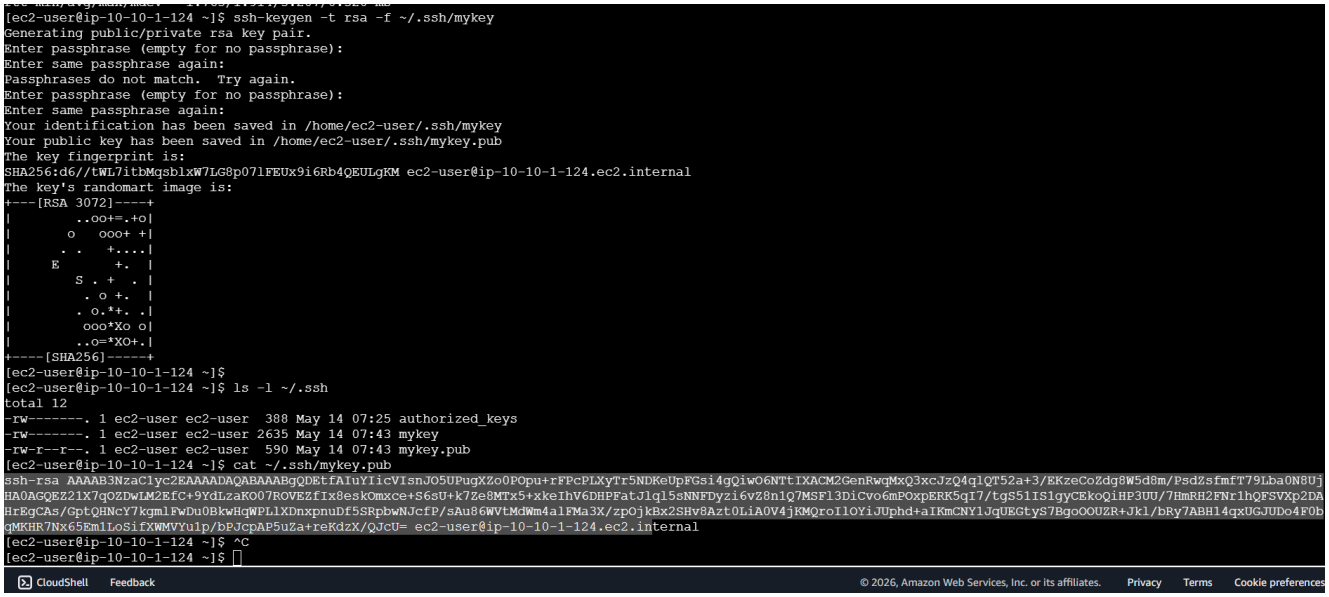
. Check the box for private-subnet and click Save associations.



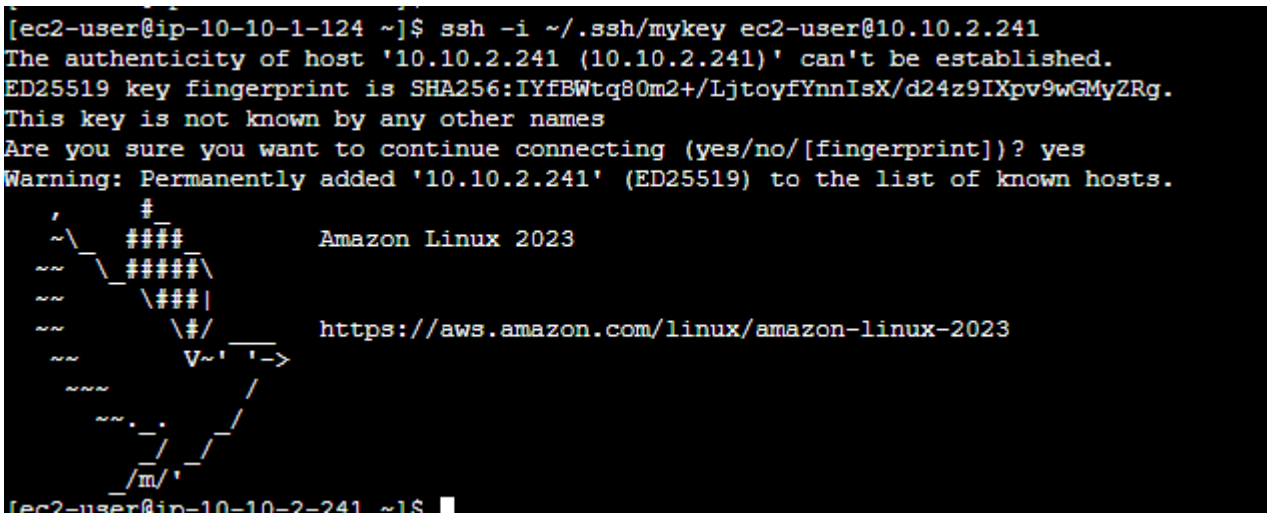


. Verify Outbound Access:

- Once again go to the EC2-1 AWS browser terminal.



- Login in to EC2-2 user through EC2-1 by using command:
  - `ssh -i ~/.ssh/mykey ec2-user@<private IP>`



- You will successfully login to EC2-2 user via NET gateway.
- test its internet access: curl google.com



You should get an HTML response. ping `8.8.8.8` will also work. This confirms EC2-2canreach the internet via the NAT Gateway.

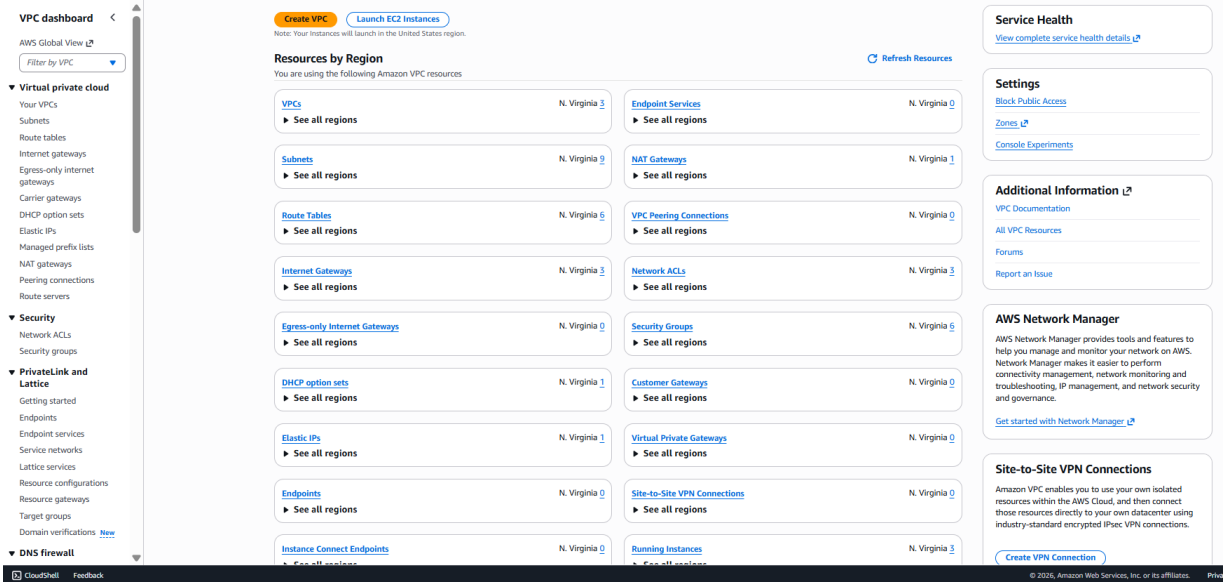
```
[ec2-user@ip-10-10-2-241 ~]$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=2.36 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=1.43 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=1.77 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=1.79 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=116 time=1.74 ms
```

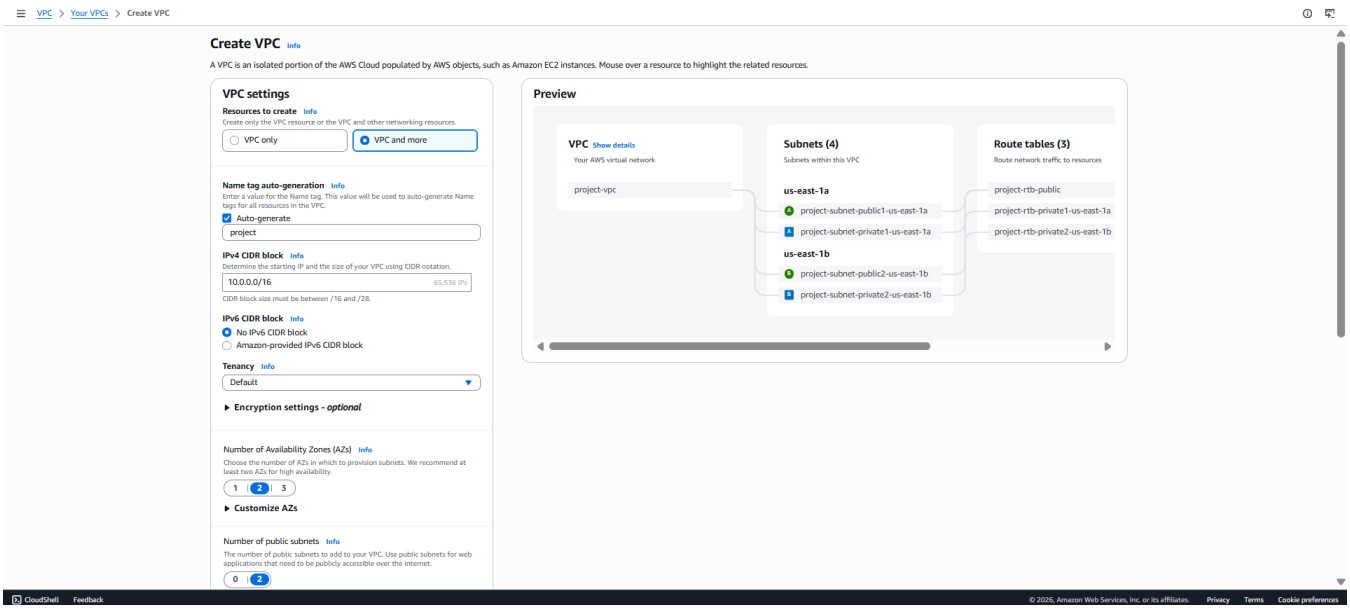
6.5. Task 5: Peer Two VPCs: Enable private, direct communication between two VPCs as if theywere on the same network.

- Instructions:

  Cr eate VPC2:

  Go to the VPC service -> Cr eate VPC.

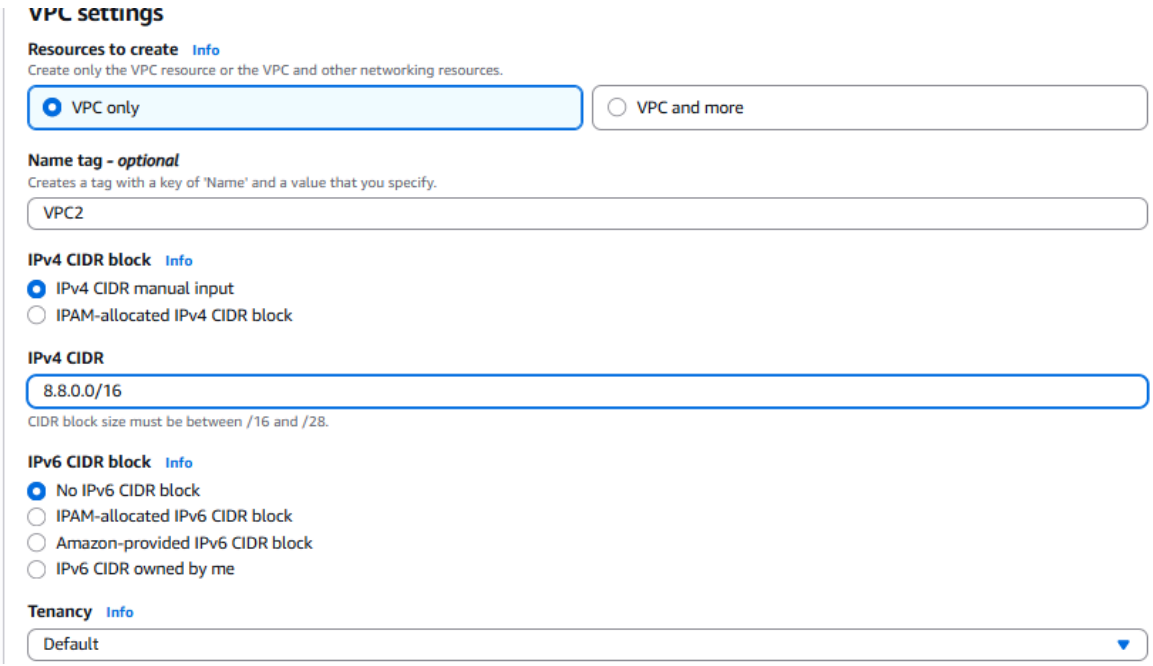




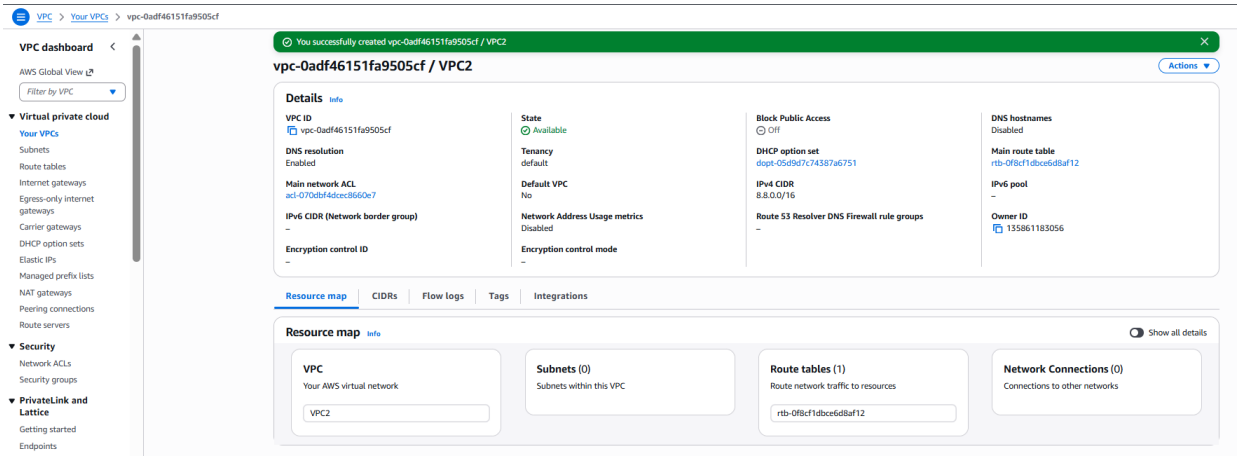
Select VPC only.

Name: VPC2.

IPv4 CIDR: `8.8.0.0/16`.

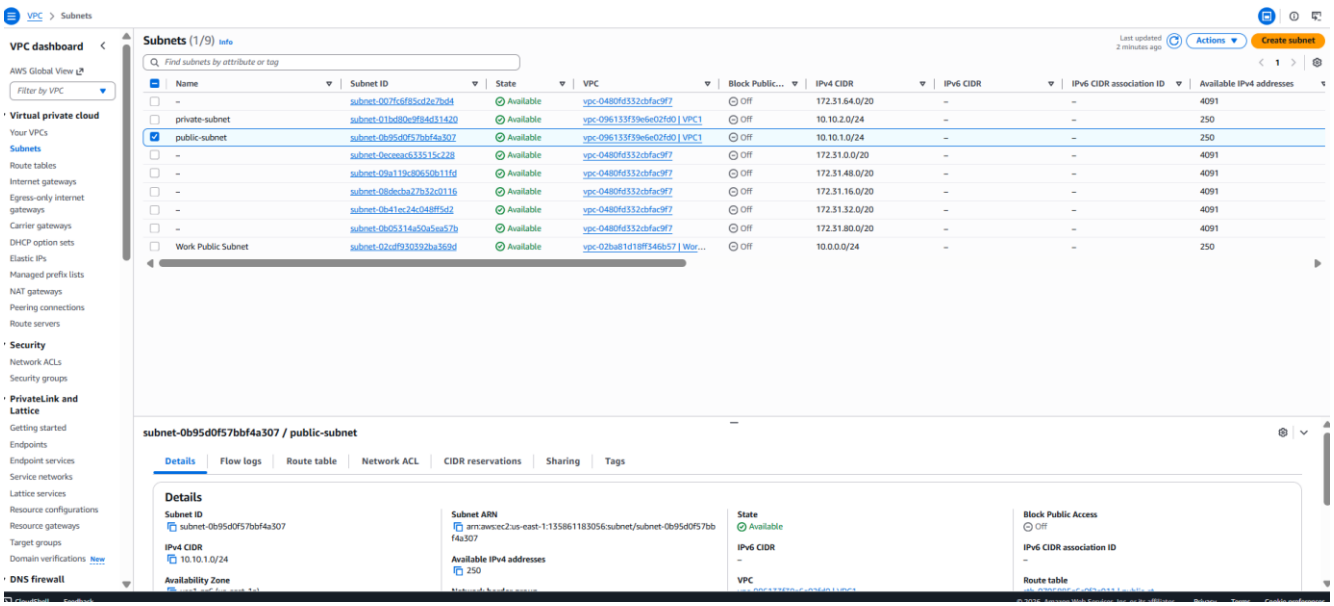


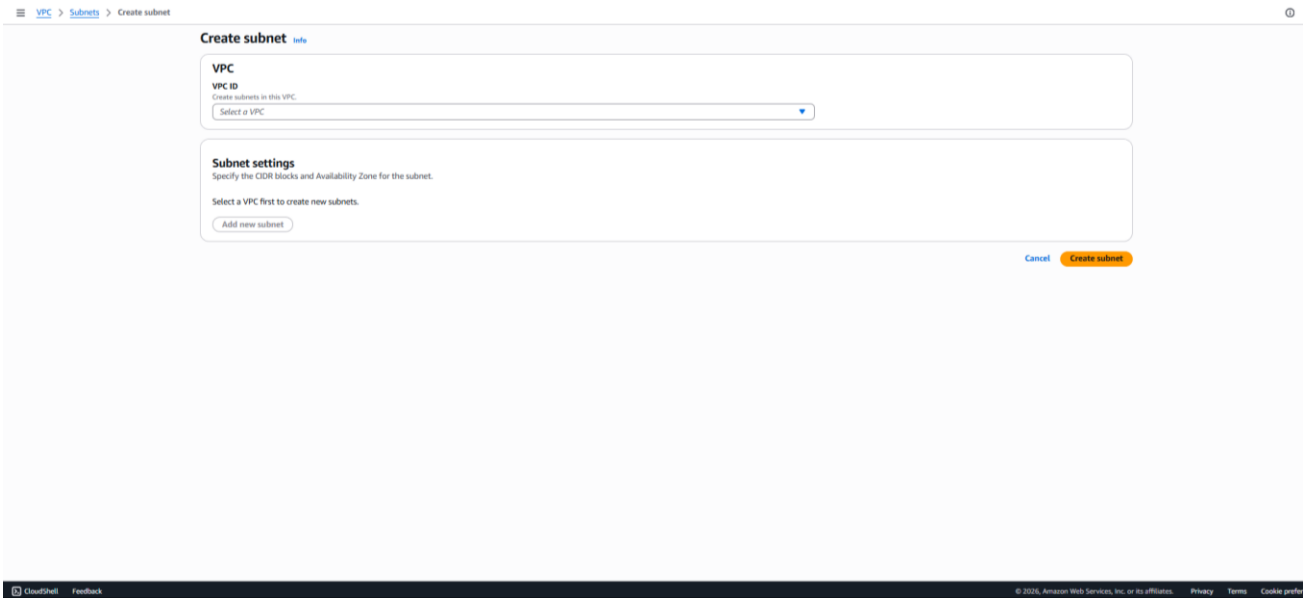
Click Cr eate VPC.



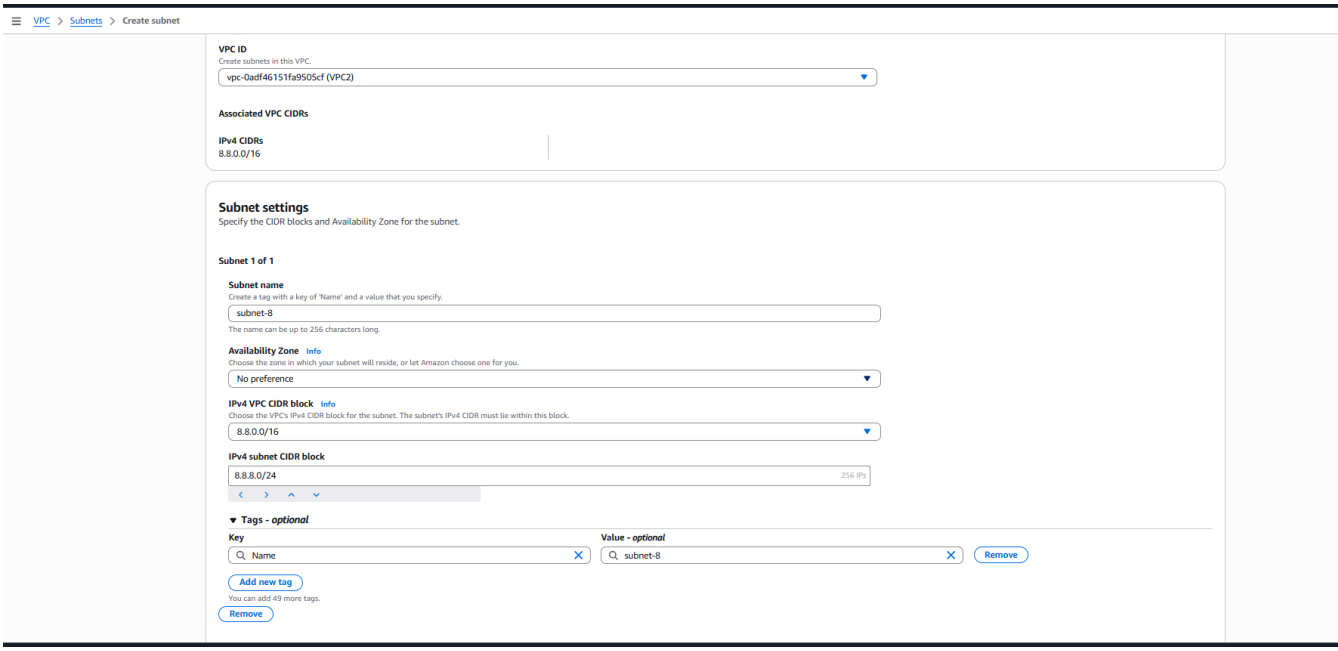
Cr eate Subnet for VPC2:

- Go to Subnets -> Cr eate subnet.

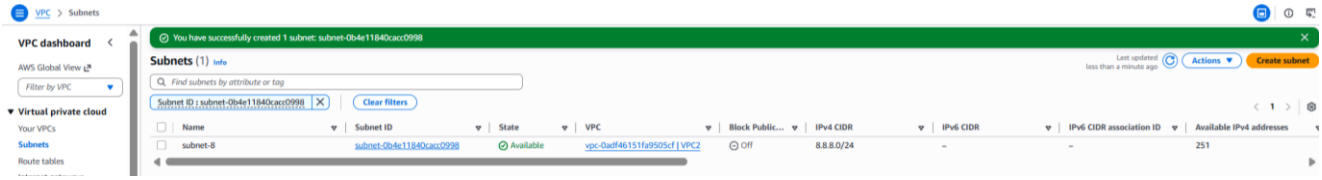




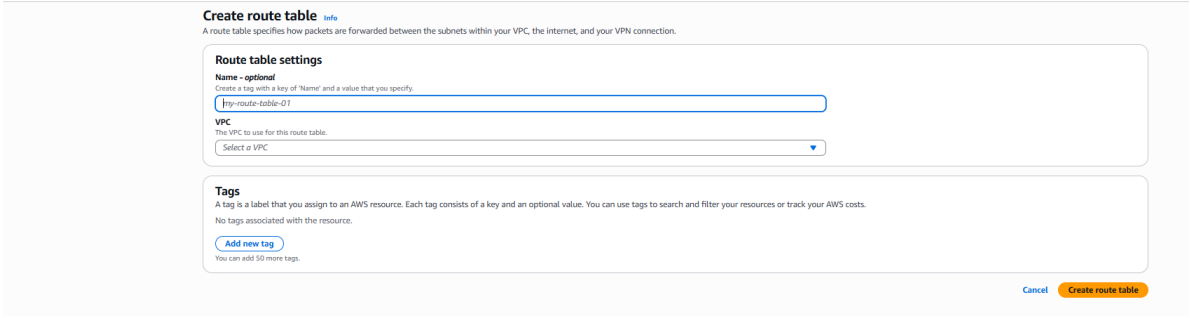
 VPC ID: VPC2.
Subnet name: subnet-8.
IPv4 CIDR: 8.8.8.0/24.



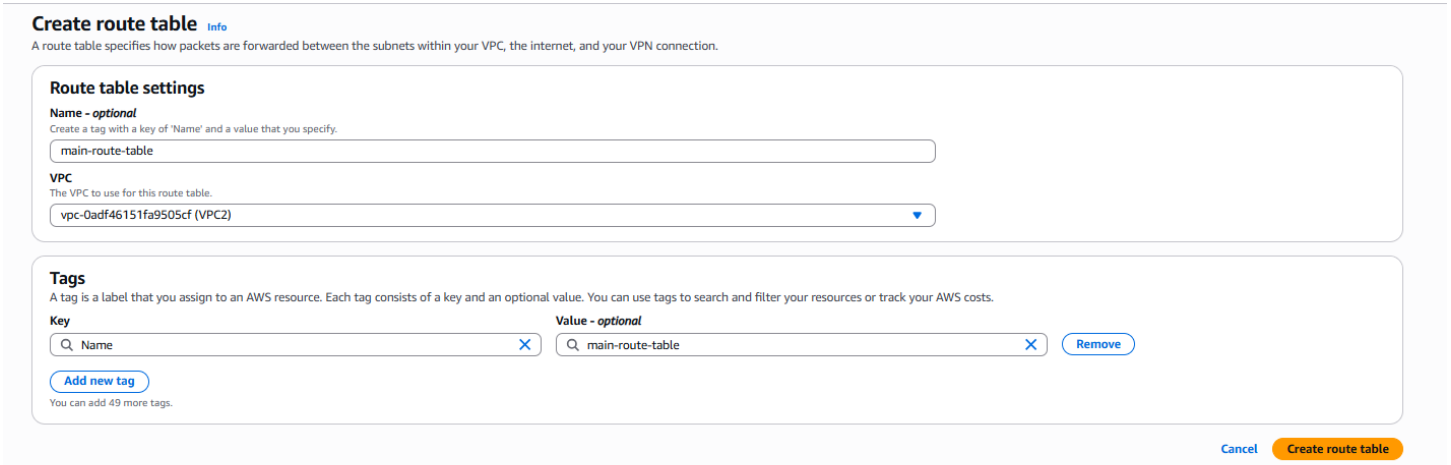
 Click Cr eate subnet.

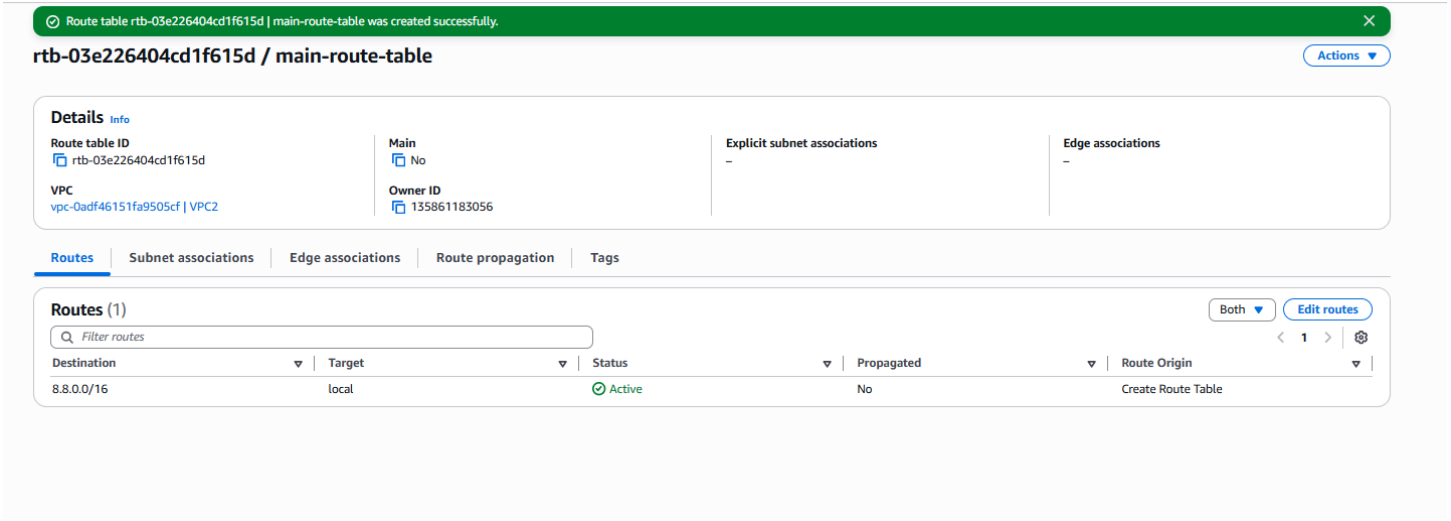


Go to Route Tables and click Cr eate route table.



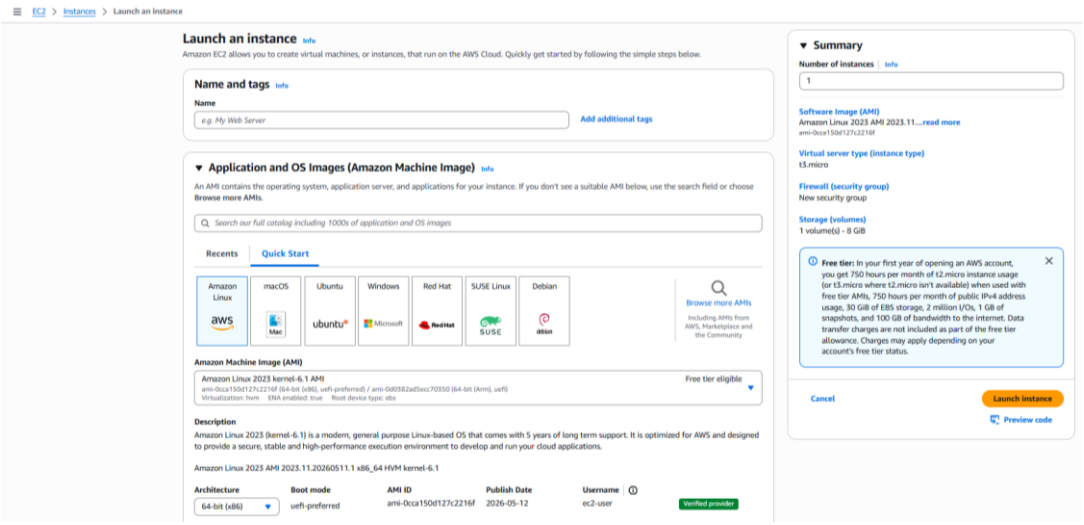
- Name: `main-route-table`
- VPC: Select VPC2.
- Click Cr eate route table.



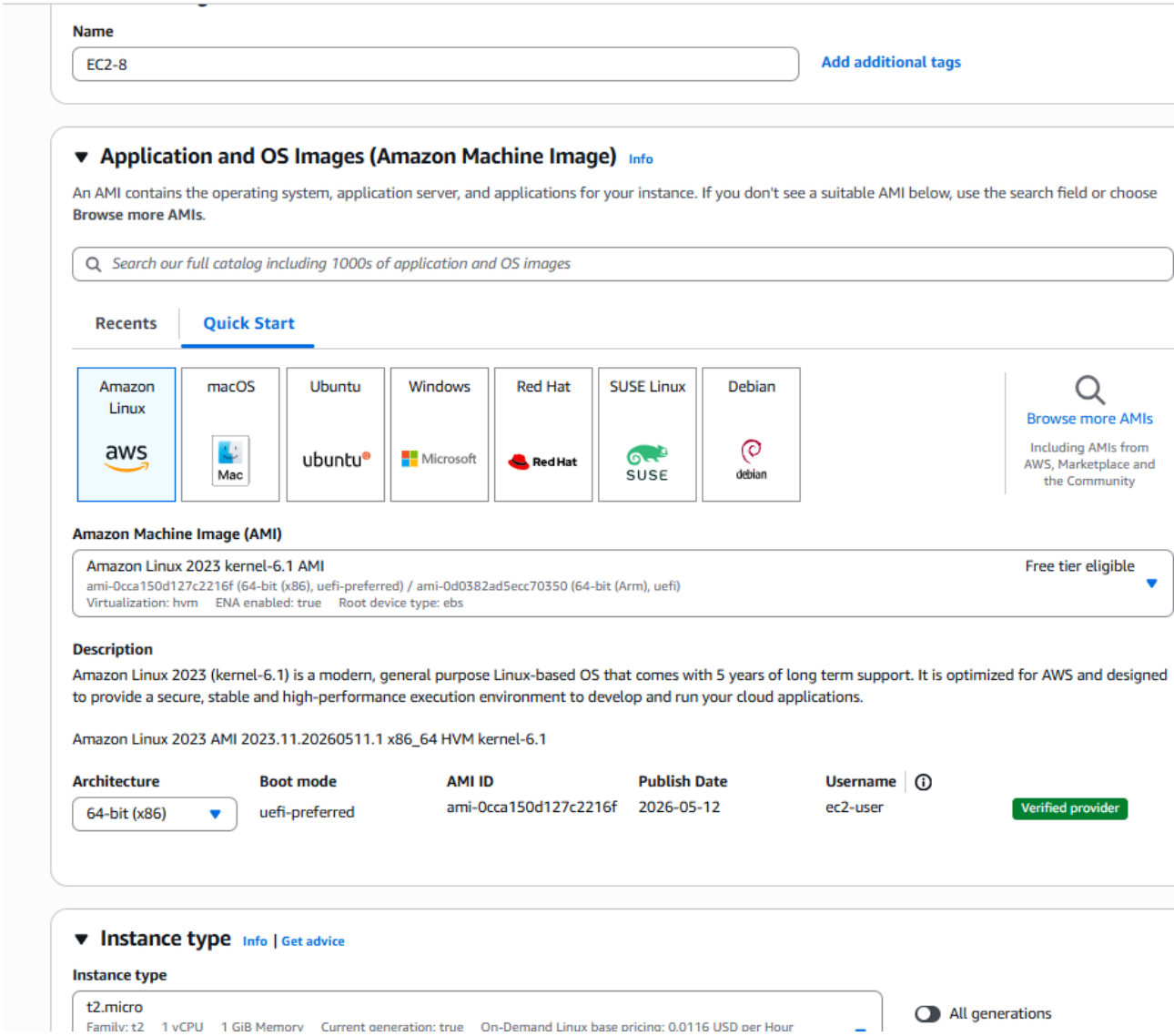


. Launch EC2-8 in VPC2:

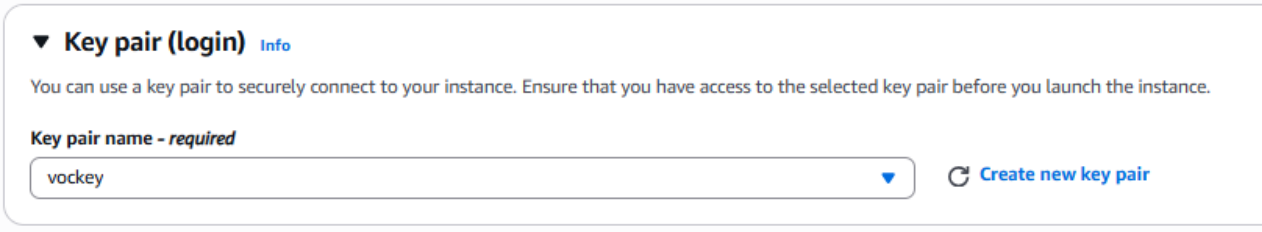
- Go to EC2 -> Launch Instance.



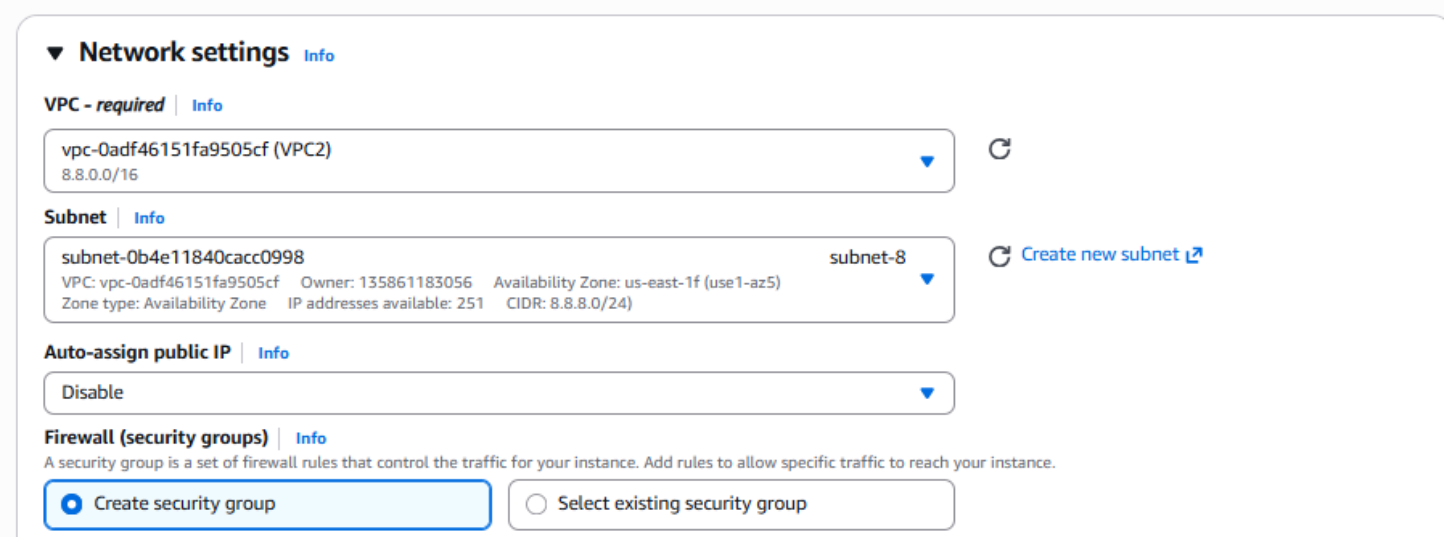
- Name: `EC2-8`. AMI: Amazon Linux 2. Type: t2.micro.



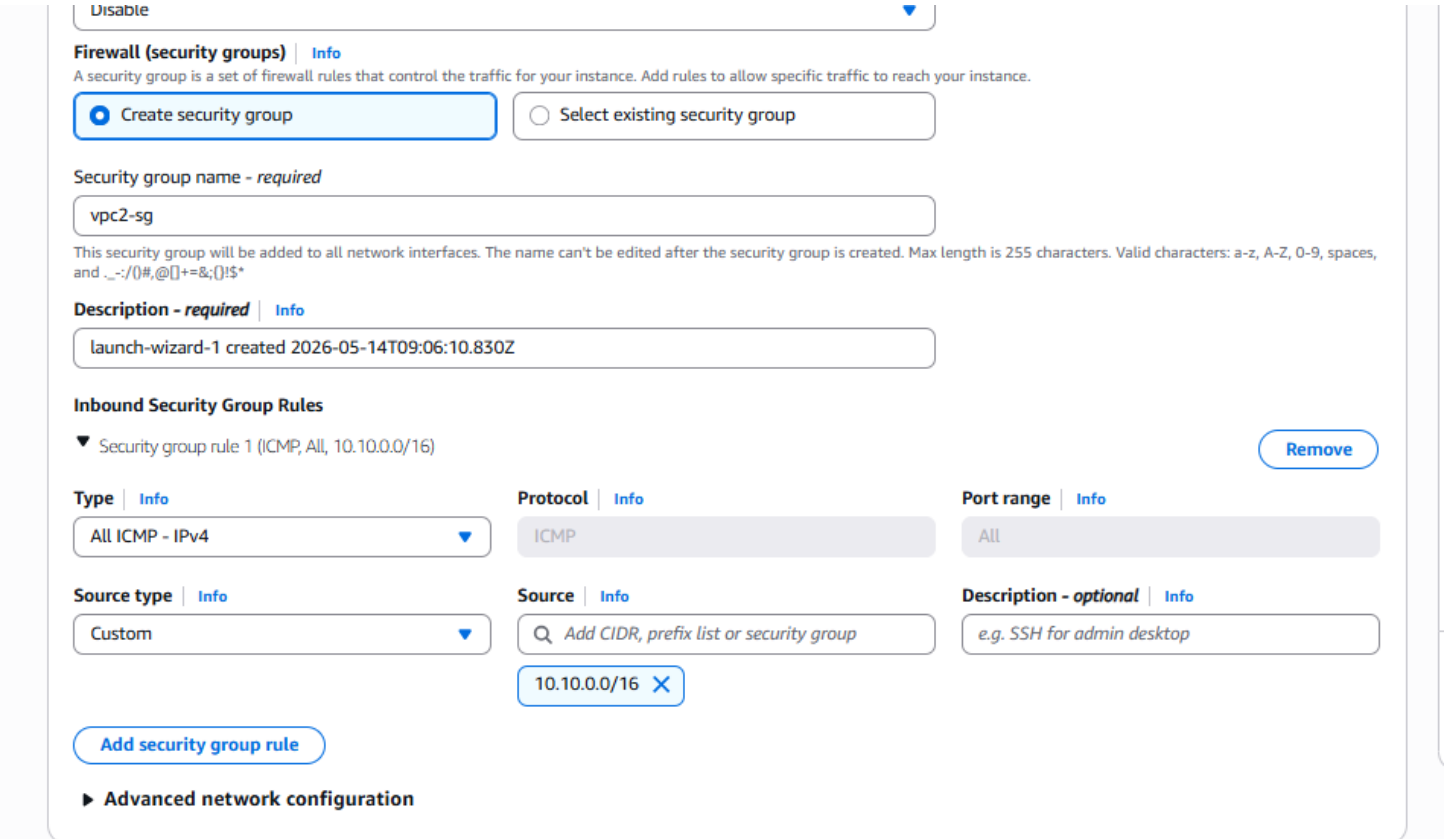
- Key pair: Select the same key pair as before.



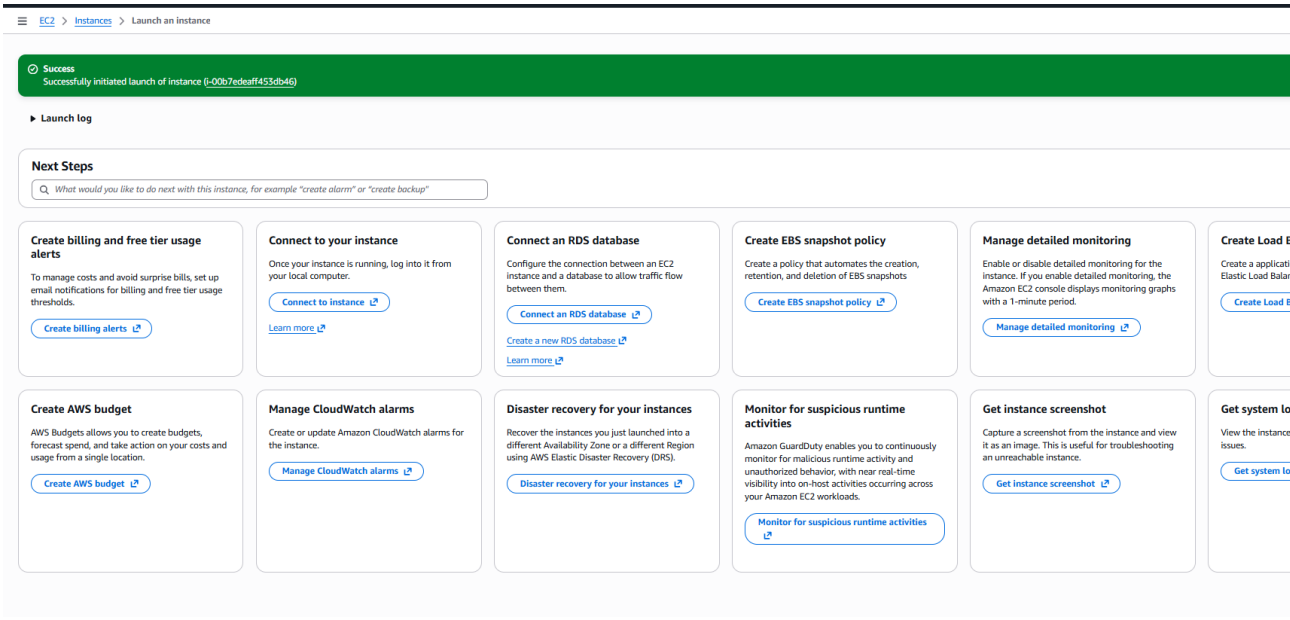
- Network settings (Edit):
  - VPC: VPC2.
  - Subnet: subnet-8.
  - Auto-assign public IP: Disable.



- Fir ewall (Security Groups):
  - Select Cr eate a new security group.
  - Security group name: `vpc2-sg`
  - Inbound rules: Add rule: Type All ICMP - IPv4, Source Custom`10.10.0.0/16` (VPC1's CIDR).

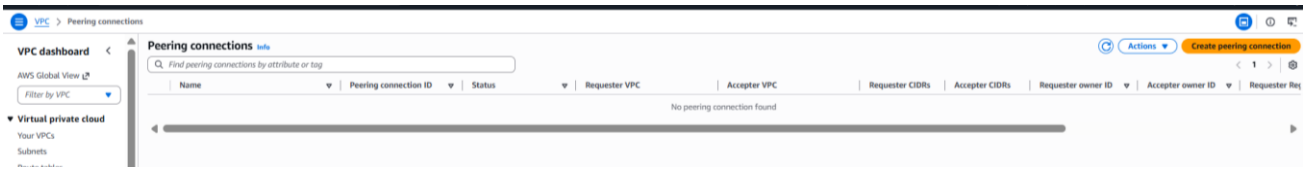


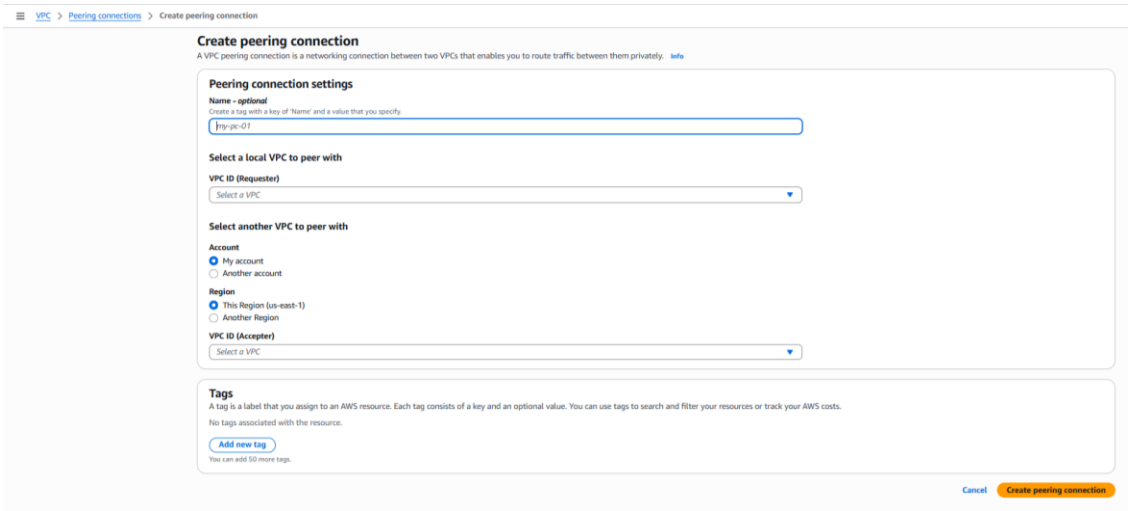
- Click Launch instance.



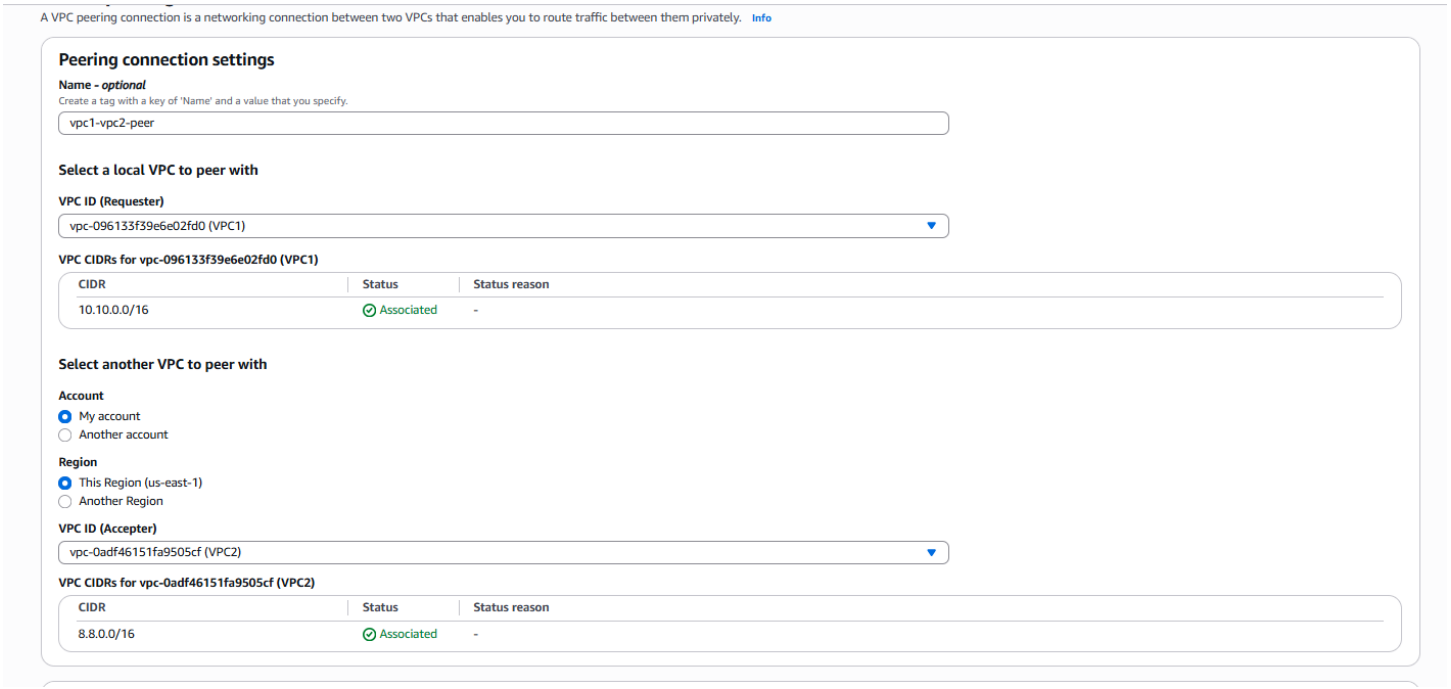
Cr eate Peering Connection:

- Go to the VPC service -> Peering connections -> Cr eate peering connection.

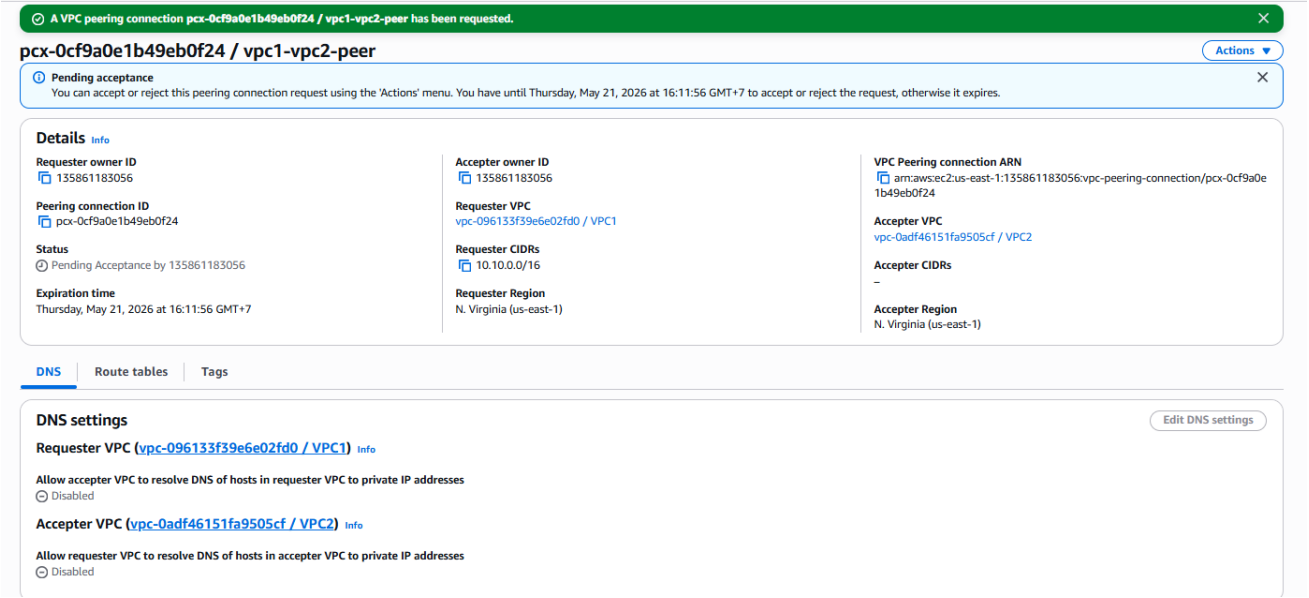




- Name: `vpc1-vpc2-peer.`
- Requester VPC: VPC1.
- Accepter VPC: VPC2.

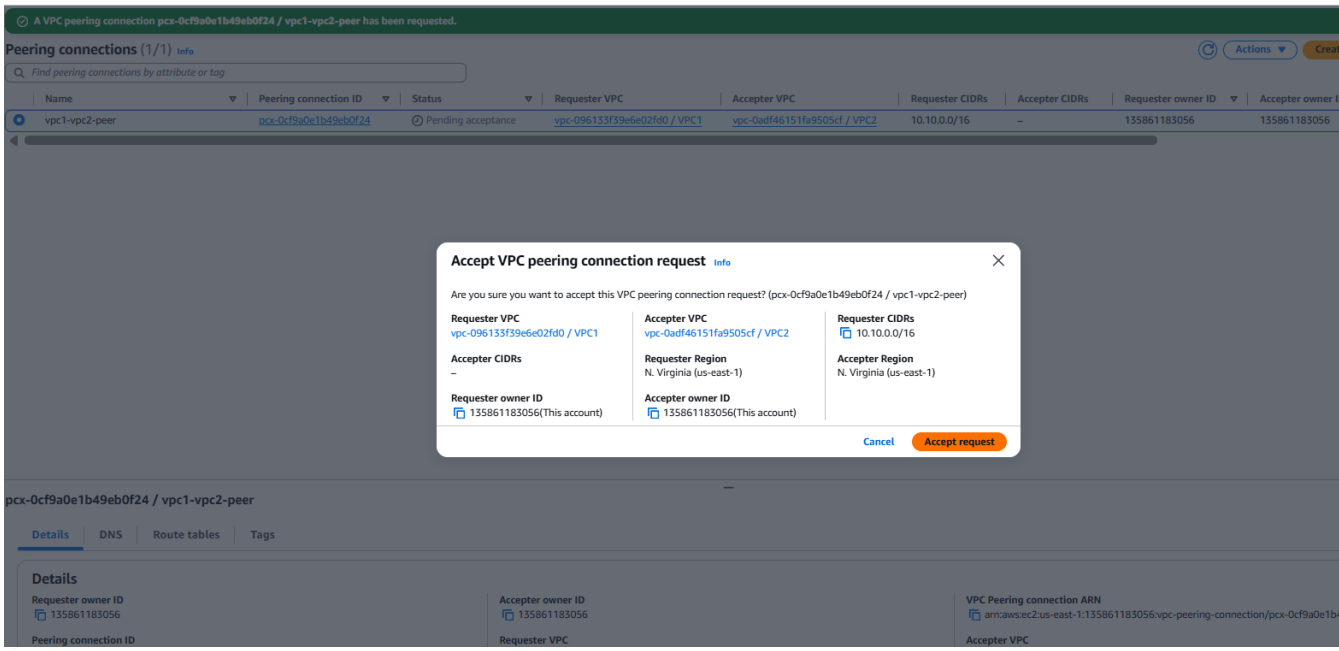


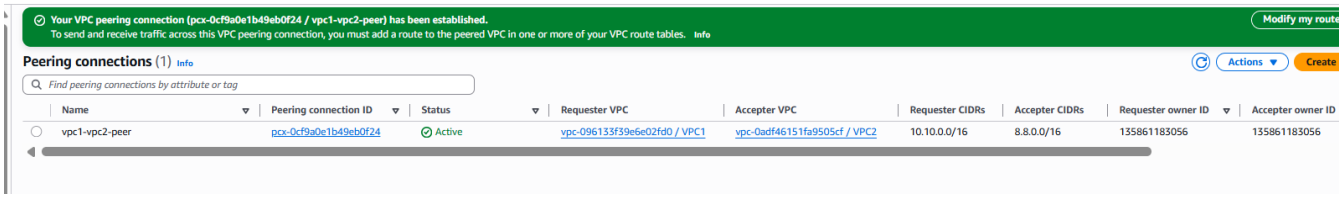
- Click Cr eate peering connection.



Accept Peering Connection:

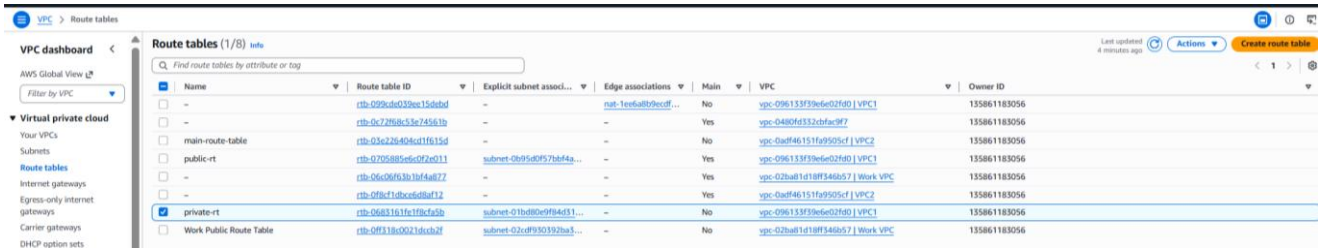
- Select the new "Pending acceptance" connection.
- Click Actions -> Accept r equest. Confirm.



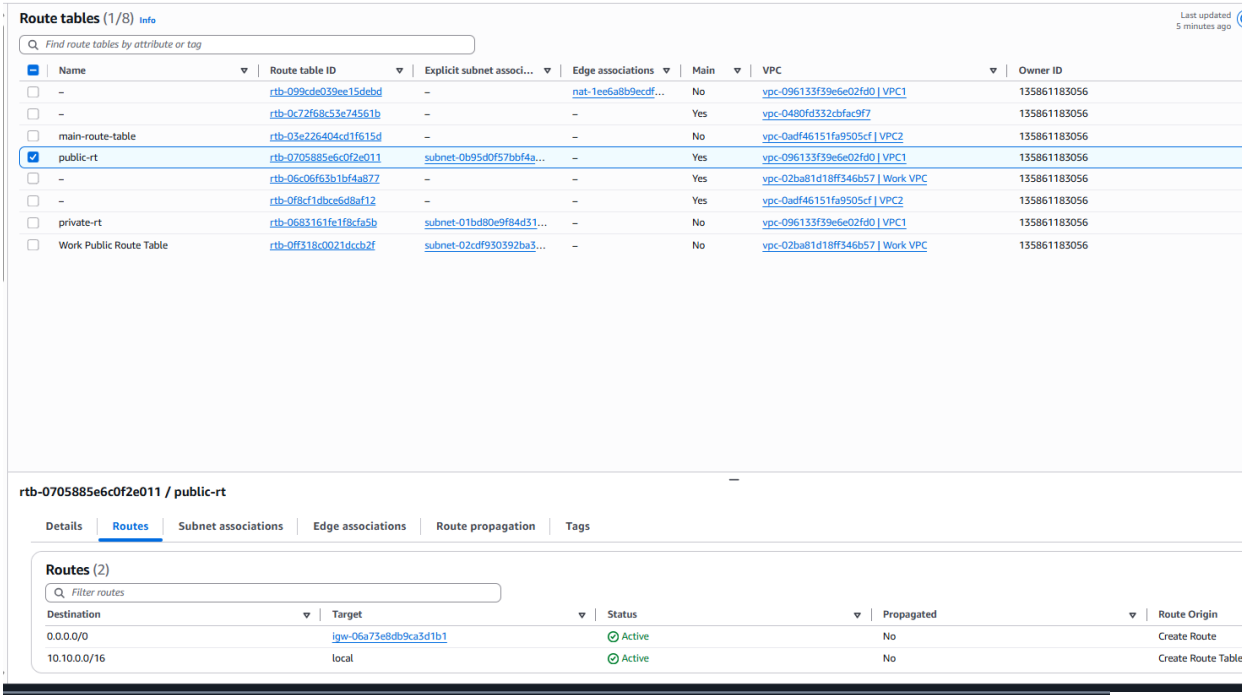


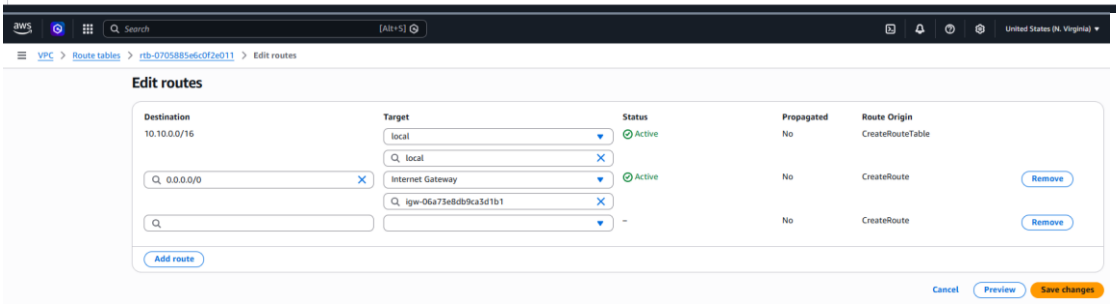
Update All Route Tables (Critical Step):

- Go to Route Tables.

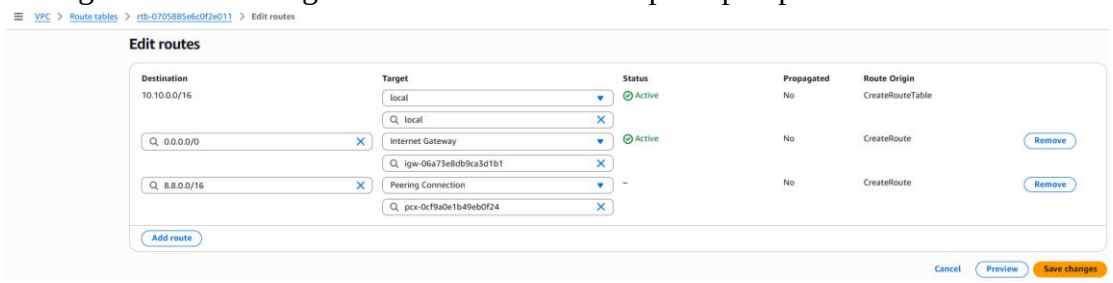


- For VPC1 (public-rt):
  - Select public-rt. Routes tab -> Edit routes -> Add route.





- Destination: `8.8.0.0/16` (VPC2's CIDR).
- Target: Select Peering Connection and choose vpc1-vpc2-peer. Save.



885e6c0f2e011

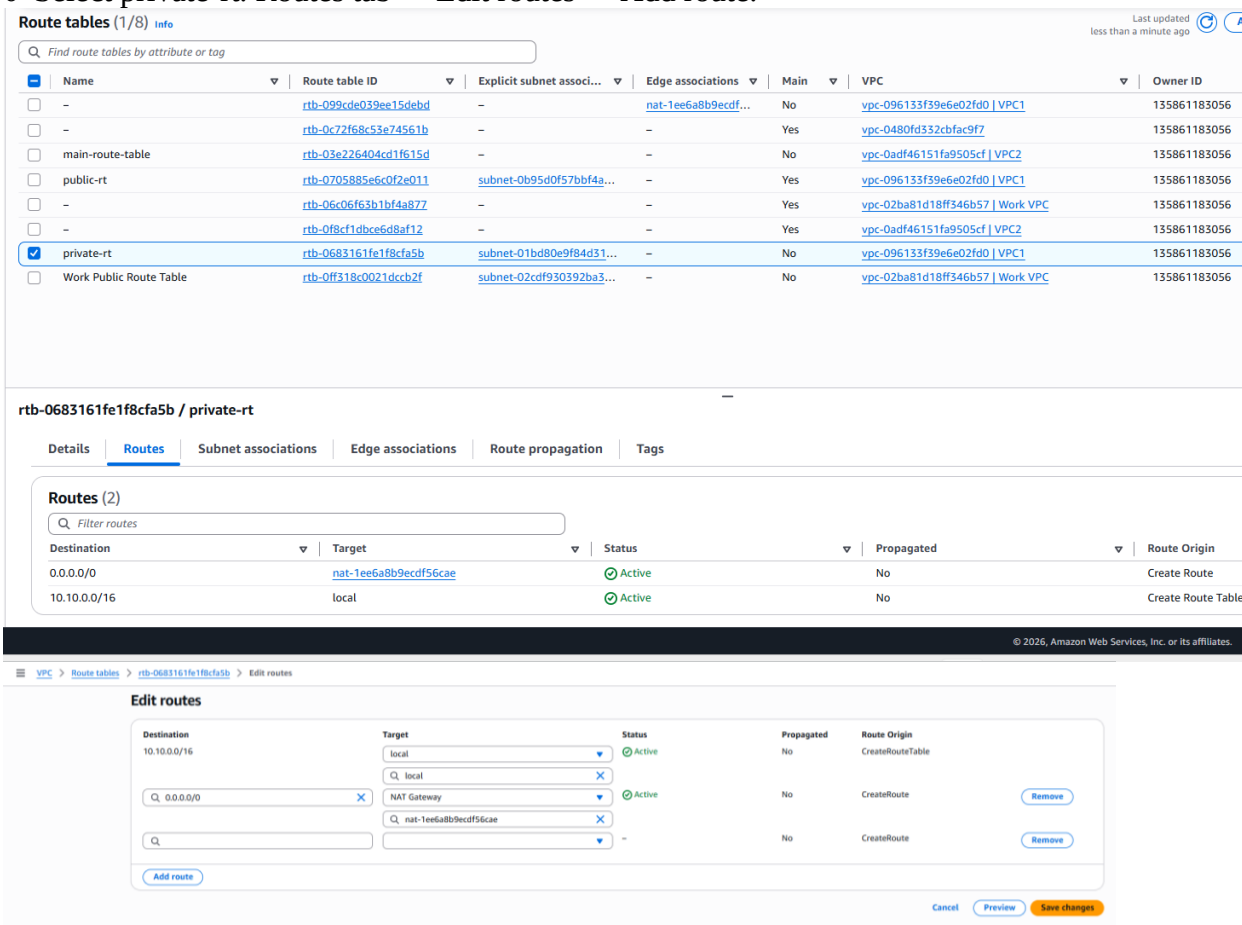
Updated routes for rtb-0705885e6c0f2e011 / public-rt successfully

Details

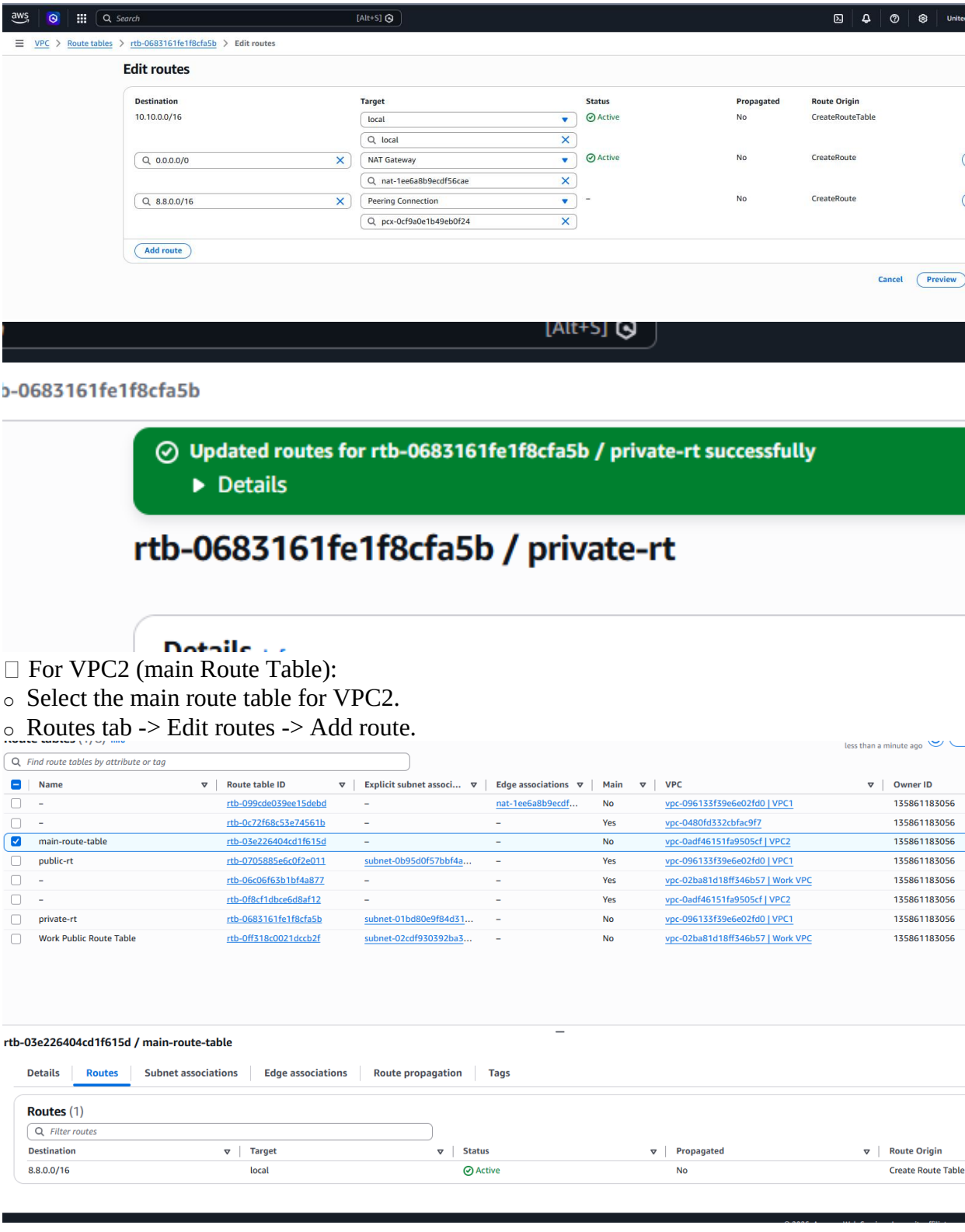
rtb-0705885e6c0f2e011 / public-rt

Details Info

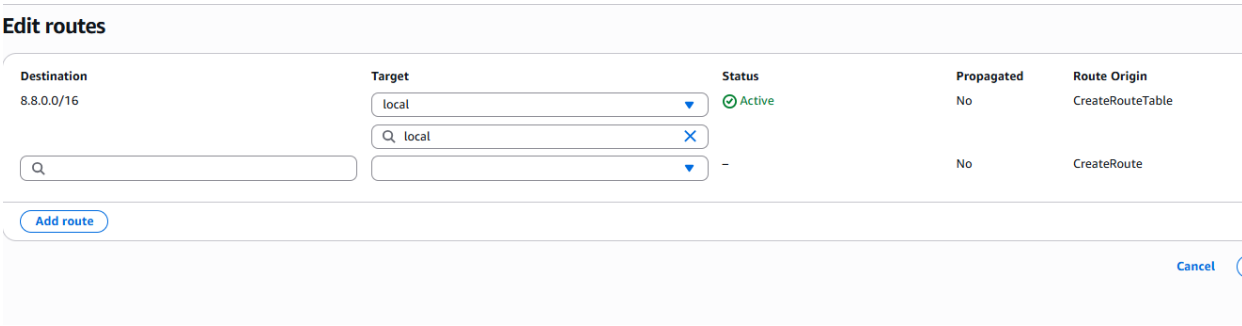
- For VPC1 (private-rt):
  - Select private-rt. Routes tab -> Edit routes -> Add route.



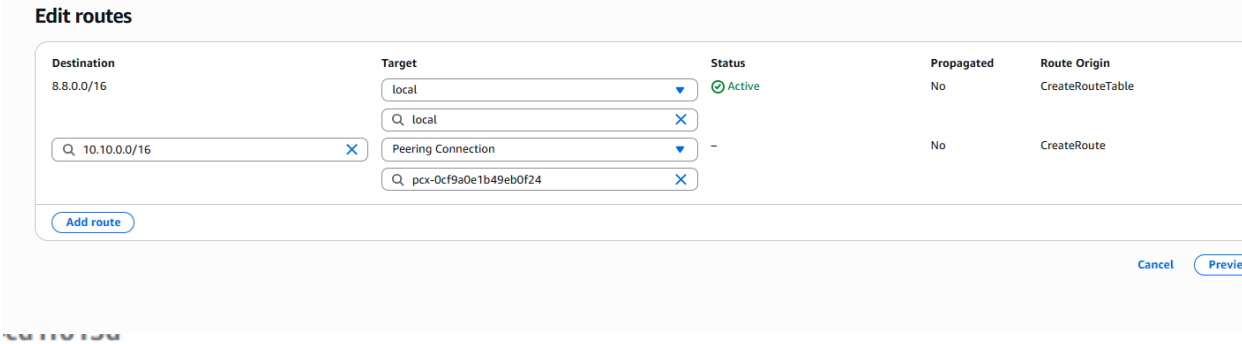
  - Destination: `8.8.0.0/16`.
  - Target: Select Peering Connection and choose vpc1-vpc2-peer. Save.

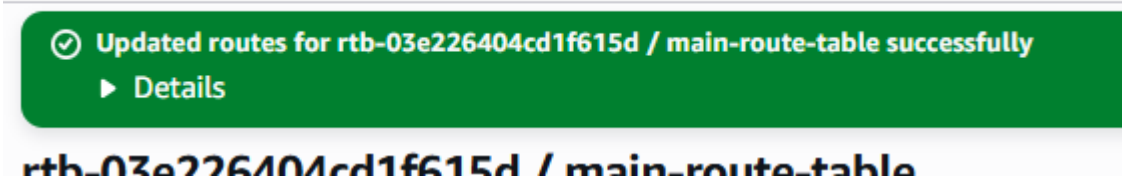


- For VPC2 (main Route Table):
  - Select the main route table for VPC2.
  - Routes tab -> Edit routes -> Add route.

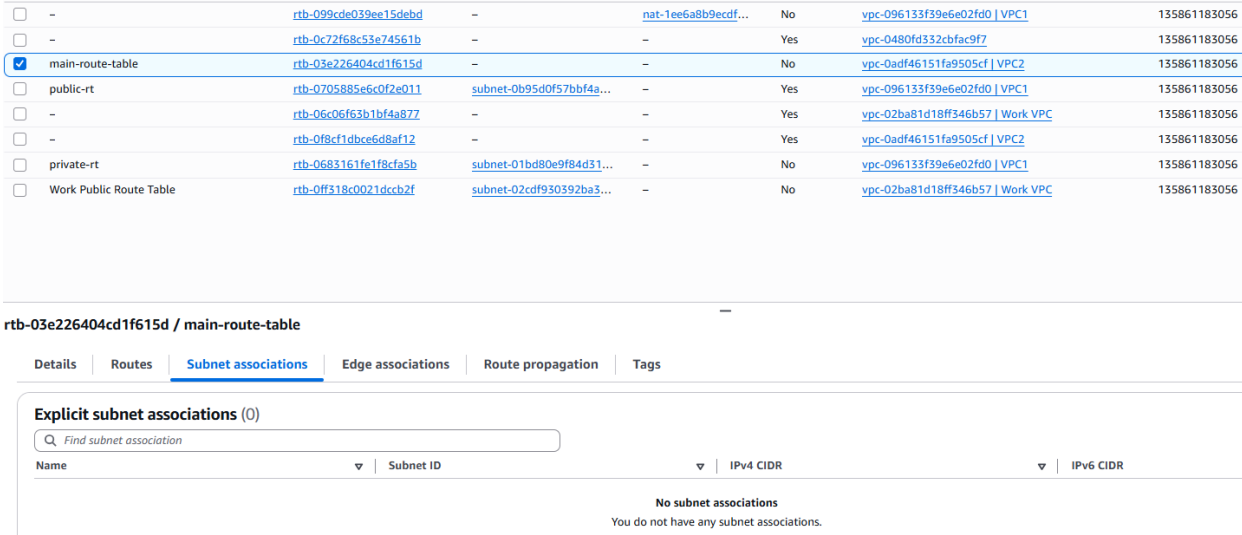


- Destination: `10.10.0.0/16` (VPC1's CIDR).
- Target: Select Peering Connection and choose vpc1-vpc2-peer. Save.

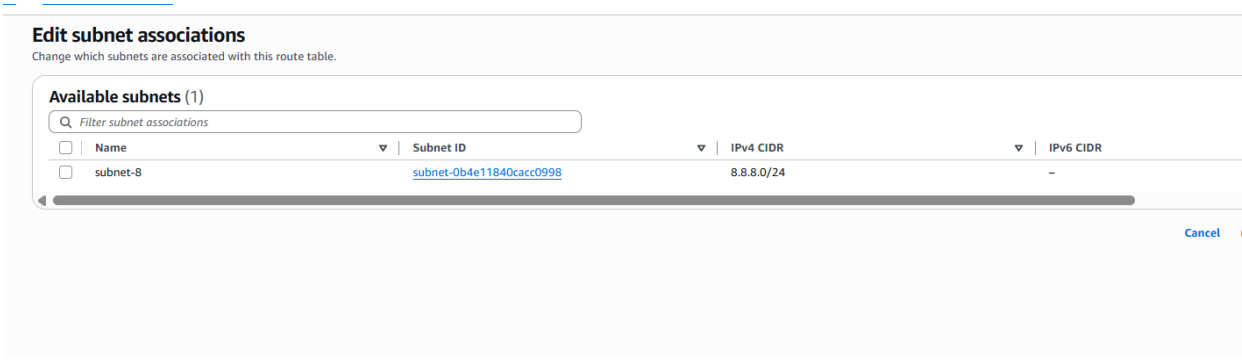




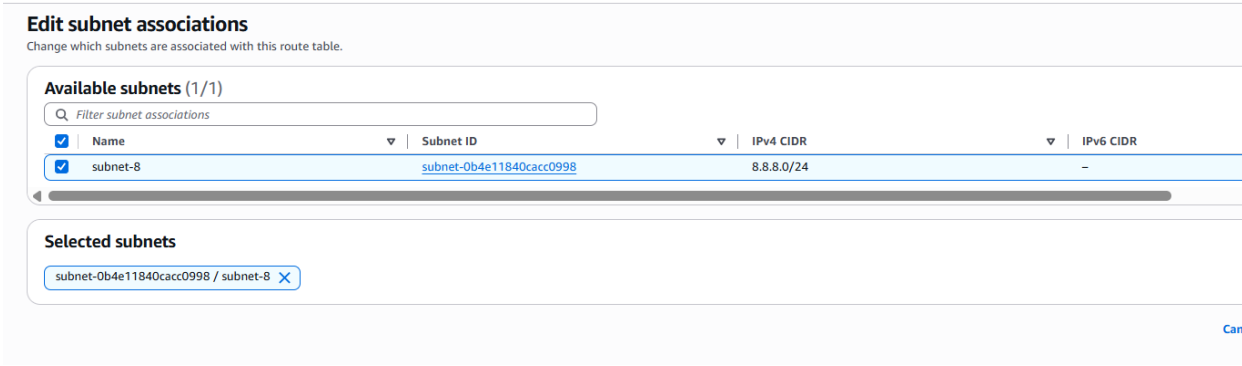
☐ Click the Subnet Associations tab for main-route-table.

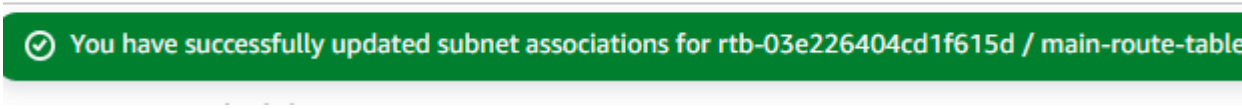


- Click Edit subnet associations.



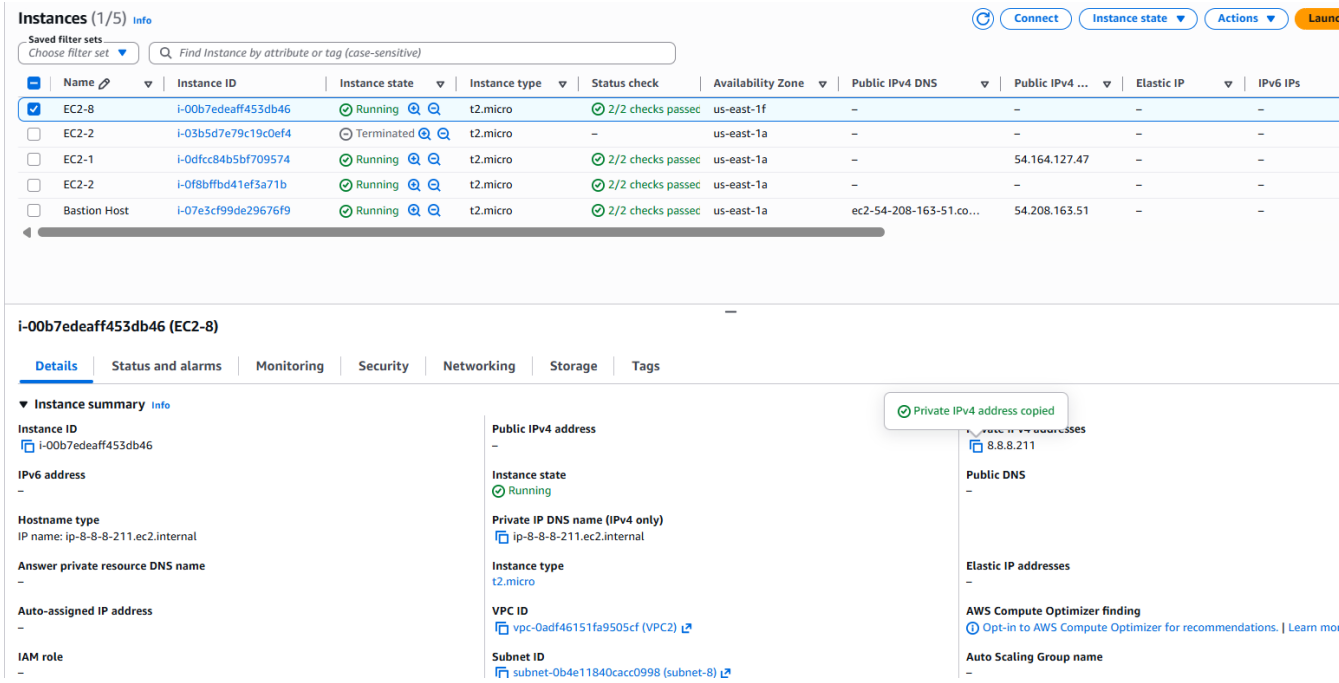
- Check the box for subnet-8 and click Save associations.

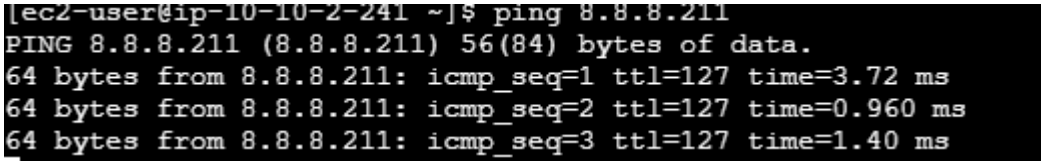




Test Peering:

- SSH into EC2-1 (in VPC1).
- Find the Private IPv4 addr ess of EC2-8 (e.g., 8.8.8.x).
- From EC2-1, ping EC2-8: ping <EC2-8-Private-IP>.
- The ping should be successful, proving the peering connection and routes are working.



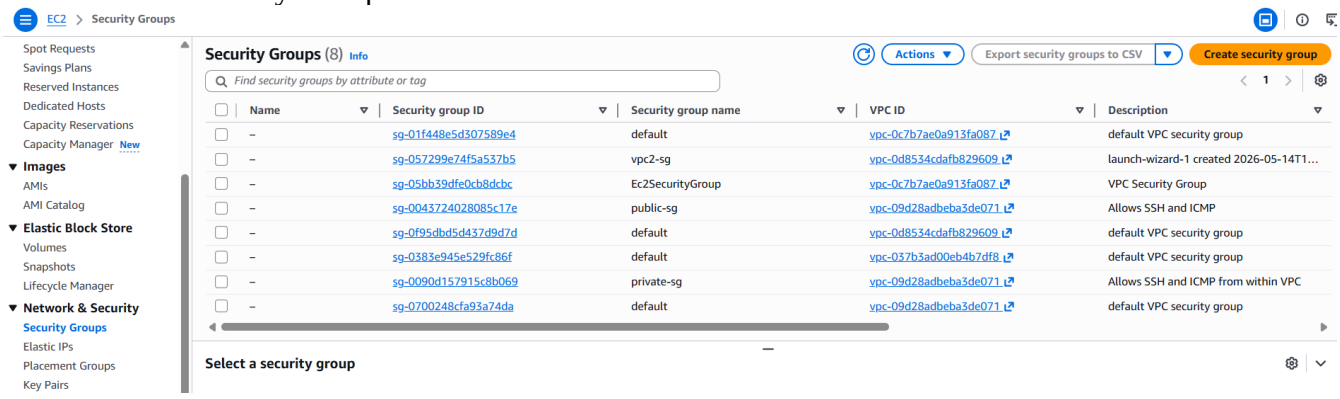


6.6. Task 6: Secur e the VPC (Fir ewalls and Endpoints): Harden your VPC by applyinglayeredsecurity (Security Groups and NACLs) and prepare for private S3 access.

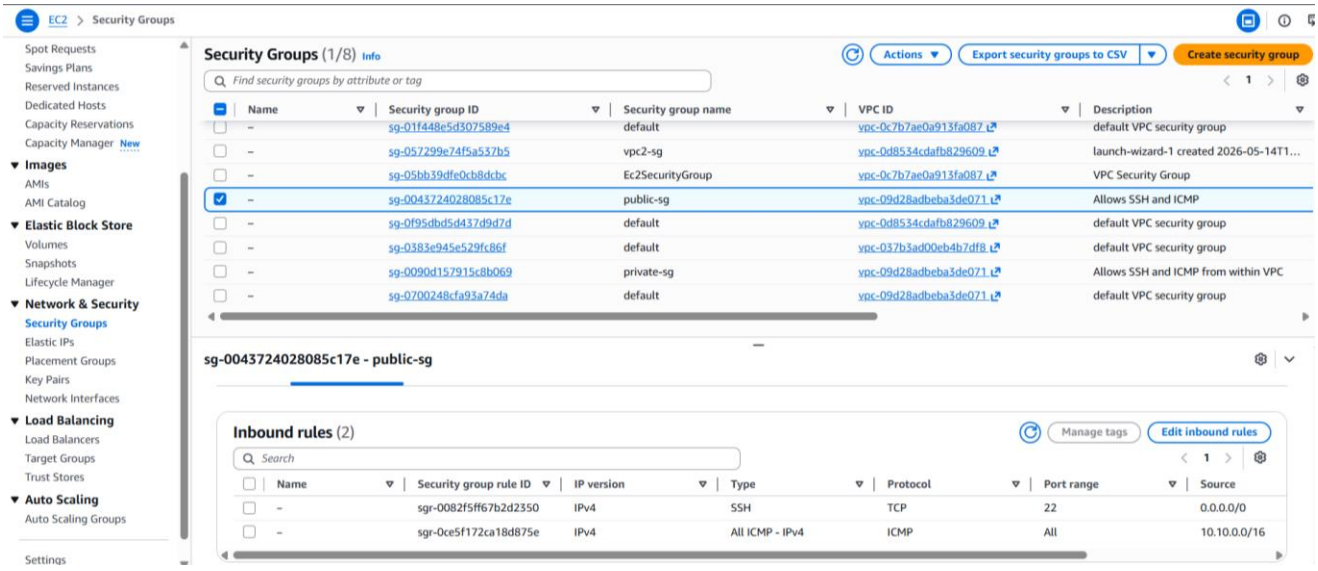
- Instructions:

  Refine Security Groups:

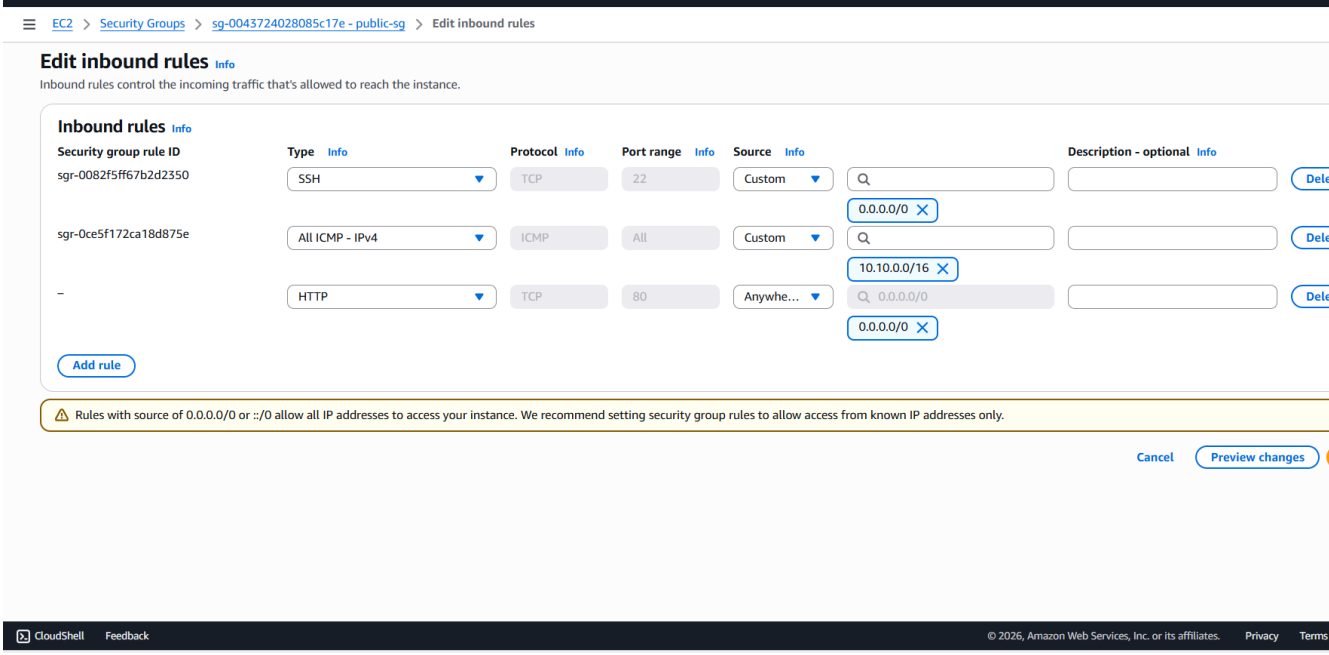
  Go to EC2 -> Security Groups.



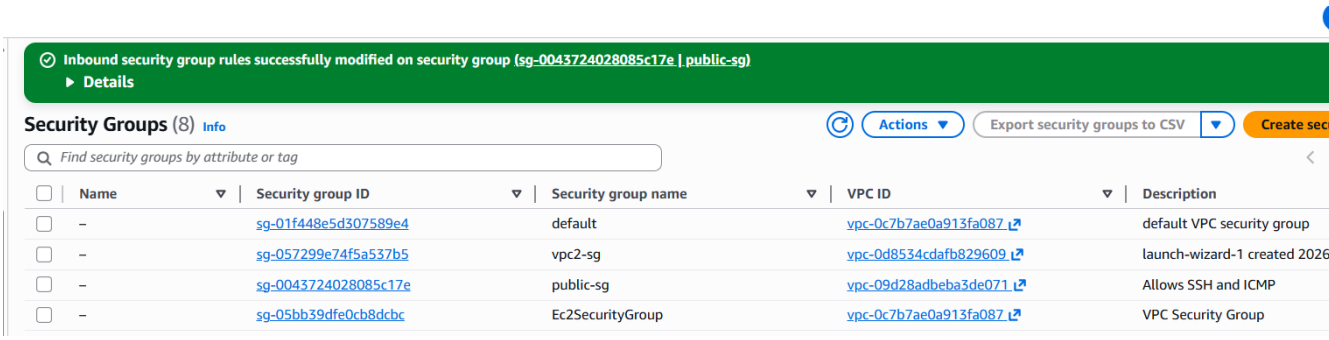
Select public-sg (for EC2-1). Click Edit inbound rules.



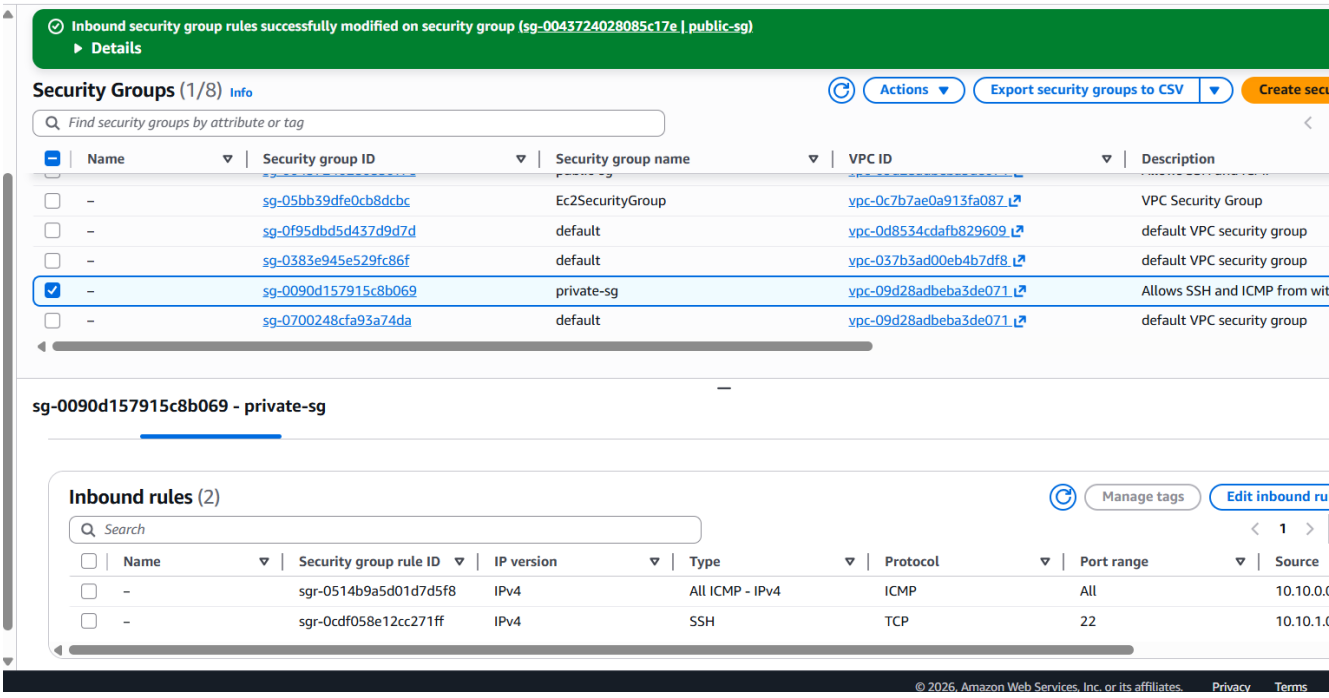
◆ Add a rule: Type HTTP, Source 0.0.0.0/0. (This allows web server traffic).



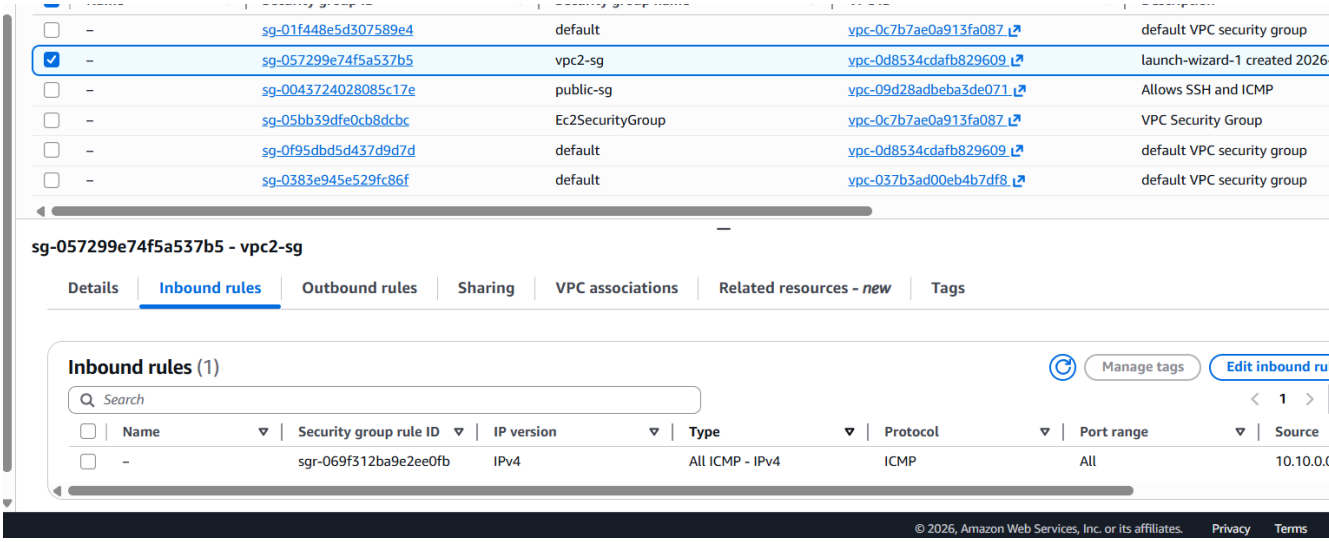
◆ Ensure your All ICMP - IPv4 rule also allows the peer VPC: Add 8.8.0.0/16as asource if not already covered.



◆ Select private-sg (for EC2-2).



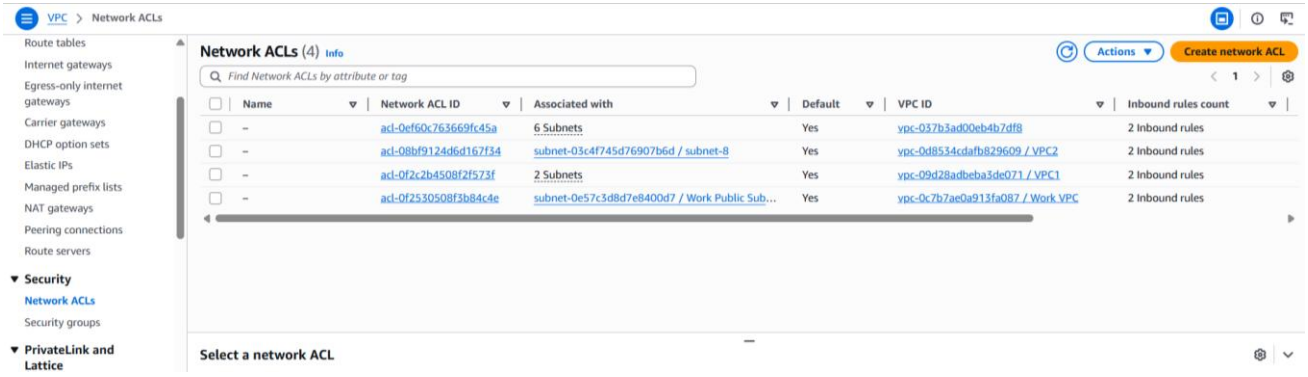
◆ Ensure its All ICMP - IPv4 rule also allows `8.8.0.0/16`.
◆ Select vpc2-sg (for EC2-8).



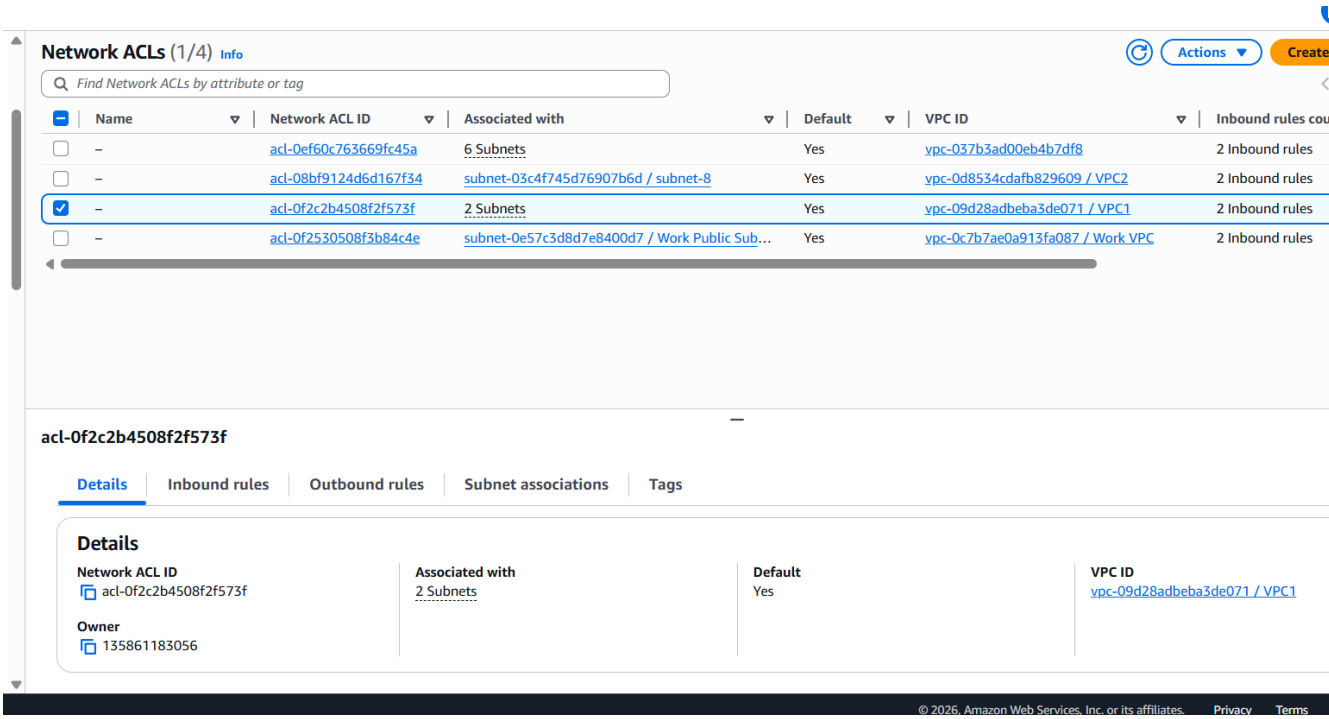
◆ Ensure its All ICMP - IPv4 rule allows 10.10.0.0/16.

Review Network ACLs (NACLs):

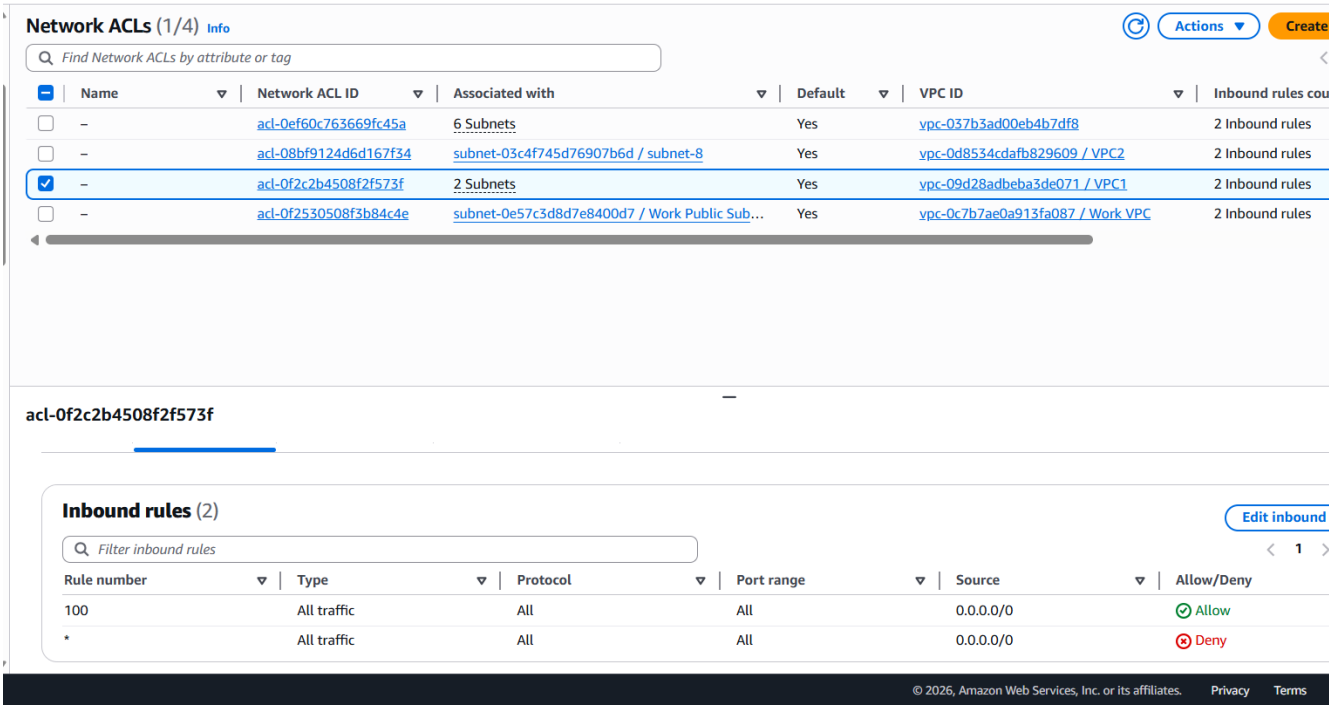
◆ Go to VPC -> Network ACLs.

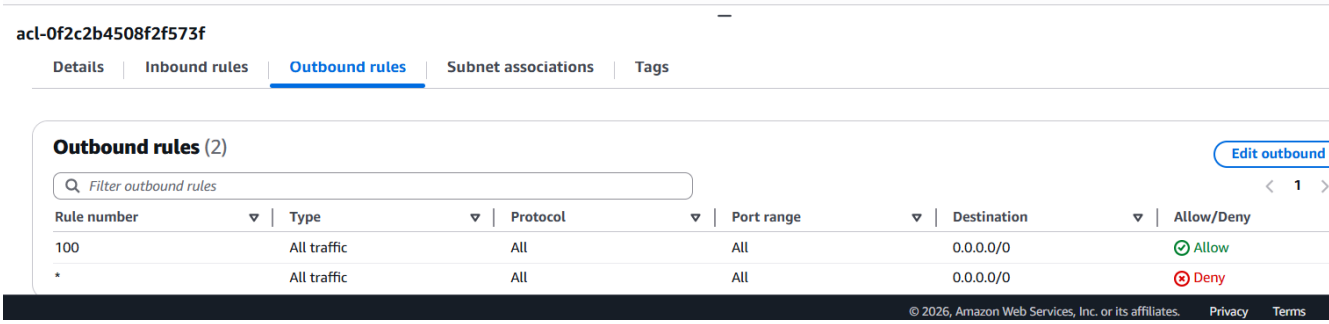


◆ Select the default NACL for VPC1.



◆ Review the Inbound Rules and Outbound Rules.



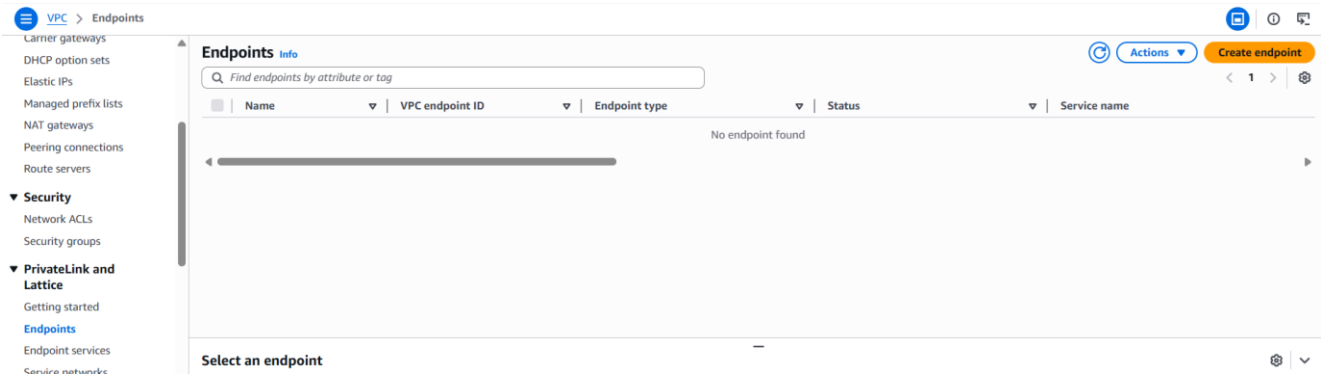


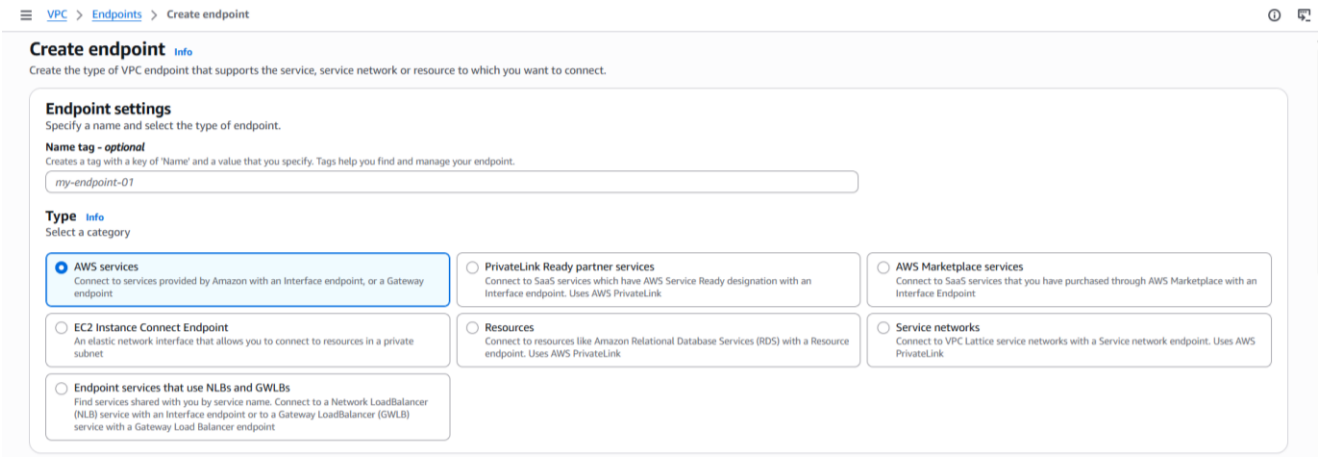
Observation: By default, the rules are 100 ALL Traffic ALLOWand * ALLTrafficDENY. This stateless configuration allows all traffic. In a real-world scenario, youwould tighten these rules to only allow specific ports (e.g., 22, 80, 443, ICMP) inbothdirections (inbound and outbound) and deny all else.

For this lab, you can leave the default NACLs as "ALLOWALL" and rely on your stateful Security Groups as the primary defense.

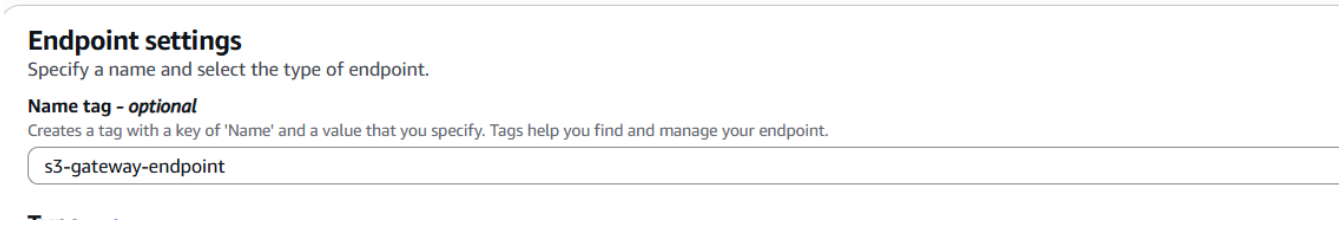
Create a VPC Endpoint for S3:

- Go to VPC -> Endpoints -> Create endpoint.

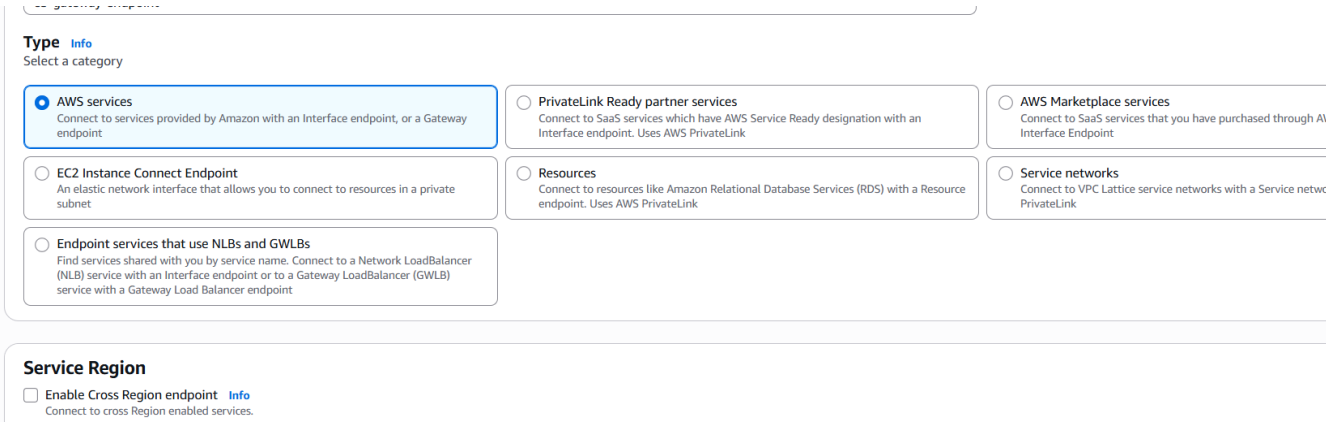




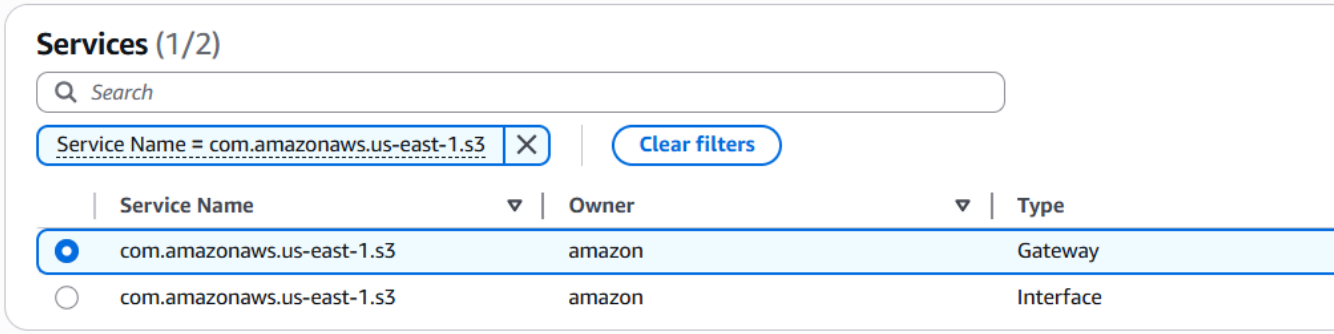
- Name: `s3-gateway-endpoint`



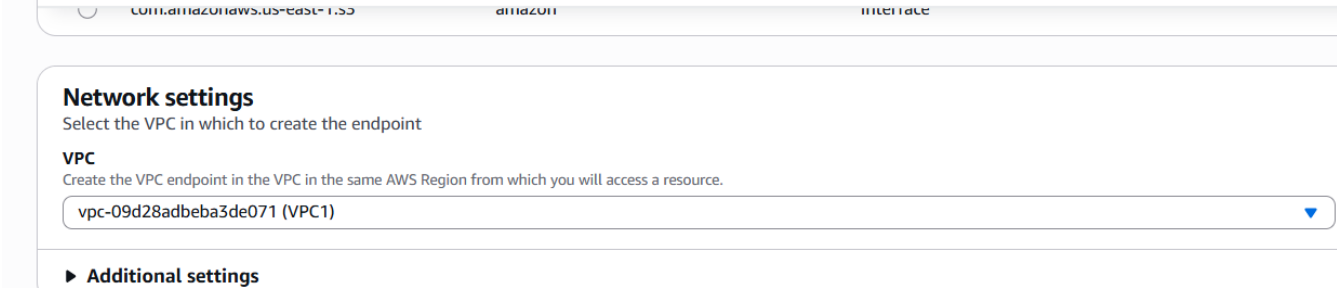
- Service category: AWS services.



- Search for "s3" and select the service with Type = Gateway. (It will look like com.amazonaws.[region].s3).



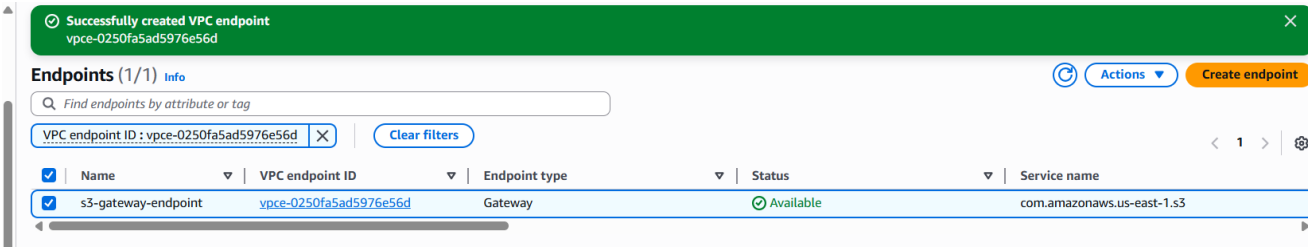
- VPC: Select VPC1.



 Route tables: Check the box for your private-rt (the one associated with private- subnet).



 Click Cr eate endpoint. This creates a private route from your private subnet directlyto S3.

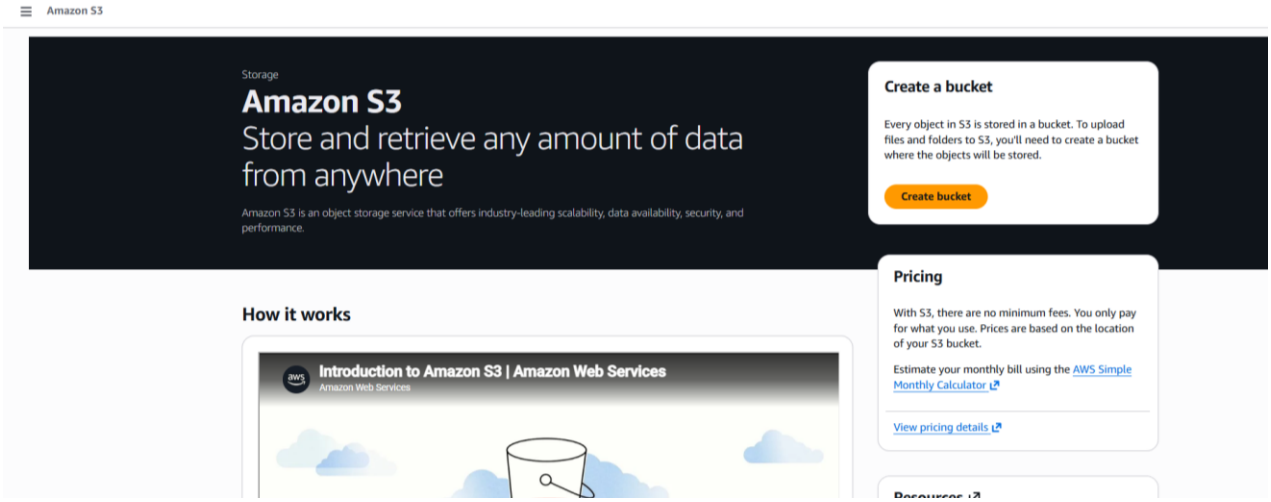


6.7. Task 7: Verify Private S3 Access through VPC Endpoint: Demonstrate secure, privateaccess to Amazon S3 from an isolated instance without using a NAT Gateway or the publicinternet.

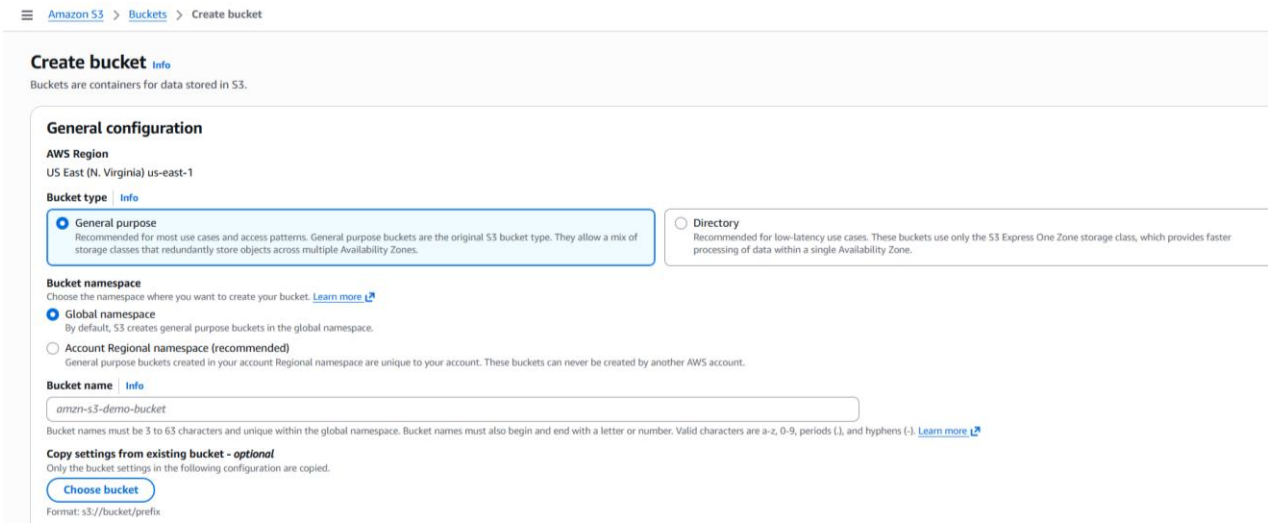
- Instructions:

  Cr eate an S3 Bucket:

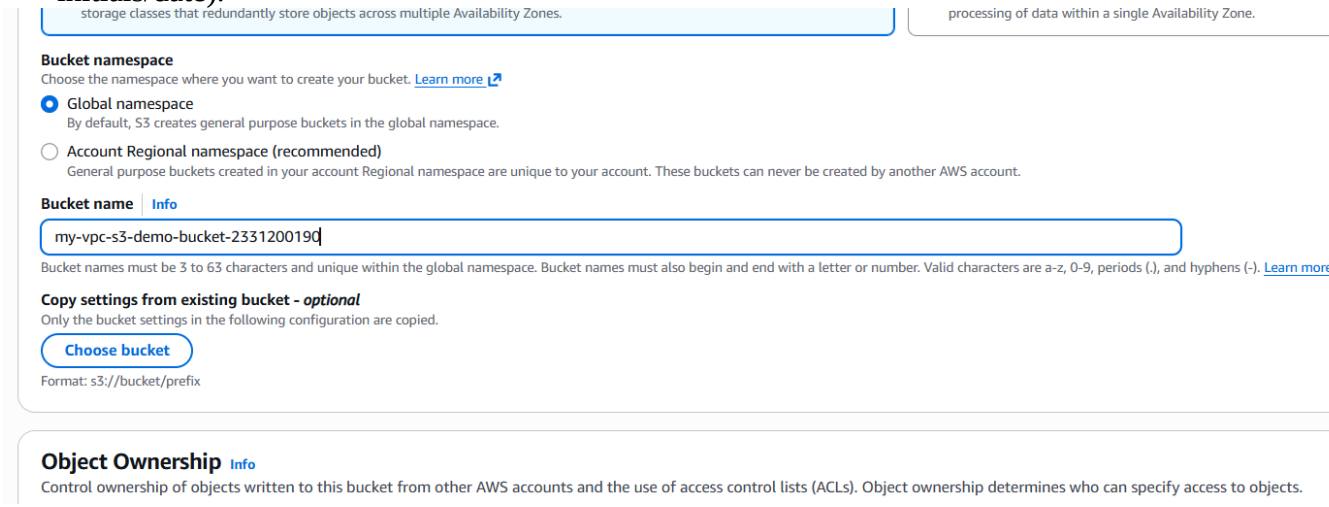
  - Navigate to the S3 service.



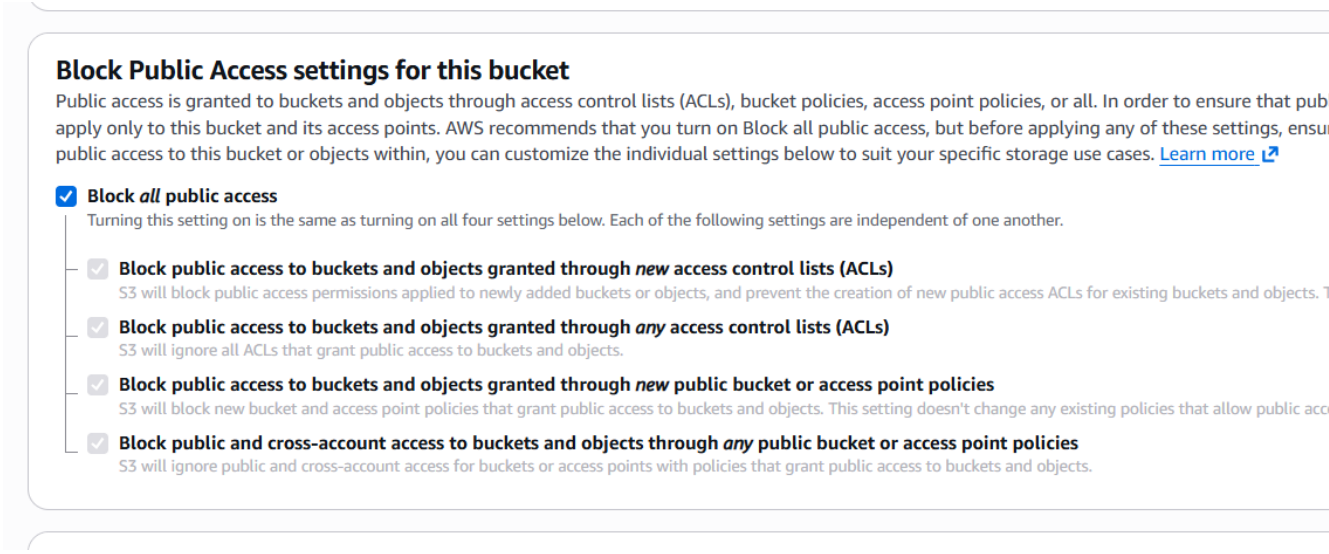
  - Click Cr eate bucket.



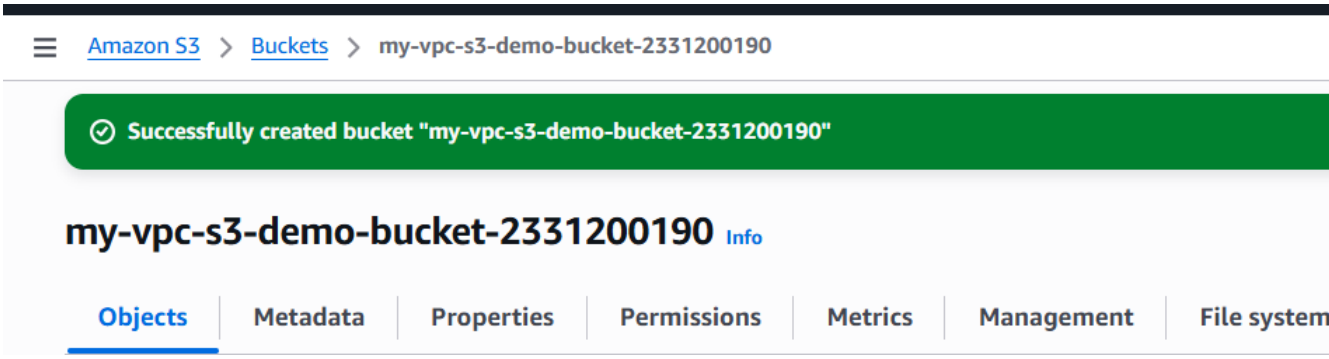
- Bucket name: `my-vpc-s3-demo-bucket-[your-unique-id]` (e.g., addyour initials/date).



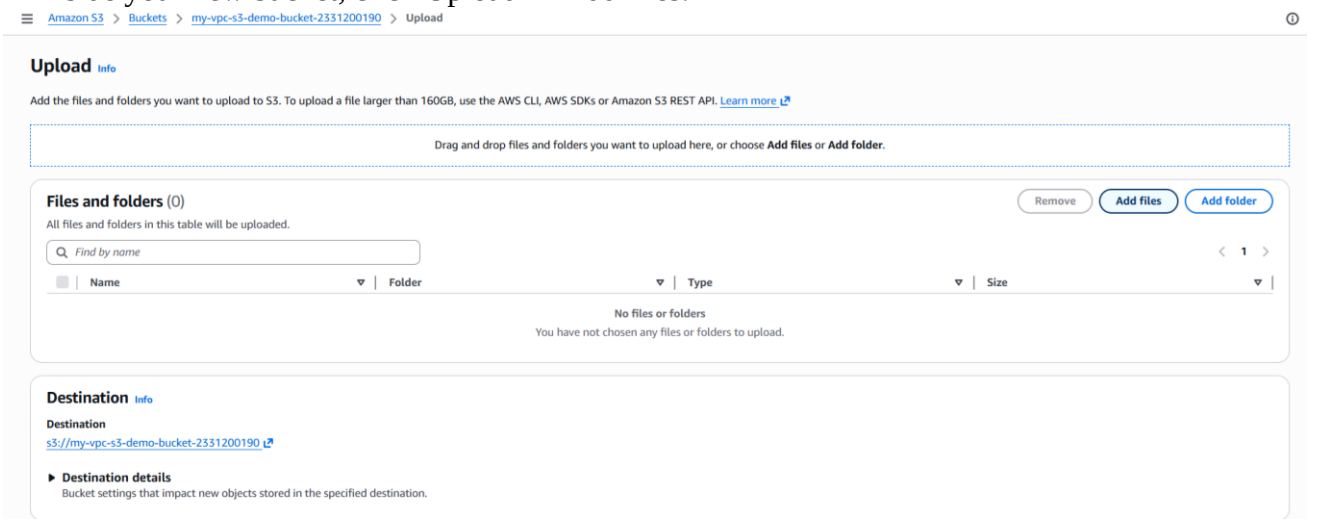
- Ensure Block all public access remains checked.



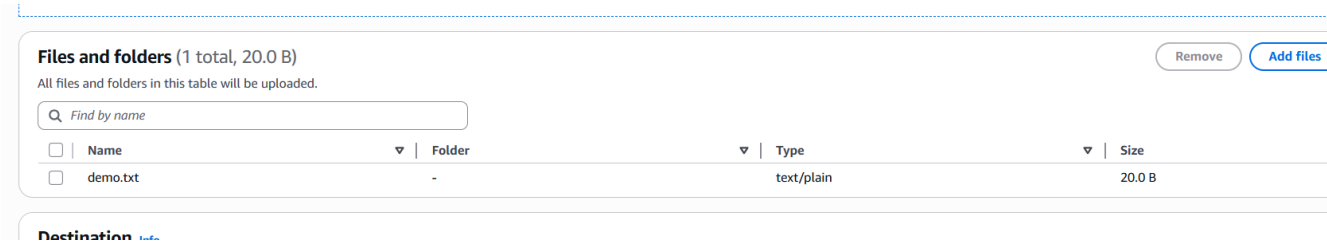
◆ Click Cr eate bucket.



◆ Inside your new bucket, click Upload -> Add files.

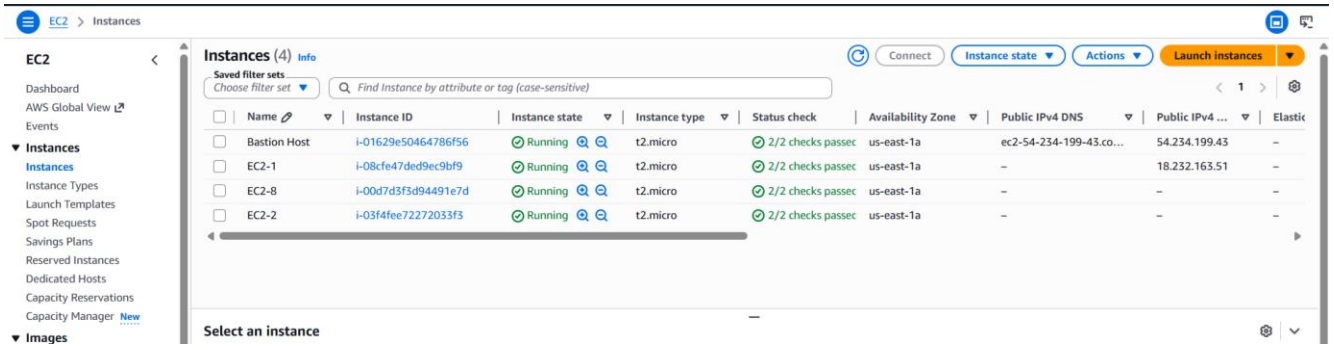


◆ Create a simple file named demo.txt or index.html on your computer and uploadit.

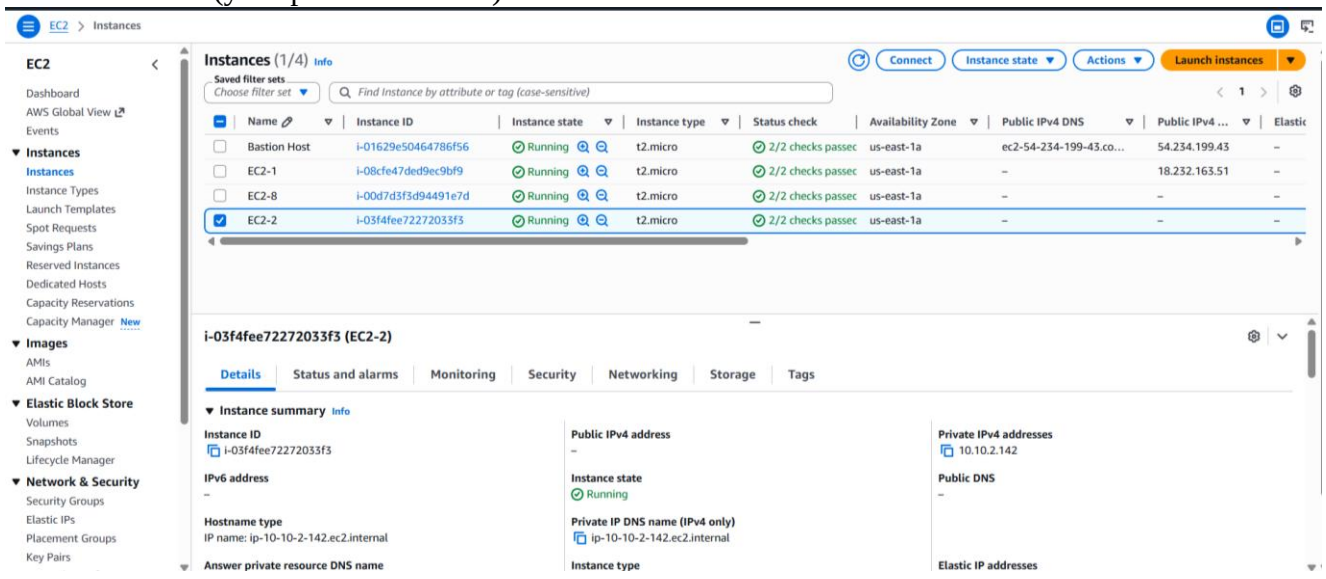


Attach Role to EC2-2:

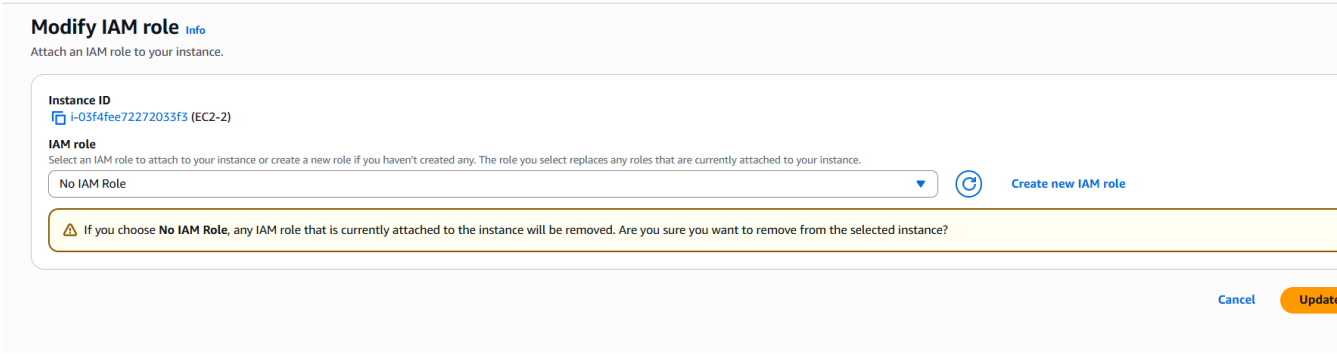
□ Go back to the EC2 service -> Instances.



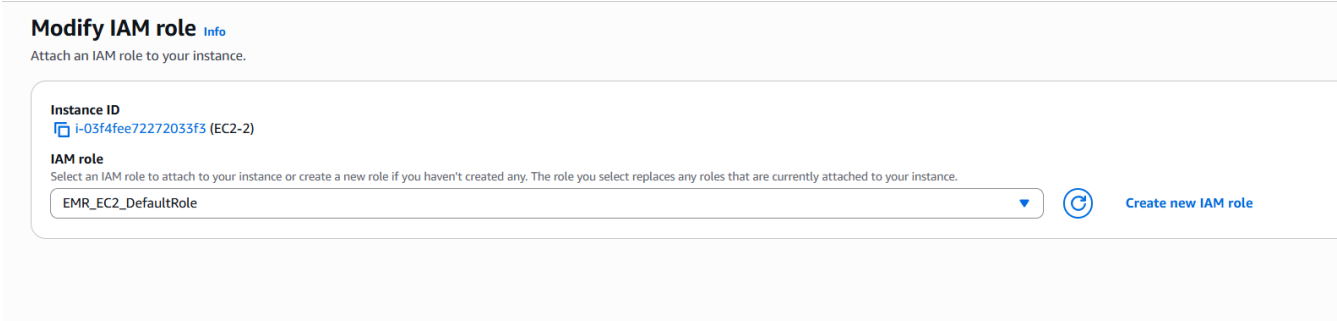
□ Select EC2-2 (your private instance).



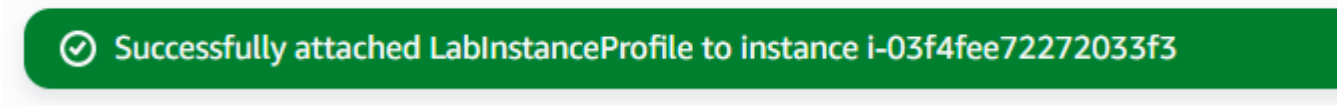
□ Click Actions -> Security -> Modify IAM role.



- Choose existing EMR_EC2_DeafulRrole from the dropdown.

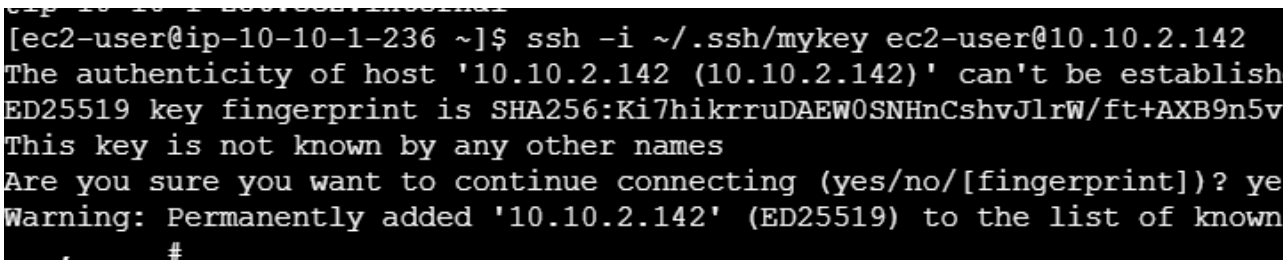


- Click Update IAM role. (It may take a moment to apply).



Test S3 Access (Success):

- SSH into EC2-1 and then "hop" into EC2-2 (as you did in Task 4).



- On EC2-2, run the following AWS CLI command:
    - `aws s3 ls`

```
[ec2-user@ip-10-10-2-142 ~]$ aws s3 ls
2026-05-14 12:49:46 my-vpc-s3-demo-bucket-2331200190
```

- You should see a list of your S3 buckets, including my-vpc-s3-demo-bucket....
- Now, list the contents of your bucket:
  - `aws s3 ls s3://my-vpc-s3-demo-bucket-[your unique-id]`

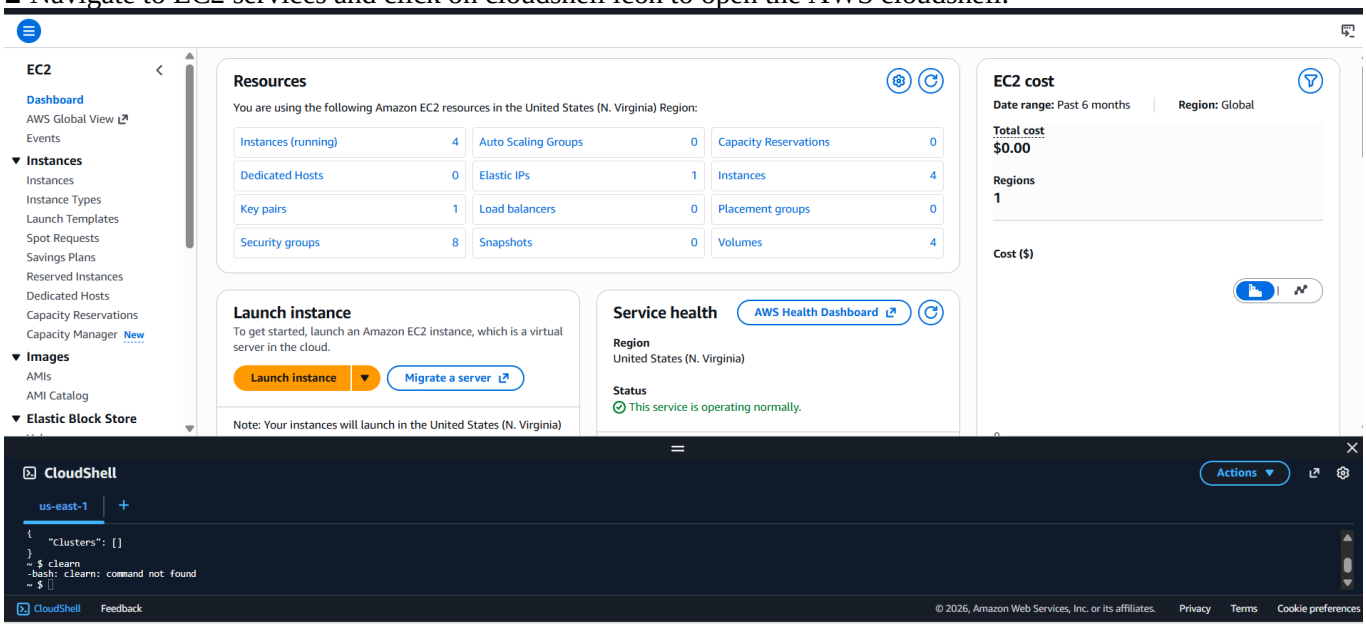
```
2026-05-14 12:49:46 my-vpc-s3-demo-bucket-2331200190
[ec2-user@ip-10-10-2-142 ~]$ aws s3 ls s3://my-vpc-s3-demo-bucket-2331200190
2026-05-14 12:52:08         20 demo.txt
```

- You should see demo.txt listed. This request worked privately through the VPCEndpoint.

- Remember to delete all resources you have created so far to avoid incurring cost.

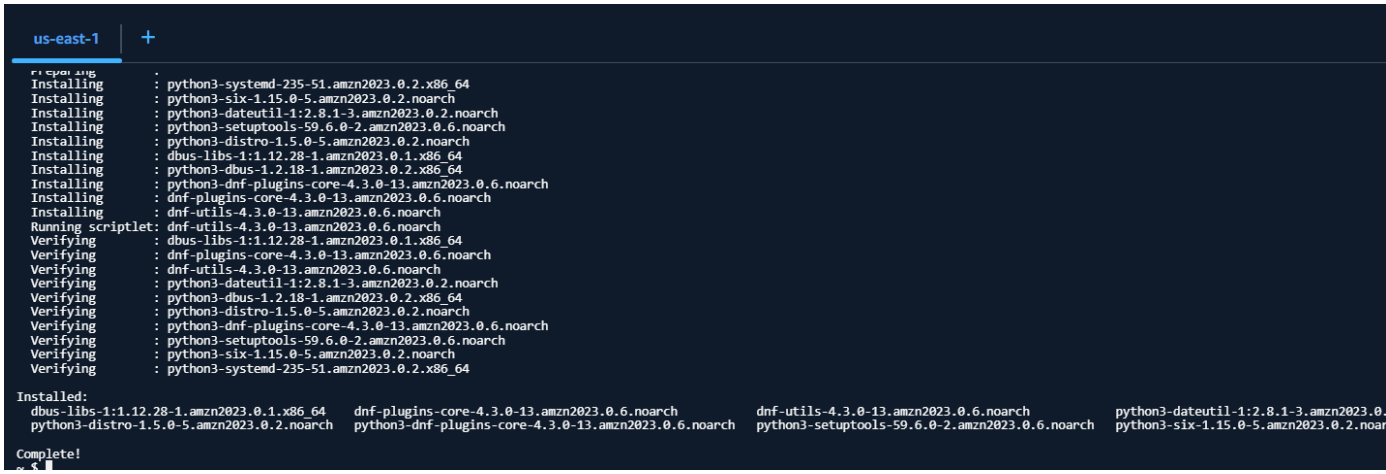
6.8. Task 8: Deploy VPC using Terr aform: Understand how to deploy a full-stack environment (VPC, subnets, gateways, and instances) automatically using Infrastructure as Code (IaC) with Terraform. This task replicates the manual work from Tasks 1 to 4 using a declarative script.

- Instructions:
  - Navigate to EC2 services and click on cloudshell icon to open the AWS cloudshell.



  - Prerequisite - Install Terraform through AWS cloudshell by following command.

    - `sudo yum install -y yum-utils`



- sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo



- sudo yum install terraform -y

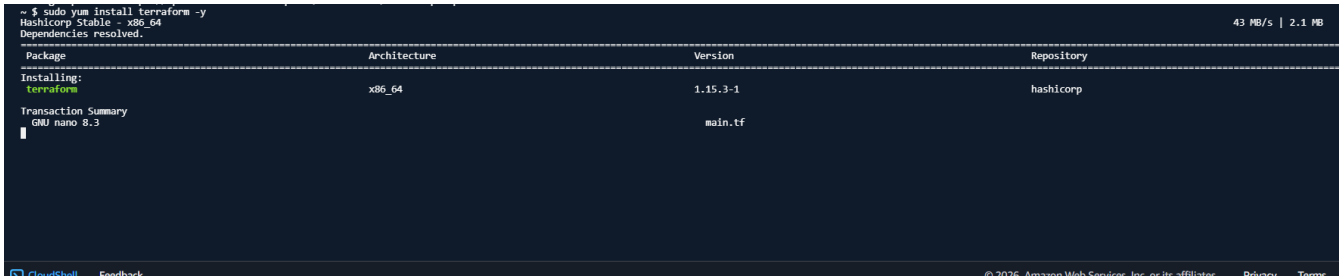


- terraform version



■ Prepare Project Directory:

- Create a new folder for your project (e.g., terraform-vpc-lab).
- Inside this folder, create the files: main.tf by following command
  - nano main.tf



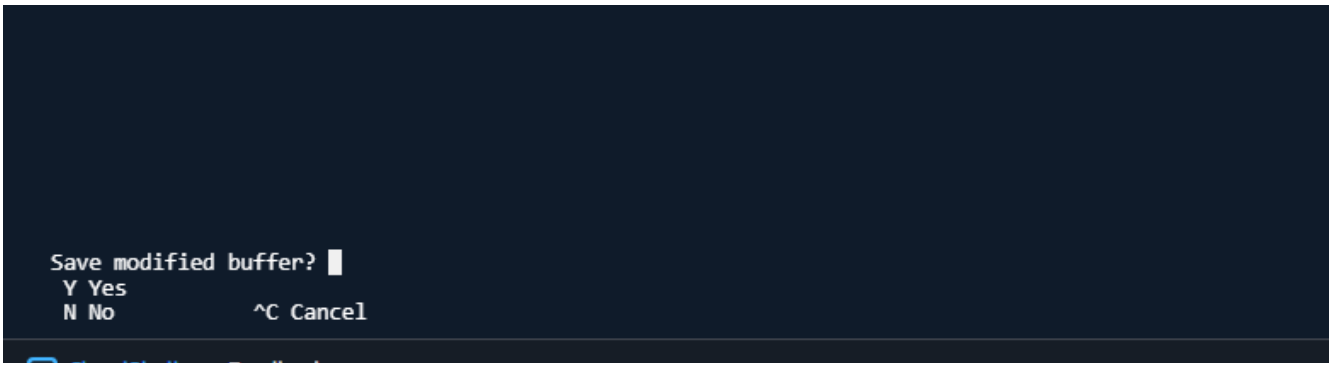
 Copy the following content to your main.tf file (this is the file you need to edit):

```
provider "aws" {
region = "us-east-1"
}
resource "aws_vpc" "test" {
cidr_block = "10.10.0.0/16"
tags = {
Name = "VPC1"
   }
}
                              resource "aws_subnet" "public" {
vpc_id = aws_vpc.test.id
cidr_block = "10.10.1.0/24"
            availability_zone = "us-east-1a"
                                    map_public_ip_on_launch = true
tags = {
Name = "public-subnet"
  }
}
```

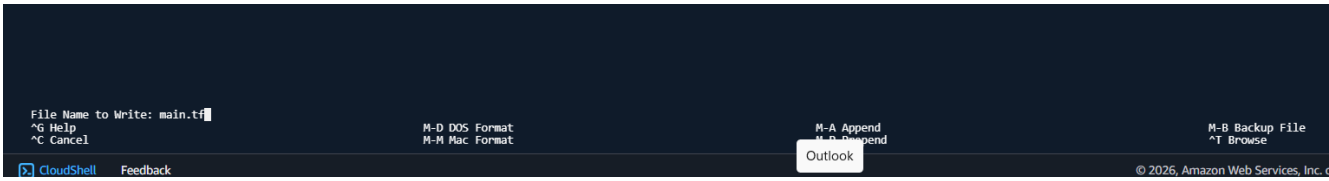
```
terraform

Transaction Summary
  GNU nano 8.3
  GNU nano 8.3
provider "aws" {
region = "us-east-1"
}
resource "aws_vpc" "test" {
cidr_block = "10.10.0.0/16"
tags = {
Name = "VPC1"
}
}
resource "aws_subnet" "public" {
vpc_id = aws_vpc.test.id
cidr_block = "10.10.1.0/24"
availability_zone = "us-east-1a"
map_public_ip_on_launch = true
tags = {
Name = "public-subnet"
}
}
```

- Then press `Ctrl+Z`.

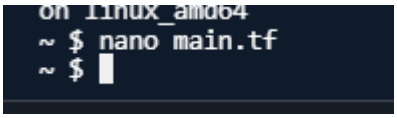


- Then will ask to save.
- Input `Y` for yes.



- Press Enter

- The `main.tf` file will saved.



■ Initialize Terr aform: Run the following command to download the necessary AWSprovider plugins:

- `terraform fmt`

```
~ $ nano main.tf
~ $ terraform fmt
main.tf
```

- `terraform init`

```
main.tf
~ $ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.45.0...
- Installed hashicorp/aws v6.45.0 (signed by HashiCorp)

Initializing the backend...


Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
~ $
```

- `terraform validate`

```
commands will detect it and remind you to d
~ $ terraform validate
Success! The configuration is valid.

~ $
```

■ Plan the Deployment: Run the plan command. This will check your script and showyouall the resources (VPC, subnets, instances, etc.) that will be created.

- `terraform plan`

```
~ $ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbol
  + create

Terraform will perform the following actions:

  # aws_subnet.public will be created
  + resource "aws_subnet" "public" {
      + arn                                            = (known after apply)
      + assign_ipv6_address_on_creation                = false
      + availability_zone                              = "us-east-1a"
      + availability_zone_id                           = (known after apply)
      + cidr_block                                     = "10.10.1.0/24"
      + enable_dns64                                   = false
      + enable_resource_name_dns_a_record_on_launch    = false
      + enable_resource_name_dns_aaaa_record_on_launch = false
      + id                                             = (known after apply)
      + ipv6_cidr_block                                = (known after apply)
      + ipv6_cidr_block_association_id                 = (known after apply)
      + ipv6_native                                    = false
      + map_public_ip_on_launch                        = true
      + owner_id                                       = (known after apply)
      + private_dns_hostname_type_on_launch            = (known after apply)
      + region                                         = "us-east-1"
      + tags                                           = {
```

● Review the output to ensure there are no errors.

```
  # aws_vpc.test will be created
  + resource "aws_vpc" "test" {
      + arn                                  = (known after apply)
      + cidr_block                           = "10.10.0.0/16"
      + default_network_acl_id               = (known after apply)
      + default_route_table_id               = (known after apply)
      + default_security_group_id            = (known after apply)
      + dhcp_options_id                      = (known after apply)
      + enable_dns_hostnames                 = (known after apply)
      + enable_dns_support                   = true
      + enable_network_address_usage_metrics = (known after apply)
      + id                                   = (known after apply)
      + instance_tenancy                     = "default"
      + ipv6_association_id                  = (known after apply)
      + ipv6_cidr_block                      = (known after apply)
      + ipv6_cidr_block_network_border_group = (known after apply)
      + main_route_table_id                  = (known after apply)
      + owner_id                             = (known after apply)
      + region                               = "us-east-1"
      + tags                                 = {
          + "Name" = "VPC1"
        }
      + tags_all                             = {
          + "Name" = "VPC1"
        }
    }

Plan: 2 to add, 0 to change, 0 to destroy.


Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
~ $
```

■ Apply the Configur ation: Run the apply command to build the infrastructure.

```
terraform apply
```

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value:
```

 Terraform will show you the plan again. Type yes and press Enter to approve thedeployment.

```
  Enter a value: yes

aws_vpc.test: Creating...
aws_vpc.test: Creation complete after 2s [id=vpc-07a85486383135f16]
aws_subnet.public: Creating...
aws_subnet.public: Still creating... [00m10s elapsed]
aws_subnet.public: Creation complete after 11s [id=subnet-0bc582de6326bec54]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
~ $
```

 Wait for 3-5 minutes as Terraform provisions all the resources. When it's finished, it will display the "Outputs".

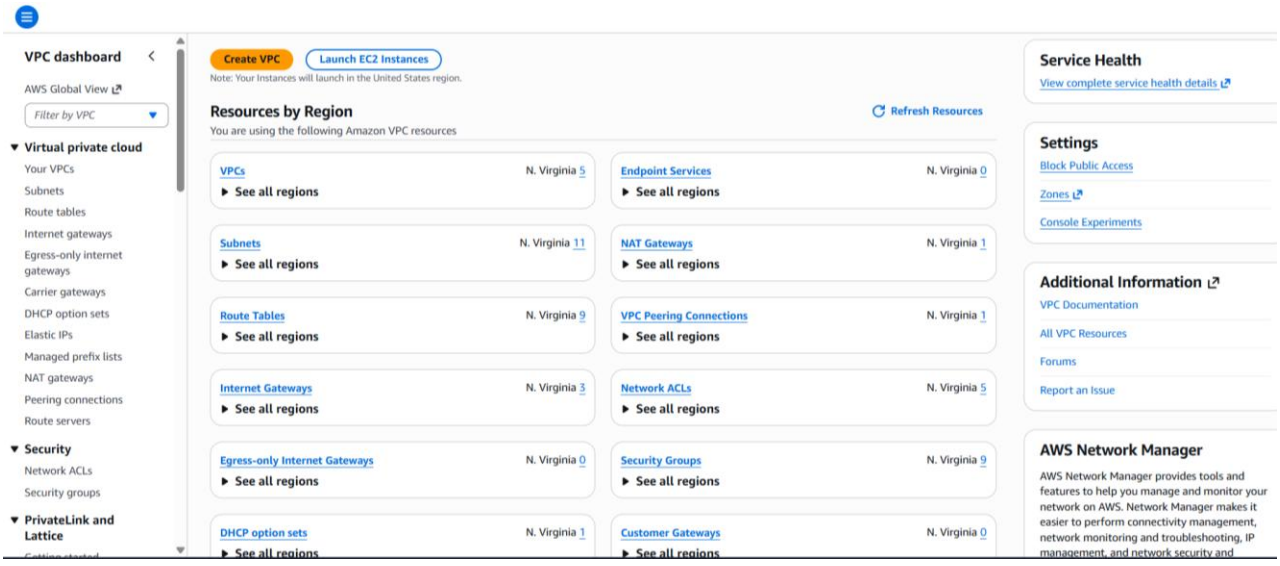

```
  Enter a value: yes

aws_vpc.test: Creating...
aws_vpc.test: Creation complete after 2s [id=vpc-07a85486383135f16]
aws_subnet.public: Creating...
aws_subnet.public: Still creating... [00m10s elapsed]
aws_subnet.public: Creation complete after 11s [id=subnet-0bc582de6326bec54]

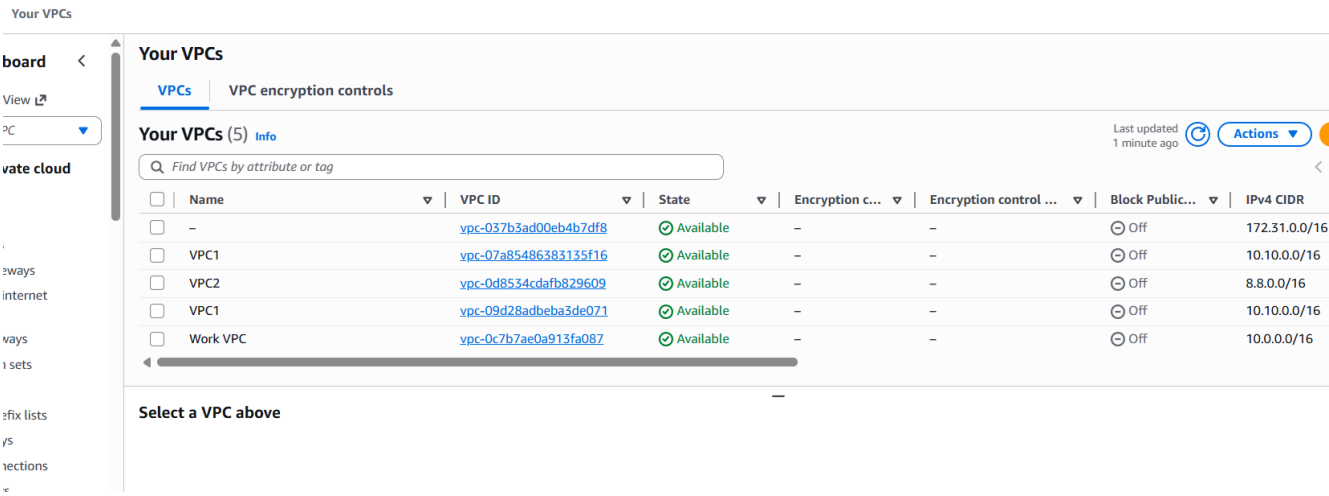
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
~ $
```


■ Verify in Console:

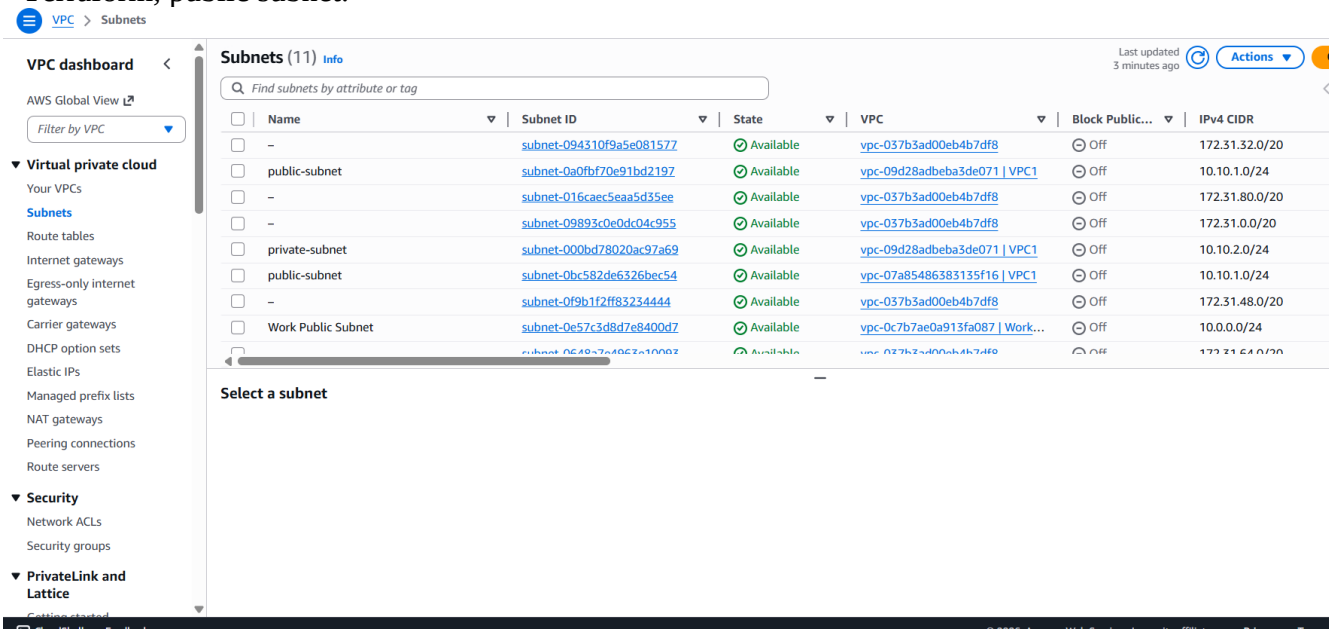
● Go to the AWS Management Console.



● Navigate to the VPC services. You will see all the new resources, such as VPC1



- Terraform, public subnet.



■ Destroy Resour ces (CRITICAL):

- To avoid incurring costs, you must destroy all resources when you are finished.
- In your terminal, run the destroy command:
  - `terraform destroy`

```
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symb
  - destroy

Terraform will perform the following actions:

  # aws_subnet.public will be destroyed
  - resource "aws_subnet" "public" {
      - arn                                            = "arn:aws:ec2:us-east-1:135861183056:subnet/subnet-0bc582de6326bec54" -> null
      - assign_ipv6_address_on_creation                = false -> null
      - availability_zone                              = "us-east-1a" -> null
      - availability_zone_id                           = "use1-az6" -> null
      - cidr_block                                     = "10.10.1.0/24" -> null
      - enable_dns64                                   = false -> null
      - enable_lni_at_device_index                     = 0 -> null
      - enable_resource_name_dns_a_record_on_launch    = false -> null
      - enable_resource_name_dns_aaaa_record_on_launch = false -> null
      - id                                             = "subnet-0bc582de6326bec54" -> null
      - ipv6_native                                    = false -> null
      - map_customer_owned_ip_on_launch                = false -> null
      - map_public_ip_on_launch                        = true -> null
      - owner_id                                       = "135861183056" -> null

CloudShell  Feedback
```

```
      - assign_generated_ipv6_cidr_block     = false -> null
      - cidr_block                           = "10.10.0.0/16" -> null
      - default_network_acl_id               = "acl-0e6f608c3bb07a4a2" -> null
      - default_route_table_id               = "rtb-075849f00ce32c323" -> null
      - default_security_group_id            = "sg-09208a0ca13f8c03d" -> null
      - dhcp_options_id                      = "dopt-05bf03f368d070c39" -> null
      - enable_dns_hostnames                 = false -> null
      - enable_dns_support                   = true -> null
      - enable_network_address_usage_metrics = false -> null
      - id                                   = "vpc-07a85486383135f16" -> null
      - instance_tenancy                     = "default" -> null
      - ipv6_netmask_length                  = 0 -> null
      - main_route_table_id                  = "rtb-075849f00ce32c323" -> null
      - owner_id                             = "135861183056" -> null
      - region                               = "us-east-1" -> null
      - tags                                 = {
          - "Name" = "VPC1"
        } -> null
      - tags_all                             = {
          - "Name" = "VPC1"
        } -> null
        # (4 unchanged attributes hidden)
    }

Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value:
```

◆ Type yes and press Enter to confirm. Terraform will now tear down and delete all theresources it created.

```
        # (4 unchanged attributes hidden)
    }

Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes
```

CloudShell Feedback

```
  There is no undo. Only 'yes' will be accepted to confirm.

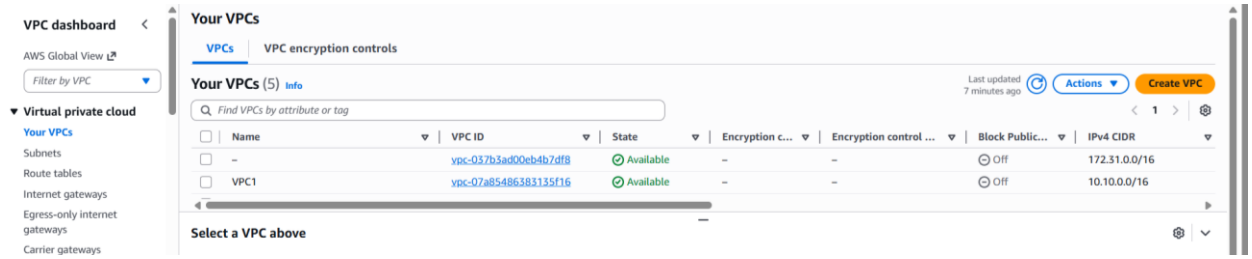
  Enter a value: yes

aws_subnet.public: Destroying... [id=subnet-0bc582de6326bec54]
aws_subnet.public: Destruction complete after 0s
aws_vpc.test: Destroying... [id=vpc-07a85486383135f16]
aws_vpc.test: Destruction complete after 1s

Destroy complete! Resources: 2 destroyed.
~ $
```

CloudShell Feedback

- After that you can verify in the VPC services by clicking refresh button. You will created VPC and subnet is deleted.



6.9. Task 9: Reflestion Question

What are the advantages of using Terraform to automate networking infrastructure in Amazon Web Services instead of configuring resources manually through the AWS Console?

Using Terraform to automate AWS networking infrastructure offers advantages over manual configuration through the AWS Management Console. While the console works for small, one-time setups, it becomes slow, error-prone, and hard to maintain as infrastructure grows. Terraform solves these problems by treating infrastructure as code (IaC), bringing the same benefits of software engineering version control, repeatability, and automation into cloud operations.

1. Speed and Automation

With Terraform, an entire network architecture VPC, subnets, route tables, internet and NAT gateways,

security groups, and EC2 instances can be provisioned in seconds using a single terraform apply command.

2. Consistency and Reduced Human Error

The AWS Console relies on the user to remember every step. Forgetting to associate a route table with a subnet, mistyping a CIDR block, or skipping an inbound rule can break the entire network. Terraform eliminates these mistakes because the configuration is defined once in code and applied exactly the same way every time.

3. Infrastructure as Code (IaC) and Version Control

Because the infrastructure lives in .tf files, it can be stored in Git, reviewed through pull requests, and rolled back if something goes wrong exactly like application code.

4. Reusability and Scalability

Once a solid VPC architecture is written in Terraform, the same code can spin up identical environments in any AWS region simply by changing a few variables.

5. Easy Modification and Dependency-Aware Teardown

Deleting a VPC manually is painful, must delete in a strict order, or AWS will block the deletion. With Terraform, a terraform destroy command automatically calculates the dependency

6. State Management

Terraform maintains a state file (terraform.tfstate) that records every resource it manages along with its real-world ID. This allows Terraform to detect drift, plan precise updates, and only modify what has actually changed.

7. Improved Team Collaboration

Teams can share Terraform files through a Git repository, ensuring everyone deploys the same infrastructure.

Conclusion

The AWS Console is useful for learning and experimenting, but it does not scale. Terraform brings automation, consistency, repeatability, and safe teardown to AWS

Dr. Rohit Kumar Kasera
Lecturer